

LLM 部署与推理优化

面试 100 题

算法岗 · AI Infra · Inference / Serving / Systems

100

核心问题

10

能力章节

2026

技术版

定位

从“会背框架”走向“能估算、能诊断、能做系统取舍”。

参考《剑指 Offer》的问题驱动学习方式：题目 → 思路 → 深挖 → 陷阱 → 追问；不复制原书文本或版式。

Edition 2026.09 · 技术内容以公开论文与主流框架文档为参考

前言 | 为什么需要一本“推理优化版剑指 Offer”

LLM 推理面试的难点不在知识点数量，而在跨层推理：同一个“模型很慢”问题，可能来自模型结构、KV Cache、GPU 内存带宽、CUDA kernel、NCCL、调度器、请求长度分布或 SLO。面试官真正想看到的是你能否建立成本模型，并用证据逐层定位。

因此本书不把 100 题做成“名词解释”。每题固定提供 30 秒回答、关键公式、Know-Why、工程场景、常见错误、追问链和评分标准。建议先尝试口头回答，再阅读深挖；第二遍只做公式和估算；第三遍按追问链模拟面试。

“真题”在这里指真题级问题：来自真实工业系统与公开技术体系中反复出现的核心考点。对于无法公开核验的公司匿名面经，不标注“某公司原题”。这样可以避免把传闻当事实。

全书主线 为什么慢 → 慢在哪里 → 这个优化为什么有效 → 它牺牲什么 → 哪些 workload 下无效 → 如何用实验验证。

三种阅读方式

- 一面速刷：只看每题“30 秒回答 + 常见错误”，目标是 45-60 秒说清。
- 二面深挖：手算关键公式，能解释 Roofline、KV、通信、Goodput 与适用边界。
- 系统设计：重点刷 Q30、Q60、Q90、Q91-Q100，把模型、框架和生产 SLO 串起来。

全书知识地图

层级	核心问题	代表题目
Workload	Prefill / Decode 的资源特征	Q1-Q10
Memory	KV、分页、前缀复用、长上下文	Q11-Q20
Scheduler	Batch、Chunk、Goodput、P/D	Q21-Q30
Kernel	Attention、GEMM、Fusion、Graph	Q31-Q40
Precision	INT4/8、FP8/4、KV Quant	Q41-Q50
Distributed	TP/PP/DP/EP/CP、NCCL	Q51-Q60
Decode Accel	Spec Decode、EAGLE、MTP	Q61-Q70
Model-System	MoE、MLA、Expert traffic	Q71-Q80
Runtime	vLLM / SGLang / TRT-LLM	Q81-Q90
Production	Benchmark、SLO、Cost、Debug	Q91-Q100

目录

01 推理性能基本原理

Q001 - Q010 · 从 Prefill/Decode、Roofline 到延迟/吞吐模型，建立所有优化问题的共同语言。

02 KV Cache 与 Attention

Q011 - Q020 · 掌握 KV 显存模型、PagedAttention、Prefix Cache、GQA/MLA 与长上下文内存问题。

03 Batching 与 Scheduling

Q021 - Q030 · 理解 Continuous Batching、Chunked Prefill、抢占、公平性、Goodput 与 P/D 分离。

04 CUDA、Attention Kernel 与 Runtime

Q031 - Q040 · 从 IO-aware Attention、Kernel Fusion 到 CUDA Graph，用 profiler 定位真正瓶颈。

05 量化与低精度推理

Q041 - Q050 · 比较 GPTQ、AWQ、SmoothQuant、FP8/FP4、KV Quantization 及其硬件约束。

06 分布式推理与通信

Q051 - Q060 · 掌握 TP/PP/DP/EP/CP，理解 NVLink/IB 拓扑与 collective 的成本模型。

07 Speculative Decoding

Q061 - Q070 · 理解 draft-verify、acceptance、Medusa/EAGLE/MTP，以及何时加速、何时负优化。

08 MoE、MLA 与模型-系统协同

Q071 - Q080 · 分析 Expert Parallel、All-to-All、负载不均衡、MLA 与现代 MoE serving。

09 vLLM / SGLang / TensorRT-LLM

Q081 - Q090 · 从框架内部抽象到技术选型，理解缓存、调度、kernel、量化与 P/D 能力。

10 Benchmark、生产部署与系统设计

Q091 - Q100 · 把算法优化落到 SLO、Goodput、成本、监控、故障定位与容量规划。

面试速查 | 8 个必须会现场写的公式

1. **E2E 延迟** $E2E \approx TTFT + (N_{out} - 1) \times TPOT$

2. **Roofline** $Performance \leq \min(\text{Peak FLOPS}, \text{Arithmetic Intensity} \times \text{Memory Bandwidth})$

3. **KV Cache** $M_{KV} = B \times T \times L \times 2 \times H_{kv} \times D_{head} \times \text{bytes}$

4. **Decode 带宽上界** $\text{tokens/s} \lesssim \text{effective HBM bandwidth} / \text{bytes-read-per-token}$

5. **通信成本** $T_{comm} \approx \alpha + \text{bytes} / \text{effective bandwidth}$

6. **Spec Decode** speedup 由 $\text{accepted tokens} / (\text{draft cost} + \text{verify cost})$ 决定

7. **Goodput** $\text{Goodput} = \text{requests meeting SLO} / \text{second}$

8. **成本** $\$/M \text{ output token} = \text{cluster } \$/h \div \text{output tokens/h} \times 10^6$

CHAPTER 01

推理性能基本原理

从 Prefill/Decode、Roofline 到延迟/吞吐模型，建立所有优化问题的共同语言。

本章题目 Q001 - Q010 · 共 10 题

本章学习目标

- 能从 Prefill/Decode 与 Roofline 解释“为什么慢”。
- 会做 token/s、显存与成本的一阶估算。
- 建立指标、SLO 与优化优先级。

建议刷题方法：先用 30 秒回答自测，再遮住正文推导公式，最后只看“追问链”进行模拟面试。

Q001 为什么 LLM 的 Prefill 和 Decode 是两种完全不同的计算？

难度 ★★★★★ · 章节 推理性能基本原理 · 关键词 prefill,decode,roofline

30 秒回答 Prefill 一次处理整段 prompt，矩阵 M 维较大，GEMM/Attention 更容易提高算术强度，常偏 compute-bound；Decode 每步只新增少量 token，却要反复读取模型权重与历史 KV，低 batch 下常偏 memory-bandwidth-bound；两阶段分别主导 TTFT 与 TPOT，因此不能用同一套优化策略。

01 | 考察本质

- 这道题不是考名词定义，而是看你能否从系统资源出发解释：Prefill 一次处理整段 prompt，矩阵 M 维较大，GEMM/Attention 更容易提高算术强度，常偏 compute-bound；Decode 每步只新增少量 token，却要反复读取模型权重与历史 KV，低 batch 下常偏 memory-bandwidth-bound；两阶段分别主导 TTFT 与 TPOT，因此不能用同一套优化策略。
- 面试中应先给结论，再给成本模型/瓶颈，再给适用边界。只背框架参数通常拿不到高分。

02 | 关键公式 / 成本模型

公式 Prefill: $Q, K, V \in \mathbb{R}^{T \times d}$; Decode: $Q \in \mathbb{R}^{B \times d}$. 低并发 Decode 的下界常近似受 weight bytes / HBM bandwidth 限制。

03 | Know-Why: 为什么成立

- 1. Prefill 一次处理整段 prompt，矩阵 M 维较大，GEMM/Attention 更容易提高算术强度，常偏 compute-bound
- 2. Decode 每步只新增少量 token，却要反复读取模型权重与历史 KV，低 batch 下常偏 memory-bandwidth-bound
- 3. 两阶段分别主导 TTFT 与 TPOT，因此不能用同一套优化策略。
- 把这一结论放进 Roofline / 内存容量 / 调度 / 通信四类模型中检查，确认瓶颈是否真的位于关键路径，而不是只优化了一个非关键算子。

04 | 工程场景

现场推演 32K prompt + 100 输出 token 的服务，优先关心 Prefill 切分与 TTFT；1K prompt + 16K 推理输出则更应优化 Decode、KV 与 speculative decoding。

05 | 常见错误

- ✗ 把 “Prefill compute-bound、Decode memory-bound” 当成绝对结论
- ✗ 高 batch、MoE、长上下文或通信开销都可能改变瓶颈。

06 | 追问链

- → 为什么增大 batch 对 Decode 特别有效？
- → 为什么 Chunked Prefill 能同时改善吞吐与 ITL？
- → 如何用 Roofline 验证你的判断？

07 | 面试官评分标准

- 及格：能给出正确概念和基本方向。
- 良好：能写出成本/显存/通信公式，能解释为什么。
- 优秀：能指出反例、适用边界，并能把问题落到 profiler、SLO 或真实系统配置。

Q002 TTFT、TPOT、ITL、E2E Latency、Throughput、Goodput 分别是什么？

难度 ★★★★★ · 章节 推理性能基本原理 · 关键词 metrics, SLO, goodput

30 秒回答 TTFT 衡量请求到首 token；TPOT/ITL 衡量生成阶段 token 间隔；E2E 是用户完整等待时间；Throughput 是单位时间完成的 token 或请求；Goodput 只统计满足 SLO 的工作量。生产系统通常应围绕 p95/p99 与 Goodput 而非平均吞吐优化。

01 | 考察本质

- 这道题不是考名词定义，而是看你能否从系统资源出发解释：TTFT 衡量请求到首 token；TPOT/ITL 衡量生成阶段 token 间隔；E2E 是用户完整等待时间；Throughput 是单位时间完成的 token 或请求；Goodput 只统计满足 SLO 的工作量。生产系统通常应围绕 p95/p99 与 Goodput 而非平均吞吐优化。
- 面试中应先给结论，再给成本模型/瓶颈，再给适用边界。只背框架参数通常拿不到高分。

02 | 关键公式 / 成本模型

公式 $E2E \approx TTFT + (N_{out} - 1) \times TPOT$ ；Goodput = 满足 TTFT/TPOT SLO 的 request/s。

03 | Know-Why：为什么成立

- 1. TTFT 衡量请求到首 token
- 2. TPOT/ITL 衡量生成阶段 token 间隔
- 3. E2E 是用户完整等待时间
- 4. Throughput 是单位时间完成的 token 或请求
- 5. Goodput 只统计满足 SLO 的工作量。生产系统通常应围绕 p95/p99 与 Goodput 而非平均吞吐优化。
- 把这一结论放进 Roofline / 内存容量 / 调度 / 通信四类模型中检查，确认瓶颈是否真的位于关键路径，而不是只优化了一个非关键算子。

04 | 工程场景

现场推演 A 系统 100 req/s 但仅 60% 满足 SLO，Goodput=60；B 系统 85 req/s 且 95% 满足 SLO，Goodput≈80.75，B 对线上更优。

05 | 常见错误

- **✗** 只报平均 latency 或峰值 tokens/s
- **✗** 忽略排队时间、首 token 与后续 token 的体验差异。

06 | 追问链

- → 为什么 p99 比平均值更重要？
- → RAG 与长 CoT 应分别重点看哪些指标？
- → Streaming 场景里 TPOT 和 E2E 谁更重要？

07 | 面试官评分标准

- 及格：能给出正确概念和基本方向。
- 良好：能写出成本/显存/通信公式，能解释为什么。
- 优秀：能指出反例、适用边界，并能把问题落到 profiler、SLO 或真实系统配置。

Q003 为什么 batch=1 的 Decode 经常是 Memory Bound?

难度 ★★★★★ · 章节 推理性能基本原理 · 关键词 decode, memory-bandwidth

30 秒回答 每生成一个 token 都要执行全模型层。batch=1 时同一份权重只服务一个 token，权重读取复用极低；现代 GPU FLOPS 增长往往快于 HBM 带宽增长，所以算力没有吃满时 HBM 已接近瓶颈。增大 batch 可以摊薄每 token 的权重读取成本。

01 | 考察本质

- 这道题不是考名词定义，而是看你能否从系统资源出发解释：每生成一个 token 都要执行全模型层。batch=1 时同一份权重只服务一个 token，权重读取复用极低；现代 GPU FLOPS 增长往往快于 HBM 带宽增长，所以算力没有吃满时 HBM 已接近瓶颈。增大 batch 可以摊薄每 token 的权重读取成本。
- 面试中应先给结论，再给成本模型/瓶颈，再给适用边界。只背框架参数通常拿不到高分。

02 | 关键公式 / 成本模型

公式 粗略上界 $\text{tokens/s} \leq \text{HBM_BW} / \text{model_weight_bytes}$ ；70B BF16 权重约 140 GB。

03 | Know-Why：为什么成立

- 1. 每生成一个 token 都要执行全模型层。batch=1 时同一份权重只服务一个 token，权重读取复用极低
- 2. 现代 GPU FLOPS 增长往往快于 HBM 带宽增长，所以算力没有吃满时 HBM 已接近瓶颈。增大 batch 可以摊薄每 token 的权重读取成本。
- 把这一结论放进 Roofline / 内存容量 / 调度 / 通信四类模型中检查，确认瓶颈是否真的位于关键路径，而不是只优化了一个非关键算子。

04 | 工程场景

现场推演 若设备有效带宽 3 TB/s，单纯按权重流量得到的理想上限约 21 token/s；实际还要扣除 KV、kernel、同步和通信。

05 | 常见错误

- ✗ 直接用理论 HBM 峰值计算线上速度
- ✗ 忽略权重并非每层都完美流式、cache/quantization/TP 都会改变 bytes。

06 | 追问链

- 为什么量化在 batch=1 往往更有效？
- batch 大到什么程度后可能转为 compute-bound？
- 如何用 Nsight 验证？

07 | 面试官评分标准

- 及格：能给出正确概念和基本方向。
- 良好：能写出成本/显存/通信公式，能解释为什么。
- 优秀：能指出反例、适用边界，并能把问题落到 profiler、SLO 或真实系统配置。

Q004 什么是 Roofline Model? 如何用于分析 LLM inference?

难度 ★★★★★ · 章节 推理性能基本原理 · 关键词 roofline, profiling

30 秒回答 Roofline 用算术强度 $AI = \text{FLOPs}/\text{Bytes}$ 把 kernel 放到“带宽屋顶”或“算力屋顶”下。先估算某算子的 FLOPs 与 HBM/缓存流量，再比较 AI 与设备 machine balance；这能回答应该优化访存、量化还是 Tensor Core/GEMM。

01 | 考察本质

- 这道题不是考名词定义，而是看你能否从系统资源出发解释：Roofline 用算术强度 $AI = \text{FLOPs}/\text{Bytes}$ 把 kernel 放到“带宽屋顶”或“算力屋顶”下。先估算某算子的 FLOPs 与 HBM/缓存流量，再比较 AI 与设备 machine balance；这能回答应该优化访存、量化还是 Tensor Core/GEMM。
- 面试中应先给结论，再给成本模型/瓶颈，再给适用边界。只背框架参数通常拿不到高分。

02 | 关键公式 / 成本模型

公式 $\text{Performance} \leq \min(\text{PeakFLOPS}, AI \times \text{MemoryBandwidth})$ 。临界 $AI^* = \text{PeakFLOPS}/\text{MemoryBandwidth}$ 。

03 | Know-Why: 为什么成立

- 1. Roofline 用算术强度 $AI = \text{FLOPs}/\text{Bytes}$ 把 kernel 放到“带宽屋顶”或“算力屋顶”下。先估算某算子的 FLOPs 与 HBM/缓存流量，再比较 AI 与设备 machine balance
- 2. 这能回答应该优化访存、量化还是 Tensor Core/GEMM。
- 把这一结论放进 Roofline / 内存容量 / 调度 / 通信四类模型中检查，确认瓶颈是否真的位于关键路径，而不是只优化了一个非关键算子。

04 | 工程场景

现场推演 同一 Linear 在 Prefill 的 M 很大时更接近 GEMM，AI 提升；Decode 小 batch 时近似 GEMV/small GEMM，AI 明显下降。

05 | 常见错误

- ✗ 只看 GPU Utilization=100% 就判断 compute-bound
- ✗ utilization 不等于 SM/Tensor/HBM 的有效利用率。

06 | 追问链

- Attention 的 AI 如何随 context 变化？
- 量化为什么会移动 Roofline 工作点？
- 通信如何加入扩展 Roofline？

07 | 面试官评分标准

- 及格：能给出正确概念和基本方向。
- 良好：能写出成本/显存/通信公式，能解释为什么。
- 优秀：能指出反例、适用边界，并能把问题落到 profiler、SLO 或真实系统配置。

Q005 如何估算一个模型 Decode 的理论 token/s?

难度 ★★★★★ · 章节 推理性能基本原理 · 关键词 estimation, bandwidth

30 秒回答 先做一阶 bandwidth model：模型权重字节、KV 读取字节、必要中间流量除以有效 HBM 带宽；再加入 batch 权重复用、TP collective、MoE all-to-all 和 kernel 效率，得到更可信的上界。估算的目的不是预测到个位数，而是识别最值得优化的资源。

01 | 考察本质

- 这道题不是考名词定义，而是看你能否从系统资源出发解释：先做一阶 bandwidth model：模型权重字节、KV 读取字节、必要中间流量除以有效 HBM 带宽；再加入 batch 权重复用、TP collective、MoE all-to-all 和 kernel 效率，得到更可信的上界。估算的目的不是预测到个位数，而是识别最值得优化的资源。
- 面试中应先给结论，再给成本模型/瓶颈，再给适用边界。只背框架参数通常拿不到高分。

02 | 关键公式 / 成本模型

公式 $t_{\text{token}} \approx \max(T_{\text{compute}}, T_{\text{HBM}}, T_{\text{comm}}) + T_{\text{overhead}}$ ；batch=1 时 $T_{\text{HBM}} \approx \text{bytes_read} / \text{effective_BW}$ 。

03 | Know-Why：为什么成立

- 1. 先做一阶 bandwidth model：模型权重字节、KV 读取字节、必要中间流量除以有效 HBM 带宽
- 2. 再加入 batch 权重复用、TP collective、MoE all-to-all 和 kernel 效率，得到更可信的上界。估算的目的不是预测到个位数，而是识别最值得优化的资源。
- 把这一结论放进 Roofline / 内存容量 / 调度 / 通信四类模型中检查，确认瓶颈是否真的位于关键路径，而不是只优化了一个非关键算子。

04 | 工程场景

现场推演 如果将 70B 权重从 BF16 约 140 GB 降到 INT4 约 35 GB，纯权重带宽下界可改善约 4×，但最终速度通常小于 4×。

05 | 常见错误

- ✗ 把参数量×字节当作唯一流量
- ✗ 忽略 KV Cache、embedding/lm_head、量化 scale/dequant、TP 通信。

06 | 追问链

- 如何把 KV 字节加入模型？
- 为什么 W4A16 可能只有 1.5× 而不是 4×？
- 如何用实测有效带宽校正估算？

07 | 面试官评分标准

- 及格：能给出正确概念和基本方向。
- 良好：能写出成本/显存/通信公式，能解释为什么。
- 优秀：能指出反例、适用边界，并能把问题落到 profiler、SLO 或真实系统配置。

Q006 输入长度和输出长度分别怎样影响成本？

难度 ★★★★★ · 章节 推理性能基本原理 · 关键词 sequence-length, cost

30 秒回答 输入长度主要影响 Prefill FLOPs、TTFT 与初始 KV；输出长度决定 Decode 步数、KV 生命周期与占用时间。总 token 相同并不意味着成本相同，因为两阶段算子形状与调度方式完全不同。

01 | 考察本质

- 这道题不是考名词定义，而是看你能否从系统资源出发解释：输入长度主要影响 Prefill FLOPs、TTFT 与初始 KV；输出长度决定 Decode 步数、KV 生命周期与占用时间。总 token 相同并不意味着成本相同，因为两阶段算子形状与调度方式完全不同。
- 面试中应先给结论，再给成本模型/瓶颈，再给适用边界。只背框架参数通常拿不到高分。

02 | 关键公式 / 成本模型

公式 Dense attention Prefill 的注意力部分随 T 增长近似 $O(T^2)$ ，Decode 每步 attention 读取历史 KV 约 $O(T)$ 。

03 | Know-Why：为什么成立

- 1. 输入长度主要影响 Prefill FLOPs、TTFT 与初始 KV
- 2. 输出长度决定 Decode 步数、KV 生命周期与占用时间。总 token 相同并不意味着成本相同，因为两阶段算子形状与调度方式完全不同。
- 把这一结论放进 Roofline / 内存容量 / 调度 / 通信四类模型中检查，确认瓶颈是否真的位于关键路径，而不是只优化了一个非关键算子。

04 | 工程场景

现场推演 32K→20 与 200→32K 总 token 类似，但前者是大 Prefill，后者是超长 Decode，会产生完全不同的 GPU/缓存/排队行为。

05 | 常见错误

- ✗ 只按“总 tokens”计费或做容量规划
- ✗ 忽略 output token 会长期占住 KV 与 scheduler slot。

06 | 追问链

- 为什么 input token 和 output token 的成本通常不同？
- 长输出会怎样恶化 p99？
- Sliding Window 如何改变公式？

07 | 面试官评分标准

- 及格：能给出正确概念和基本方向。
- 良好：能写出成本/显存/通信公式，能解释为什么。
- 优秀：能指出反例、适用边界，并能把问题落到 profiler、SLO 或真实系统配置。

Q007 为什么 Batch Size 不是越大越好？

难度 ★★★★★ · 章节 推理性能基本原理 · 关键词 batching, SLO

30 秒回答 batch 增大提高权重复用与吞吐，但会增加排队、KV 占用、单 step 工作量与尾延迟。线上最优点由 SLO、到达率、长度分布和显存共同决定，而不是最大可装 batch。

01 | 考察本质

- 这道题不是考名词定义，而是看你能否从系统资源出发解释：batch 增大提高权重复用与吞吐，但会增加排队、KV 占用、单 step 工作量与尾延迟。线上最优点由 SLO、到达率、长度分布和显存共同决定，而不是最大可装 batch。
- 面试中应先给结论，再给成本模型/瓶颈，再给适用边界。只背框架参数通常拿不到高分。

02 | 关键公式 / 成本模型

公式 目标常写为 $\max \text{Goodput s.t. } p_{99}(\text{TTFT}) < \text{SLO}_1, p_{99}(\text{TPOT}) < \text{SLO}_2, \text{memory} \leq \text{budget}$ 。

03 | Know-Why：为什么成立

- 1. batch 增大提高权重复用与吞吐，但会增加排队、KV 占用、单 step 工作量与尾延迟。线上最优点由 SLO、到达率、长度分布和显存共同决定，而不是最大可装 batch。
- 把这一结论放进 Roofline / 内存容量 / 调度 / 通信四类模型中检查，确认瓶颈是否真的位于关键路径，而不是只优化了一个非关键算子。

04 | 工程场景

现场推演 离线生成可用更大 batch 追求 tokens/s；交互式 Chat 需限制 iteration token budget 保护 ITL。

05 | 常见错误

- ✗ 把 OOM 前的最大 batch 当最优
- ✗ 忽略长短请求混批造成 head-of-line blocking。

06 | 追问链

- Continuous batching 如何改变 batch 定义？
- 如何根据 SLO 自动调 batch？
- 为什么 MoE 对 batch 更敏感？

07 | 面试官评分标准

- 及格：能给出正确概念和基本方向。
- 良好：能写出成本/显存/通信公式，能解释为什么。
- 优秀：能指出反例、适用边界，并能把问题落到 profiler、SLO 或真实系统配置。

Q008 参数量和推理成本有什么关系？MoE 为什么不同？

难度 ★★★★★ · 章节 推理性能基本原理 · 关键词 MoE, active-params

30 秒回答 Dense 模型每 token 激活几乎全部参数，因此总参数量与权重带宽、FLOPs 强相关；MoE 总参数决定内存容量，但每 token 只激活少数 expert，计算量由 active params 决定。MoE 又新增 routing、dispatch 与 all-to-all，因此“参数更多但每 token FLOPs 更低”并不等于延迟一定低。

01 | 考察本质

- 这道题不是考名词定义，而是看你能否从系统资源出发解释：Dense 模型每 token 激活几乎全部参数，因此总参数量与权重带宽、FLOPs 强相关；MoE 总参数决定内存容量，但每 token 只激活少数 expert，计算量由 active params 决定。MoE 又新增 routing、dispatch 与 all-to-all，因此“参数更多但每 token FLOPs 更低”并不等于延迟一定低。
- 面试中应先给结论，再给成本模型/瓶颈，再给适用边界。只背框架参数通常拿不到高分。

02 | 关键公式 / 成本模型

公式 Dense: $\text{active} \approx \text{total}$ ；MoE: $\text{active} \ll \text{total}$ ，且 $T \approx \max(\text{expert_compute}, \text{dispatch}/\text{AllToAll})$ 。

03 | Know-Why：为什么成立

- 1. Dense 模型每 token 激活几乎全部参数，因此总参数量与权重带宽、FLOPs 强相关
- 2. MoE 总参数决定内存容量，但每 token 只激活少数 expert，计算量由 active params 决定。MoE 又新增 routing、dispatch 与 all-to-all，因此“参数更多但每 token FLOPs 更低”并不等于延迟一定低。
- 把这一结论放进 Roofline / 内存容量 / 调度 / 通信四类模型中检查，确认瓶颈是否真的位于关键路径，而不是只优化了一个非关键算子。

04 | 工程场景

现场推演 671B 总参数、约 37B active 的 MoE 需要容纳巨大权重池，但单 token 计算量更接近几十 B active 参数模型。

05 | 常见错误

- ✗ 用 active FLOPs 直接预测 MoE latency
- ✗ 忽略专家不均衡和跨节点通信。

06 | 追问链

- → 为什么 EP 比 TP 更自然？
- → 低并发为何让 MoE 变慢？
- → shared expert 的系统意义是什么？

07 | 面试官评分标准

- 及格：能给出正确概念和基本方向。
- 良好：能写出成本/显存/通信公式，能解释为什么。
- 优秀：能指出反例、适用边界，并能把问题落到 profiler、SLO 或真实系统配置。

Q009 推理慢，你如何判断是 Compute、HBM、Network 还是 CPU Bottleneck?

难度 ★★★★★ · 章节 推理性能基本原理 · 关键词 profiling,debugging

30 秒回答 先从 request 指标拆成 queue/TTFT/TPOT，再看 GPU timeline。若 GPU 长空洞，查 CPU 调度/launch；若 HBM 接近上限而 Tensor Core 低，偏 memory-bound；若 NCCL 占据关键路径，查通信与 topology；若 Tensor Core 接近峰值，才优先优化 compute。

01 | 考察本质

- 这道题不是考名词定义，而是看你能否从系统资源出发解释：先从 request 指标拆成 queue/TTFT/TPOT，再看 GPU timeline。若 GPU 长空洞，查 CPU 调度/launch；若 HBM 接近上限而 Tensor Core 低，偏 memory-bound；若 NCCL 占据关键路径，查通信与 topology；若 Tensor Core 接近峰值，才优先优化 compute。
- 面试中应先给结论，再给成本模型/瓶颈，再给适用边界。只背框架参数通常拿不到高分。

02 | 关键公式 / 成本模型

公式 诊断路径：Request → Scheduler → GPU Timeline → Kernel → Memory → NCCL/Network → CPU。

03 | Know-Why：为什么成立

- 1. 先从 request 指标拆成 queue/TTFT/TPOT，再看 GPU timeline。若 GPU 长空洞，查 CPU 调度/launch
- 2. 若 HBM 接近上限而 Tensor Core 低，偏 memory-bound
- 3. 若 NCCL 占据关键路径，查通信与 topology
- 4. 若 Tensor Core 接近峰值，才优先优化 compute。
- 把这一结论放进 Roofline / 内存容量 / 调度 / 通信四类模型中检查，确认瓶颈是否真的位于关键路径，而不是只优化了一个非关键算子。

04 | 工程场景

现场推演 同样是 TPOT 变慢：可能是 batch 下降导致 HBM 效率下降，也可能是 NCCL straggler；仅看服务级指标无法区分。

05 | 常见错误

- ✗ 一上来替换 kernel
- ✗ 只看 nvidia-smi GPU Util
- ✗ 不做 fixed-shape controlled benchmark。

06 | 追问链

- → Nsight Systems 与 Nsight Compute 分别回答什么？
- → 如何识别 CPU launch gap？
- → 如何定位某个 rank 的 straggler？

07 | 面试官评分标准

- 及格：能给出正确概念和基本方向。
- 良好：能写出成本/显存/通信公式，能解释为什么。

- 优秀：能指出反例、适用边界，并能把问题落到 profiler、SLO 或真实系统配置。

Q010 给你一个推理服务，正确的优化顺序是什么？

难度 ★★★★★ · 章节 推理性能基本原理 · 关键词 methodology, benchmark

30 秒回答 先定义 workload 与 SLO，再建立可复现 baseline，按 Prefill/Decode 与 request/scheduler/kernel/hardware 分层定位瓶颈，最后只对瓶颈施加优化并做 A/B 回归。优化必须同时检查质量、p99、吞吐、显存与成本。

01 | 考察本质

- 这道题不是考名词定义，而是看你能否从系统资源出发解释：先定义 workload 与 SLO，再建立可复现 baseline，按 Prefill/Decode 与 request/scheduler/kernel/hardware 分层定位瓶颈，最后只对瓶颈施加优化并做 A/B 回归。优化必须同时检查质量、p99、吞吐、显存与成本。
- 面试中应先给结论，再给成本模型/瓶颈，再给适用边界。只背框架参数通常拿不到高分。

02 | 关键公式 / 成本模型

公式 Measure → Model → Change one variable → Re-measure。

03 | Know-Why：为什么成立

- 1. 先定义 workload 与 SLO，再建立可复现 baseline，按 Prefill/Decode 与 request/scheduler/kernel/hardware 分层定位瓶颈，最后只对瓶颈施加优化并做 A/B 回归。优化必须同时检查质量、p99、吞吐、显存与成本。
- 把这一结论放进 Roofline / 内存容量 / 调度 / 通信四类模型中检查，确认瓶颈是否真的位于关键路径，而不是只优化了一个非关键算子。

04 | 工程场景

现场推演 不要在 512→512 synthetic benchmark 上得出“线上提速 2×”；应回放真实长度和到达分布。

05 | 常见错误

- ✗ 先开所有优化开关
- ✗ 一次改变多个参数导致无法归因
- ✗ 只看平均值。

06 | 追问链

- → 如何设计 benchmark matrix?
- → 哪些优化需要质量回归?
- → 如何判断优化已经到收益递减点?

07 | 面试官评分标准

- 及格：能给出正确概念和基本方向。
- 良好：能写出成本/显存/通信公式，能解释为什么。
- 优秀：能指出反例、适用边界，并能把问题落到 profiler、SLO 或真实系统配置。

CHAPTER 02

KV Cache 与 Attention

掌握 KV 显存模型、PagedAttention、Prefix Cache、GQA/MLA 与长上下文内存问题。

本章题目 Q011 - Q020 · 共 10 题

本章学习目标

- 能现场计算 KV Cache。
- 能解释 PagedAttention/Prefix Cache 的内存管理本质。
- 理解 GQA/MLA/长上下文的系统价值。

建议刷题方法：先用 30 秒回答自测，再遮住正文推导公式，最后只看“追问链”进行模拟面试。

Q011 KV Cache 显存怎么计算？

难度 ★★★★★ · 章节 KV Cache 与 Attention · 关键词 KV-cache,memory

30 秒回答 每层每 token 需要保存 K 与 V。对 GQA/MQA，KV head 数而不是 Q head 数决定 cache 大小；还要乘层数、head_dim、dtype、batch 与上下文长度。部署前必须用这个公式估算最大并发。

01 | 考察本质

- 这道题不是考名词定义，而是看你能否从系统资源出发解释：每层每 token 需要保存 K 与 V。对 GQA/MQA，KV head 数而不是 Q head 数决定 cache 大小；还要乘层数、head_dim、dtype、batch 与上下文长度。部署前必须用这个公式估算最大并发。
- 面试中应先给结论，再给成本模型/瓶颈，再给适用边界。只背框架参数通常拿不到高分。

02 | 关键公式 / 成本模型

公式 $M_{KV} = B \times T \times L \times 2(K,V) \times H_{kv} \times D_{head} \times \text{bytes_per_elem}$ 。

03 | Know-Why：为什么成立

- 1. 每层每 token 需要保存 K 与 V。对 GQA/MQA，KV head 数而不是 Q head 数决定 cache 大小
- 2. 还要乘层数、head_dim、dtype、batch 与上下文长度。部署前必须用这个公式估算最大并发。
- 把这一结论放进 Roofline / 内存容量 / 调度 / 通信四类模型中检查，确认瓶颈是否真的位于关键路径，而不是只优化了一个非关键算子。

04 | 工程场景

现场推演 同一 32K context，GQA 将 H_{kv} 从 32 降到 8，可把 KV 主体约降到 1/4。

05 | 常见错误

- ✗ 用 hidden_size 直接代入而忽略 GQA
- ✗ 忘记 K/V 的因子 2
- ✗ 未给 block metadata/workspace 留余量。

06 | 追问链

- → 给定模型结构现场算 100 并发显存。
- → FP8 KV 能省多少？
- → MLA 的公式如何变化？

07 | 面试官评分标准

- 及格：能给出正确概念和基本方向。
- 良好：能写出成本/显存/通信公式，能解释为什么。
- 优秀：能指出反例、适用边界，并能把问题落到 profiler、SLO 或真实系统配置。

Q012 MHA、MQA、GQA 为什么会极大影响推理成本？

难度 ★★★★★ · 章节 KV Cache 与 Attention · 关键词 MHA,GQA,MQA

30 秒回答 三者主要差在多个 query head 是否共享 K/V。MQA 只有一组 KV，GQA 用若干组 KV，显著降低 KV 容量和 Decode 读取带宽；代价是模型表示能力与 kernel 实现需要折中。

01 | 考察本质

- 这道题不是考名词定义，而是看你能否从系统资源出发解释：三者主要差在多个 query head 是否共享 K/V。MQA 只有一组 KV，GQA 用若干组 KV，显著降低 KV 容量和 Decode 读取带宽；代价是模型表示能力与 kernel 实现需要折中。
- 面试中应先给结论，再给成本模型/瓶颈，再给适用边界。只背框架参数通常拿不到高分。

02 | 关键公式 / 成本模型

公式 $KV \text{ memory} \propto H_{kv}$ ；MHA $H_{kv}=H_q$ ，MQA $H_{kv}=1$ ，GQA 介于两者。

03 | Know-Why：为什么成立

- 1. 三者主要差在多个 query head 是否共享 K/V。MQA 只有一组 KV，GQA 用若干组 KV，显著降低 KV 容量和 Decode 读取带宽
- 2. 代价是模型表示能力与 kernel 实现需要折中。
- 把这一结论放进 Roofline / 内存容量 / 调度 / 通信四类模型中检查，确认瓶颈是否真的位于关键路径，而不是只优化了一个非关键算子。

04 | 工程场景

现场推演 对长上下文和大并发，KV 往往先于权重成为容量瓶颈，因此 GQA 的服务收益可能大于其训练侧直觉。

05 | 常见错误

- ✗ 只说“GQA 更快”却解释不了为什么
- ✗ 忽略不同 attention backend 对 GQA shape 的优化差异。

06 | 追问链

- → 为什么 Llama 系模型广泛使用 GQA？
- → MQA 会带来什么质量风险？
- → GQA 与 TP 如何交互？

07 | 面试官评分标准

- 及格：能给出正确概念和基本方向。
- 良好：能写出成本/显存/通信公式，能解释为什么。
- 优秀：能指出反例、适用边界，并能把问题落到 profiler、SLO 或真实系统配置。

Q013 PagedAttention 到底解决了什么？

难度 ★★★★★ · 章节 KV Cache 与 Attention · 关键词 PagedAttention, KV

30 秒回答 它主要解决动态 KV Cache 的内存管理，而非改变 attention 数学。把逻辑序列映射到固定大小的物理 block，避免要求连续显存，减少 external fragmentation，并支持更自然的共享与 copy-on-write。

01 | 考察本质

- 这道题不是考名词定义，而是看你能否从系统资源出发解释：它主要解决动态 KV Cache 的内存管理，而非改变 attention 数学。把逻辑序列映射到固定大小的物理 block，避免要求连续显存，减少 external fragmentation，并支持更自然的共享与 copy-on-write。
- 面试中应先给结论，再给成本模型/瓶颈，再给适用边界。只背框架参数通常拿不到高分。

02 | 关键公式 / 成本模型

公式 Logical block table → physical KV blocks；请求长度增长时按需分配 block。

03 | Know-Why：为什么成立

- 1. 它主要解决动态 KV Cache 的内存管理，而非改变 attention 数学。把逻辑序列映射到固定大小的物理 block，避免要求连续显存，减少 external fragmentation，并支持更自然的共享与 copy-on-write。
- 把这一结论放进 Roofline / 内存容量 / 调度 / 通信四类模型中检查，确认瓶颈是否真的位于关键路径，而不是只优化了一个非关键算子。

04 | 工程场景

现场推演 Beam search 或共享 system prompt 可让多个序列引用相同物理块，在分叉时才复制写入。

05 | 常见错误

- ✗ 把 PagedAttention 解释成 “更快的 softmax”
- ✗ 忽略 page table/last-block internal fragmentation 的代价。

06 | 追问链

- → 它和 OS virtual memory 的类比哪里成立、哪里不成立？
- → block size 怎么选？
- → 为什么它让 continuous batching 更可行？

07 | 面试官评分标准

- 及格：能给出正确概念和基本方向。
- 良好：能写出成本/显存/通信公式，能解释为什么。
- 优秀：能指出反例、适用边界，并能把问题落到 profiler、SLO 或真实系统配置。

Q014 KV block/page size 为什么不能无限小?

难度 ★★★★★ · 章节 KV Cache 与 Attention · 关键词 KV,page-size

30 秒回答 小 page 减少尾块浪费、提高 prefix 匹配粒度，但会增加 block table、metadata、寻址与 kernel 调度开销，并可能降低连续访存效率。大 page 相反。最优值依赖 context 分布、prefix reuse 与 backend kernel。

01 | 考察本质

- 这道题不是考名词定义，而是看你能否从系统资源出发解释：小 page 减少尾块浪费、提高 prefix 匹配粒度，但会增加 block table、metadata、寻址与 kernel 调度开销，并可能降低连续访存效率。大 page 相反。最优值依赖 context 分布、prefix reuse 与 backend kernel。
- 面试中应先给结论，再给成本模型/瓶颈，再给适用边界。只背框架参数通常拿不到高分。

02 | 关键公式 / 成本模型

公式 $Waste \approx (page_size - last_page_used) / page_size$; $metadata \approx O(num_pages)$ 。

03 | Know-Why: 为什么成立

- 1. 小 page 减少尾块浪费、提高 prefix 匹配粒度，但会增加 block table、metadata、寻址与 kernel 调度开销，并可能降低连续访存效率。大 page 相反。最优值依赖 context 分布、prefix reuse 与 backend kernel。
- 把这一结论放进 Roofline / 内存容量 / 调度 / 通信四类模型中检查，确认瓶颈是否真的位于关键路径，而不是只优化了一个非关键算子。

04 | 工程场景

现场推演 短请求占多数时大 page 浪费明显；超长上下文中 page 太小又会让 page table 和 gather 变重。

05 | 常见错误

- ✗ 只根据显存碎片选 page size
- ✗ 不进行 end-to-end TTFT/TPOT 测试。

06 | 追问链

- → Prefix cache 命中粒度如何受 page size 影响?
- → 不同 head_dim 会改变最优值吗?
- → 如何做 sweep?

07 | 面试官评分标准

- 及格：能给出正确概念和基本方向。
- 良好：能写出成本/显存/通信公式，能解释为什么。
- 优秀：能指出反例、适用边界，并能把问题落到 profiler、SLO 或真实系统配置。

Q015 什么叫 KV Cache 的 Internal Fragmentation?

难度 ★★★☆☆ · 章节 KV Cache 与 Attention · 关键词 fragmentation, KV

30 秒回答 分页能解决很多外部碎片，但每条序列最后一个未填满 block 仍会浪费空间，这就是 internal fragmentation。请求很短或 page 很大时比例尤其显著。

01 | 考察本质

- 这道题不是考名词定义，而是看你能否从系统资源出发解释：分页能解决很多外部碎片，但每条序列最后一个未填满 block 仍会浪费空间，这就是 internal fragmentation。请求很短或 page 很大时比例尤其显著。
- 面试中应先给结论，再给成本模型/瓶颈，再给适用边界。只背框架参数通常拿不到高分。

02 | 关键公式 / 成本模型

公式 若 $\text{page}=16$ token，最后只用 3 token，则该页 13/16 空间未使用。

03 | Know-Why：为什么成立

- 1. 分页能解决很多外部碎片，但每条序列最后一个未填满 block 仍会浪费空间，这就是 internal fragmentation。请求很短或 page 很大时比例尤其显著。
- 把这一结论放进 Roofline / 内存容量 / 调度 / 通信四类模型中检查，确认瓶颈是否真的位于关键路径，而不是只优化了一个非关键算子。

04 | 工程场景

现场推演 1000 个并发短请求即使每个只浪费半页，累计显存也可能相当可观。

05 | 常见错误

- ✗ 以为 PagedAttention 完全消除碎片
- ✗ 忽略 block table 本身也有成本。

06 | 追问链

- → 如何估算期望浪费？
- → 变长 page 是否可行？
- → 为什么固定 page 更利于 kernel？

07 | 面试官评分标准

- 及格：能给出正确概念和基本方向。
- 良好：能写出成本/显存/通信公式，能解释为什么。
- 优秀：能指出反例、适用边界，并能把问题落到 profiler、SLO 或真实系统配置。

Q016 Prefix Caching 为什么有时收益巨大，有时几乎没用？

难度 ★★★★★ · 章节 KV Cache 与 Attention · 关键词 prefix-cache, cache

30 秒回答 收益由共享前缀长度、命中率、前缀计算成本和 cache 占用共同决定。多轮 Chat、固定 system prompt、RAG 公

共文档、agent 模板往往可显著复用；完全随机 prompt 则几乎无收益。

01 | 考察本质

- 这道题不是考名词定义，而是看你能否从系统资源出发解释：收益由共享前缀长度、命中率、前缀计算成本和 cache 占用共同决定。多轮 Chat、固定 system prompt、RAG 公共文档、agent 模板往往可显著复用；完全随机 prompt 则几乎无收益。
- 面试中应先给结论，再给成本模型/瓶颈，再给适用边界。只背框架参数通常拿不到高分。

02 | 关键公式 / 成本模型

公式 $\text{Saved compute} \approx \text{hit_rate} \times \text{reused_prefix_tokens} \times \text{prefill_cost_per_token}$ 。

03 | Know-Why：为什么成立

- 1. 收益由共享前缀长度、命中率、前缀计算成本和 cache 占用共同决定。多轮 Chat、固定 system prompt、RAG 公共文档、agent 模板往往可显著复用
- 2. 完全随机 prompt 则几乎无收益。
- 把这一结论放进 Roofline / 内存容量 / 调度 / 通信四类模型中检查，确认瓶颈是否真的位于关键路径，而不是只优化了一个非关键算子。

04 | 工程场景

现场推演 同一 20K 文档被 100 次问答重复使用时，缓存其 KV 可以避免大量重复 Prefill；若每个文档都唯一则没有这种收益。

05 | 常见错误

- **✗** 只看 cache hit request 数，不看复用 token 数
- **✗** cache 过大挤压可服务的 active KV。

06 | 追问链

- → Hash collision 怎么处理？
- → 多租户下 cache 是否安全？
- → Cache eviction 策略怎么选？

07 | 面试官评分标准

- 及格：能给出正确概念和基本方向。
- 良好：能写出成本/显存/通信公式，能解释为什么。
- 优秀：能指出反例、适用边界，并能把问题落到 profiler、SLO 或真实系统配置。

Q017 RadixAttention 与普通 Prefix Cache 有什么不同？

难度 ★★★★★ · 章节 KV Cache 与 Attention · 关键词 SGLang, RadixAttention

30 秒回答 RadixAttention 用 radix tree 保存可共享的 token prefix，从而支持最长前缀匹配、分支复用与 cache-aware scheduling。它不仅是缓存结构，还把“哪些请求共享 prefix”暴露给调度器。

01 | 考察本质

- 这道题不是考名词定义，而是看你能否从系统资源出发解释：RadixAttention 用 radix tree 保存可共享的 token prefix，从而支持最长前缀匹配、分支复用与 cache-aware scheduling。它不仅是缓存结构，还把“哪些请求共享 prefix”暴露给调度器。
- 面试中应先给结论，再给成本模型/瓶颈，再给适用边界。只背框架参数通常拿不到高分。

02 | 关键公式 / 成本模型

公式 公共 token 序列形成树的共享边；请求的 longest-prefix match 决定可复用 KV。

03 | Know-Why：为什么成立

- 1. RadixAttention 用 radix tree 保存可共享的 token prefix，从而支持最长前缀匹配、分支复用与 cache-aware scheduling。它不仅是缓存结构，还把“哪些请求共享 prefix”暴露给调度器。
- 把这一结论放进 Roofline / 内存容量 / 调度 / 通信四类模型中检查，确认瓶颈是否真的位于关键路径，而不是只优化了一个非关键算子。

04 | 工程场景

现场推演 ABCDEF、ABCGHI、ABCXYZ 共享 ABC；若调度器连续执行相近前缀请求，可以提高缓存局部性。

05 | 常见错误

- ✗ 把它等同于字符串 trie 而忽略 KV 引用、eviction 与调度
- ✗ 忽略 tokenization 后才是匹配单位。

06 | 追问链

- → 为什么 cache-aware scheduling 可能伤害公平性？
- → 多模态 prefix 怎么处理？
- → 与 hash-based prefix caching 比较。

07 | 面试官评分标准

- 及格：能给出正确概念和基本方向。
- 良好：能写出成本/显存/通信公式，能解释为什么。
- 优秀：能指出反例、适用边界，并能把问题落到 profiler、SLO 或真实系统配置。

Q018 为什么 KV Cache 也值得量化？

难度 ★★★★★ · 章节 KV Cache 与 Attention · 关键词 KV-quant, FP8

30 秒回答 长上下文与大并发下 KV 容量按 $B \times T$ 线性增长，而且 Decode 每步都要读历史 KV。将 BF16 KV 降为 FP8 理论上可约减半容量和流量，从而增加并发或上下文；但需要 runtime quant/dequant 与 attention kernel 原生支持。

01 | 考察本质

- 这道题不是考名词定义，而是看你能否从系统资源出发解释：长上下文与大并发下 KV 容量按 $B \times T$ 线性增长，而且 Decode 每步都要读历史 KV。将 BF16 KV 降为 FP8 理论上可约减半容量和流量，从而增加并发或上下文；但需要 runtime quant/dequant 与 attention kernel 原生支持。
- 面试中应先给结论，再给成本模型/瓶颈，再给适用边界。只背框架参数通常拿不到高分。

02 | 关键公式 / 成本模型

公式 $KV \text{ bytes} \propto \text{dtype_bytes}$ ；BF16→FP8 约 2B→1B。

03 | Know-Why：为什么成立

- 1. 长上下文与大并发下 KV 容量按 $B \times T$ 线性增长，而且 Decode 每步都要读历史 KV。将 BF16 KV 降为 FP8 理论上可约减半容量和流量，从而增加并发或上下文
- 2. 但需要 runtime quant/dequant 与 attention kernel 原生支持。
- 把这一结论放进 Roofline / 内存容量 / 调度 / 通信四类模型中检查，确认瓶颈是否真的位于关键路径，而不是只优化了一个非关键算子。

04 | 工程场景

现场推演 模型权重已经 INT4 但 128K context 仍 OOM 时，继续压权重收益有限，KV quantization 可能更直接。

05 | 常见错误

- ✗ 只测短上下文精度
- ✗ 忽略长序列误差积累和 scale granularity。

06 | 追问链

- → FP8 KV 的 scale 何时计算？
- → Per-head vs per-tensor scale？
- → 如何做 long-context 回归？

07 | 面试官评分标准

- 及格：能给出正确概念和基本方向。
- 良好：能写出成本/显存/通信公式，能解释为什么。
- 优秀：能指出反例、适用边界，并能把问题落到 profiler、SLO 或真实系统配置。

Q019 Sliding Window Attention 如何降低推理复杂度？

难度 ★★★★★ · 章节 KV Cache 与 Attention · 关键词 sliding-window, long-context

30 秒回答 只保留或访问最近 W 个 token，使 Decode attention 的历史长度从 T 截断为 W，KV 占用和读取不再随无限历史增长。它属于模型语义/结构改变，不能像 serving 参数一样无损开启。

01 | 考察本质

- 这道题不是考名词定义，而是看你能否从系统资源出发解释：只保留或访问最近 W 个 token，使 Decode attention 的历史长度从 T 截断为 W ，KV 占用和读取不再随无限历史增长。它属于模型语义/结构改变，不能像 serving 参数一样无损开启。
- 面试中应先给结论，再给成本模型/瓶颈，再给适用边界。只背框架参数通常拿不到高分。

02 | 关键公式 / 成本模型

公式 Full decode attention $O(T)$ per step; windowed $O(W)$ ，KV 从 $O(T)$ 变 $O(W)$ 。

03 | Know-Why：为什么成立

- 1. 只保留或访问最近 W 个 token，使 Decode attention 的历史长度从 T 截断为 W ，KV 占用和读取不再随无限历史增长。它属于模型语义/结构改变，不能像 serving 参数一样无损开启。
- 把这一结论放进 Roofline / 内存容量 / 调度 / 通信四类模型中检查，确认瓶颈是否真的位于关键路径，而不是只优化了一个非关键算子。

04 | 工程场景

现场推演 128K 对话若模型层使用 8K sliding window，缓存与读取可大幅下降，但模型对窗口外信息的直接访问受限。

05 | 常见错误

- ✗ 在未训练支持的模型上强行截断 KV
- ✗ 忽略混合 full/sliding layers 的 hybrid cache。

06 | 追问链

- → Mistral 类模型为何采用 sliding window?
- → Hybrid KV manager 怎么设计?
- → 和 retrieval memory 有何区别?

07 | 面试官评分标准

- 及格：能给出正确概念和基本方向。
- 良好：能写出成本/显存/通信公式，能解释为什么。
- 优秀：能指出反例、适用边界，并能把问题落到 profiler、SLO 或真实系统配置。

Q020 MLA 为什么对推理优化特别重要？

难度 ★★★★★ · 章节 KV Cache 与 Attention · 关键词 MLA, DeepSeek

30 秒回答 MLA 将传统 K/V 表示压到低维 latent，再在 attention 中恢复需要的信息，大幅缩小 KV Cache 与 Decode 访存。它体现模型架构与 serving 的协同设计，但需要专用 attention kernel、RoPE 处理和并行策略。

01 | 考察本质

- 这道题不是考名词定义，而是看你能否从系统资源出发解释：MLA 将传统 K/V 表示压到低维 latent，再在 attention 中恢复需要的信息，大幅缩小 KV Cache 与 Decode 访存。它体现模型架构与 serving 的协同设计，但需要专用 attention kernel、RoPE 处理和并行策略。

- 面试中应先给结论，再给成本模型/瓶颈，再给适用边界。只背框架参数通常拿不到高分。

02 | 关键公式 / 成本模型

公式 传统 $KV \propto H_{kv} \times D_{head}$ ；MLA 主要缓存低维 latent c_KV，缓存维度可显著更小。

03 | Know-Why：为什么成立

- 1. MLA 将传统 K/V 表示压到低维 latent，再在 attention 中恢复需要的信息，大幅缩小 KV Cache 与 Decode 访存。它体现模型架构与 serving 的协同设计，但需要专用 attention kernel、RoPE 处理和并行策略。
- 把这一结论放进 Roofline / 内存容量 / 调度 / 通信四类模型中检查，确认瓶颈是否真的位于关键路径，而不是只优化了一个非关键算子。

04 | 工程场景

现场推演 大规模 MoE 模型权重很大，如果 KV 仍按传统 MHA 保存，长上下文服务成本会非常高；MLA 可缓解这一瓶颈。

05 | 常见错误

- **✗** 只把 MLA 当“另一种 GQA”
- **✗** 忽略 latent projection、decoupled RoPE 和 kernel implementation。

06 | 追问链

- → MLA decode 为什么需要专门 kernel？
- → 与 GQA 的缓存公式如何比较？
- → MLA 与 EP/TP 怎么组合？

07 | 面试官评分标准

- 及格：能给出正确概念和基本方向。
- 良好：能写出成本/显存/通信公式，能解释为什么。
- 优秀：能指出反例、适用边界，并能把问题落到 profiler、SLO 或真实系统配置。

CHAPTER 03

Batching 与 Scheduling

理解 Continuous Batching、Chunked Prefill、抢占、公平性、Goodput 与 P/D 分离。

本章题目 Q021 - Q030 · 共 10 题

本章学习目标

- 能解释 continuous batching 与 chunked prefill。
- 能在吞吐、公平性与 tail latency 间做权衡。
- 理解 P/D 分离的收益条件。

建议刷题方法：先用 30 秒回答自测，再遮住正文推导公式，最后只看“追问链”进行模拟面试。

Q021 Static Batching 和 Continuous Batching 有什么区别？

难度 ★★★★★ · 章节 Batching 与 Scheduling · 关键词 continuous-batching,scheduler

30 秒回答 Static batch 在一批请求全部完成前很难插入新请求，短请求会被长请求拖住；Continuous batching 在每个 decoding iteration 动态移除完成请求、加入新请求，提高 GPU 利用率并降低 head-of-line blocking。

01 | 考察本质

- 这道题不是考名词定义，而是看你能否从系统资源出发解释：Static batch 在一批请求全部完成前很难插入新请求，短请求会被长请求拖住；Continuous batching 在每个 decoding iteration 动态移除完成请求、加入新请求，提高 GPU 利用率并降低 head-of-line blocking。
- 面试中应先给结论，再给成本模型/瓶颈，再给适用边界。只背框架参数通常拿不到高分。

02 | 关键公式 / 成本模型

公式 调度粒度从 request/batch 生命周期缩小到 iteration/token step。

03 | Know-Why：为什么成立

- 1. Static batch 在一批请求全部完成前很难插入新请求，短请求会被长请求拖住
- 2. Continuous batching 在每个 decoding iteration 动态移除完成请求、加入新请求，提高 GPU 利用率并降低 head-of-line blocking。
- 把这一结论放进 Roofline / 内存容量 / 调度 / 通信四类模型中检查，确认瓶颈是否真的位于关键路径，而不是只优化了一个非关键算子。

04 | 工程场景

现场推演 一个 10-token 请求和 1000-token 请求同批，静态方式可能让短请求等待整个长请求；continuous batching 可在短请求完成后立刻释放 slot。

05 | 常见错误

- ✗ 把 continuous batching 等同于动态 batch size
- ✗ 核心是 iteration-level admission/retirement。

06 | 追问链

- → 为什么 continuous batching 需要更灵活的 KV allocator？
- → 如何处理不同 sequence length？
- → 对 CUDA Graph 有何挑战？

07 | 面试官评分标准

- 及格：能给出正确概念和基本方向。
- 良好：能写出成本/显存/通信公式，能解释为什么。
- 优秀：能指出反例、适用边界，并能把问题落到 profiler、SLO 或真实系统配置。

Q022 Orca 的 Iteration-level Scheduling 为什么重要?

难度 ★★★★★ · 章节 Batching 与 Scheduling · 关键词 Orca,scheduling

30 秒回答 Orca 把调度单位从完整 request 改成 iteration，允许对正在生成的序列与新请求细粒度重组；同时利用 selective batching 处理不同算子可否批处理的问题。它是现代 LLM serving 调度思想的重要起点。

01 | 考察本质

- 这道题不是考名词定义，而是看你能否从系统资源出发解释：Orca 把调度单位从完整 request 改成 iteration，允许对正在生成的序列与新请求细粒度重组；同时利用 selective batching 处理不同算子可否批处理的问题。它是现代 LLM serving 调度思想的重要起点。
- 面试中应先给结论，再给成本模型/瓶颈，再给适用边界。只背框架参数通常拿不到高分。

02 | 关键公式 / 成本模型

公式 每个 decode iteration 都可重新形成 batch。

03 | Know-Why: 为什么成立

- 1. Orca 把调度单位从完整 request 改成 iteration，允许对正在生成的序列与新请求细粒度重组
- 2. 同时利用 selective batching 处理不同算子可否批处理的问题。它是现代 LLM serving 调度思想的重要起点。
- 把这一结论放进 Roofline / 内存容量 / 调度 / 通信四类模型中检查，确认瓶颈是否真的位于关键路径，而不是只优化了一个非关键算子。

04 | 工程场景

现场推演 请求 A 结束后，下一 iteration 立即让等待队列中的 B 进入，无需等 batch 其余序列结束。

05 | 常见错误

- ✗ 只记论文名不理解“为什么 request-level batching 浪费”。

06 | 追问链

- → Selective batching 是什么？
- → 它和 vLLM continuous batching 有何关系？
- → 迭代调度如何影响公平性？

07 | 面试官评分标准

- 及格：能给出正确概念和基本方向。
- 良好：能写出成本/显存/通信公式，能解释为什么。
- 优秀：能指出反例、适用边界，并能把问题落到 profiler、SLO 或真实系统配置。

Q023 为什么 Prefill 会干扰 Decode?

难度 ★★★★★ · 章节 Batching 与 Scheduling · 关键词 prefill-interference,ITL

30 秒回答 Prefill 通常是大矩阵、计算密集；Decode 每 step 对时延敏感。若一个超长 Prefill 独占 GPU 数十毫秒甚至更久，

已有 streaming 请求无法及时生成下一个 token，ITL/p99 会恶化。

01 | 考察本质

- 这道题不是考名词定义，而是看你能否从系统资源出发解释：Prefill 通常是大矩阵、计算密集；Decode 每 step 对时延敏感。若一个超长 Prefill 独占 GPU 数十毫秒甚至更久，已有 streaming 请求无法及时生成下一个 token，ITL/p99 会恶化。
- 面试中应先给结论，再给成本模型/瓶颈，再给适用边界。只背框架参数通常拿不到高分。

02 | 关键公式 / 成本模型

公式 Generation stall $\approx$ long prefill kernel/iteration 持续时间。

03 | Know-Why：为什么成立

- 1. Prefill 通常是大矩阵、计算密集
- 2. Decode 每 step 对时延敏感。若一个超长 Prefill 独占 GPU 数十毫秒甚至更久，已有 streaming 请求无法及时生成下一个 token，ITL/p99 会恶化。
- 把这一结论放进 Roofline / 内存容量 / 调度 / 通信四类模型中检查，确认瓶颈是否真的位于关键路径，而不是只优化了一个非关键算子。

04 | 工程场景

现场推演 交互服务中突然进入 64K RAG 请求，若整段 Prefill 一次执行，其他用户会感觉“打字突然停顿”。

05 | 常见错误

- ✗ 只看 aggregate tokens/s，忽略 decode stall
- ✗ 把 Prefill 与 Decode 混合视为天然有利。

06 | 追问链

- → 为什么 prefill/decode 资源特征互补又会互相干扰？
- → 如何用 chunking 控制 stall？
- → 何时直接做 P/D 分离？

07 | 面试官评分标准

- 及格：能给出正确概念和基本方向。
- 良好：能写出成本/显存/通信公式，能解释为什么。
- 优秀：能指出反例、适用边界，并能把问题落到 profiler、SLO 或真实系统配置。

Q024 Chunked Prefill 为什么能解决这个问题？

难度 ★★★★★ · 章节 Batching 与 Scheduling · 关键词 chunked-prefill, Sarathi

30 秒回答 将长 prompt 拆成若干 token chunk，每轮只用剩余 token budget 处理一部分 Prefill，并优先保护 Decode。这样限制单个 Prefill 的最长阻塞时间，同时可以把 compute-heavy Prefill 与 memory-heavy Decode 混到同一 iteration 提高利用率。

01 | 考察本质

- 这道题不是考名词定义，而是看你能否从系统资源出发解释：将长 prompt 拆成若干 token chunk，每轮只用剩余 token budget 处理一部分 Prefill，并优先保护 Decode。这样限制单个 Prefill 的最长阻塞时间，同时可以把 compute-heavy Prefill 与 memory-heavy Decode 混到同一 iteration 提高利用率。
- 面试中应先给结论，再给成本模型/瓶颈，再给适用边界。只背框架参数通常拿不到高分。

02 | 关键公式 / 成本模型

公式 Long Prefill $T \rightarrow \text{ceil}(T/C)$ chunks; C 受 $\text{max_num_batched_tokens}$ /token budget 控制。

03 | Know-Why：为什么成立

- 1. 将长 prompt 拆成若干 token chunk，每轮只用剩余 token budget 处理一部分 Prefill，并优先保护 Decode。这样限制单个 Prefill 的最长阻塞时间，同时可以把 compute-heavy Prefill 与 memory-heavy Decode 混到同一 iteration 提高利用率。
- 把这一结论放进 Roofline / 内存容量 / 调度 / 通信四类模型中检查，确认瓶颈是否真的位于关键路径，而不是只优化了一个非关键算子。

04 | 工程场景

现场推演 32K prompt 不再一次吞掉整个 iteration，而可按 2K/4K chunk 与 decode 交替。

05 | 常见错误

- ✗ chunk 越小越好
- ✗ 过小会增加调度、kernel 和重复边界处理开销。

06 | 追问链

- → chunk size 如何影响 TTFT 和 TPOT?
- → 为什么 vLLM V1 默认优先 Decode?
- → 多模态 Prefill 怎么切?

07 | 面试官评分标准

- 及格：能给出正确概念和基本方向。
- 良好：能写出成本/显存/通信公式，能解释为什么。
- 优秀：能指出反例、适用边界，并能把问题落到 profiler、SLO 或真实系统配置。

Q025 Scheduler 中的 $\text{max_num_batched_tokens}$ 本质是什么?

难度 ★★★★★ · 章节 Batching 与 Scheduling · 关键词 vLLM, token-budget

30 秒回答 它是一次 engine iteration 可处理的 token compute budget，而非简单“batch size”。它同时约束 decode token、新 Prefill token 与 chunked Prefill，决定每 step 工作量、吞吐和尾延迟。

01 | 考察本质

- 这道题不是考名词定义，而是看你能否从系统资源出发解释：它是一次 engine iteration 可处理的 token compute budget，而非简单“batch size”。它同时约束 decode token、新 Prefill token 与 chunked Prefill，决定每 step 工作量、吞吐和尾延迟。
- 面试中应先给结论，再给成本模型/瓶颈，再给适用边界。只背框架参数通常拿不到高分。

02 | 关键公式 / 成本模型

公式 $\Sigma \text{ scheduled_tokens_per_request} \leq \text{max_num_batched_tokens}$ 。

03 | Know-Why：为什么成立

- 1. 它是一次 engine iteration 可处理的 token compute budget，而非简单“batch size”。它同时约束 decode token、新 Prefill token 与 chunked Prefill，决定每 step 工作量、吞吐和尾延迟。
- 把这一结论放进 Roofline / 内存容量 / 调度 / 通信四类模型中检查，确认瓶颈是否真的位于关键路径，而不是只优化了一个非关键算子。

04 | 工程场景

现场推演 降低预算通常保护 ITL；提高预算通常增加 Prefill 吞吐，但每轮更长。

05 | 常见错误

- ✗ 把它和 max_num_seqs 混为一谈
- ✗ 只调一个参数而忽略长度分布与 KV 容量。

06 | 追问链

- → 如何联调 max_num_seqs?
- → 预算过大为什么可能让 p99 变差?
- → DP 下为什么要对齐 prefill cadence?

07 | 面试官评分标准

- 及格：能给出正确概念和基本方向。
- 良好：能写出成本/显存/通信公式，能解释为什么。
- 优秀：能指出反例、适用边界，并能把问题落到 profiler、SLO 或真实系统配置。

Q026 FCFS 和 Cache-aware Scheduling 怎么选?

难度 ★★★★★ · 章节 Batching 与 Scheduling · 关键词 scheduling, cache-aware

30 秒回答 FCFS 简单、公平、可预测；Cache-aware 策略优先让 prefix 相近或缓存命中高的请求执行，可减少 Prefill 重算并提高吞吐。生产系统通常需在 cache benefit 与 waiting time/priority 之间加权，避免 starvation。

01 | 考察本质

- 这道题不是考名词定义，而是看你能否从系统资源出发解释：FCFS 简单、公平、可预测；Cache-aware 策略优先让 prefix 相近或缓存命中高的请求执行，可减少 Prefill 重算并提高吞吐。生产系统通常需在 cache benefit 与 waiting time/priority 之间加权，避免 starvation。
- 面试中应先给结论，再给成本模型/瓶颈，再给适用边界。只背框架参数通常拿不到高分。

02 | 关键公式 / 成本模型

公式 $\text{priority} \approx \alpha \cdot \text{cache_saved_compute} + \beta \cdot \text{waiting_age} + \gamma \cdot \text{tenant_priority}$ 。

03 | Know-Why：为什么成立

- 1. FCFS 简单、公平、可预测
- 2. Cache-aware 策略优先让 prefix 相近或缓存命中高的请求执行，可减少 Prefill 重算并提高吞吐。生产系统通常需在 cache benefit 与 waiting time/priority 之间加权，避免 starvation。
- 把这一结论放进 Roofline / 内存容量 / 调度 / 通信四类模型中检查，确认瓶颈是否真的位于关键路径，而不是只优化了一个非关键算子。

04 | 工程场景

现场推演 Agent workload 大量共享 tool/system prefix 时 LPM/cache-aware 很有效；完全随机 chat 则收益弱。

05 | 常见错误

- **✗** 为了缓存命中无限重排请求
- **✗** 忽略租户公平与 deadline。

06 | 追问链

- → 如何设计 aging?
- → Cache locality 和 SLO 冲突时怎么办?
- → 如何量化 cache_saved_compute?

07 | 面试官评分标准

- 及格：能给出正确概念和基本方向。
- 良好：能写出成本/显存/通信公式，能解释为什么。
- 优秀：能指出反例、适用边界，并能把问题落到 profiler、SLO 或真实系统配置。

Q027 如何解决调度中的 Starvation?

难度 ★★★★★ · 章节 Batching 与 Scheduling · 关键词 fairness, starvation

30 秒回答 给等待时间引入 aging、设置最大等待时长或 deadline、按租户配额和优先级做加权；任何只追求 locality/短作业优先的策略都必须证明不会让某类请求无限等待。

01 | 考察本质

- 这道题不是考名词定义，而是看你能否从系统资源出发解释：给等待时间引入 aging、设置最大等待时长或 deadline、按租户配额和优先级做加权；任何只追求 locality/短作业优先的策略都必须证明不会让某类请求无限等待。
- 面试中应先给结论，再给成本模型/瓶颈，再给适用边界。只背框架参数通常拿不到高分。

02 | 关键公式 / 成本模型

公式 $\text{effective_priority} = \text{base_priority} + f(\text{wait_time})$ ，其中 f 随等待时间单调增强。

03 | Know-Why：为什么成立

- 1. 给等待时间引入 aging、设置最大等待时长或 deadline、按租户配额和优先级做加权
- 2. 任何只追求 locality/短作业优先的策略都必须证明不会让某类请求无限等待。
- 把这一结论放进 Roofline / 内存容量 / 调度 / 通信四类模型中检查，确认瓶颈是否真的位于关键路径，而不是只优化了一个非关键算子。

04 | 工程场景

现场推演 缓存命中高的请求不断到来时，纯 LPM 可能长期压住无共享 prefix 的请求；aging 能逐步提升后者权重。

05 | 常见错误

- ✗ 只平均公平，不看 p99 tenant fairness
- ✗ 优先级倒置。

06 | 追问链

- → SRPT 是否适合 LLM?
- → 输出长度未知怎么估计?
- → 多租户 quota 如何和 cache locality 共存?

07 | 面试官评分标准

- 及格：能给出正确概念和基本方向。
- 良好：能写出成本/显存/通信公式，能解释为什么。
- 优秀：能指出反例、适用边界，并能把问题落到 profiler、SLO 或真实系统配置。

Q028 KV Cache 不够时应该 Swap、Recompute 还是 Reject?

难度 ★★★★★ · 章节 Batching 与 Scheduling · 关键词 preemption, KV

30 秒回答 这是“带宽换计算换排队”的选择。Swap 把 KV 移到 CPU/远端，受传输延迟影响；Recompute 丢弃 KV 后重做 Prefill，消耗 GPU compute；Reject/queue 用 admission control 保证稳定。现代框架还可利用分层缓存、context parallel 或更积极的 watermark。

01 | 考察本质

- 这道题不是考名词定义，而是看你能否从系统资源出发解释：这是“带宽换计算换排队”的选择。Swap 把 KV 移到 CPU/远端，受传输延迟影响；Recompute 丢弃 KV 后重做 Prefill，消耗 GPU compute；Reject/queue 用 admission control 保证稳定。现代框架还可用分层缓存、context parallel 或更积极的 watermark。
- 面试中应先给结论，再给成本模型/瓶颈，再给适用边界。只背框架参数通常拿不到高分。

02 | 关键公式 / 成本模型

公式 比较 $T_{\text{swap}} = \text{bytes}/\text{BW}_{\text{link}}$ 与 $T_{\text{recompute}} = \text{prefill_compute_time}$ 。

03 | Know-Why：为什么成立

- 1. 这是“带宽换计算换排队”的选择。Swap 把 KV 移到 CPU/远端，受传输延迟影响
- 2. Recompute 丢弃 KV 后重做 Prefill，消耗 GPU compute
- 3. Reject/queue 用 admission control 保证稳定。现代框架还可用分层缓存、context parallel 或更积极的 watermark。
- 把这一结论放进 Roofline / 内存容量 / 调度 / 通信四类模型中检查，确认瓶颈是否真的位于关键路径，而不是只优化了一个非关键算子。

04 | 工程场景

现场推演 PCIe 较慢、prompt 不长时 recompute 可能比 swap 更好；超长共享 prefix 则 offload/cache 可能值得。

05 | 常见错误

- **✗** 认为 swap 永远优于 recompute
- **✗** 忽略 vLLM V1 已移除旧式 GPU↔CPU KV swapping 路径这一版本差异。

06 | 追问链

- → 如何做 admission control?
- → 何时预留 KV watermark?
- → HiCache 属于哪类策略?

07 | 面试官评分标准

- 及格：能给出正确概念和基本方向。
- 良好：能写出成本/显存/通信公式，能解释为什么。
- 优秀：能指出反例、适用边界，并能把问题落到 profiler、SLO 或真实系统配置。

Q029 为什么生产系统应该优化 Goodput，而不是最高 Throughput?

难度 ★★★★★ · 章节 Batching 与 Scheduling · 关键词 goodput, production

30 秒回答 Throughput 把超时请求也算“完成”，但业务只关心满足 TTFT/TPOT/端到端 SLO 的请求。Goodput 把性能目标和用户体验直接绑定，能够解释为什么过度 batch 虽提高 tokens/s 却让系统实际可服务能力下降。

01 | 考察本质

- 这道题不是考名词定义，而是看你能否从系统资源出发解释：Throughput 把超时请求也算“完成”，但业务只关心满足 TTFT/TPOT/端到端 SLO 的请求。Goodput 把性能目标和用户体验直接绑定，能够解释为什么过度 batch 虽提高 tokens/s 却让系统实际可服务能力下降。
- 面试中应先给结论，再给成本模型/瓶颈，再给适用边界。只背框架参数通常拿不到高分。

02 | 关键公式 / 成本模型

公式 $\text{Goodput}(\lambda) = \lambda \times P(\text{request meets all SLOs})$ 。

03 | Know-Why：为什么成立

- 1. Throughput 把超时请求也算“完成”，但业务只关心满足 TTFT/TPOT/端到端 SLO 的请求。Goodput 把性能目标和用户体验直接绑定，能够解释为什么过度 batch 虽提高 tokens/s 却让系统实际可服务能力下降。
- 把这一结论放进 Roofline / 内存容量 / 调度 / 通信四类模型中检查，确认瓶颈是否真的位于关键路径，而不是只优化了一个非关键算子。

04 | 工程场景

现场推演 调大 batch 后 throughput +15%，但 p99 TPOT 超标请求从 2% 升到 30%，Goodput 可能下降。

05 | 常见错误

- **✗ 只使用单一 SLO**
- **✗ 真实服务往往有不同 priority/tenant/model 的多 SLO。**

06 | 追问链

- → 如何测最大可持续 arrival rate?
- → Goodput 与 cost/token 如何联合优化?
- → SLO 违约是排队还是执行导致?

07 | 面试官评分标准

- 及格：能给出正确概念和基本方向。
- 良好：能写出成本/显存/通信公式，能解释为什么。
- 优秀：能指出反例、适用边界，并能把问题落到 profiler、SLO 或真实系统配置。

Q030 什么是 Prefill-Decode Disaggregation?

难度 ★★★★★ · 章节 Batching 与 Scheduling · 关键词 PD-disaggregation, DistServe

30 秒回答 把 Prefill 和 Decode 放到不同 GPU pool，分别选择并行策略和容量，避免 compute-heavy 与 latency-sensitive memory-heavy 阶段互相干扰；关键代价是生成后的 KV 必须高效传输到 Decode 节点，因此 network/topology 与 KV transfer overlap 成为核心。

01 | 考察本质

- 这道题不是考名词定义，而是看你能否从系统资源出发解释：把 Prefill 和 Decode 放到不同 GPU pool，分别选择并行策略和容量，避免 compute-heavy 与 latency-sensitive memory-heavy 阶段互相干扰；关键代价是生成后的 KV 必须高效传输到 Decode 节点，因此 network/topology 与 KV transfer overlap 成为核心。
- 面试中应先给结论，再给成本模型/瓶颈，再给适用边界。只背框架参数通常拿不到高分。

02 | 关键公式 / 成本模型

公式 $T_{total} \approx T_{queue_p} + T_{prefill} + T_{KV_transfer} + T_{queue_d} + T_{decode}$ 。只有 $T_{KV_transfer}$ 可被隐藏/摊薄时才值得。

03 | Know-Why：为什么成立

- 1. 把 Prefill 和 Decode 放到不同 GPU pool，分别选择并行策略和容量，避免 compute-heavy 与 latency-sensitive memory-heavy 阶段互相干扰
- 2. 关键代价是生成后的 KV 必须高效传输到 Decode 节点，因此 network/topology 与 KV transfer overlap 成为核心。
- 把这一结论放进 Roofline / 内存容量 / 调度 / 通信四类模型中检查，确认瓶颈是否真的位于关键路径，而不是只优化了一个非关键算子。

04 | 工程场景

现场推演 Prefill pool 可用高吞吐 TP/CP，Decode pool 可按更大 replication/EP 优化；两池按输入/输出负载独立扩容。

05 | 常见错误

- **✗** 只看到“分离就更快”
- **✗** 若网络慢或 prompt 很短，KV transfer 可能吞掉全部收益。

06 | 追问链

- → KV 如何跨节点传？
- → P/D pool 比例如何容量规划？
- → 什么 workload 不适合 disaggregation？

07 | 面试官评分标准

- 及格：能给出正确概念和基本方向。
- 良好：能写出成本/显存/通信公式，能解释为什么。
- 优秀：能指出反例、适用边界，并能把问题落到 profiler、SLO 或真实系统配置。

CHAPTER 04

CUDA、Attention Kernel 与 Runtime

从 IO-aware Attention、Kernel Fusion 到 CUDA Graph，用 profiler 定位真正瓶颈。

本章题目 Q031 - Q040 · 共 10 题

本章学习目标

- 能区分 IO、compute、launch 与 runtime overhead。
- 会用 Nsight 设计证据链。
- 理解 Attention/GEMM kernel 的硬件适配。

建议刷题方法：先用 30 秒回答自测，再遮住正文推导公式，最后只看“追问链”进行模拟面试。

Q031 FlashAttention 为什么更快?

难度 ★★★★★ · 章节 CUDA、Attention Kernel 与 Runtime · 关键词 FlashAttention, IO

30 秒回答 它不近似 attention，而是通过 tiling 与 online softmax 减少 QK^T 、softmax 中间矩阵在 HBM 的反复读写，把更多工作留在 SRAM/register 中。关键收益来自 IO complexity 降低，而非 FLOPs 大幅减少。

01 | 考察本质

- 这道题不是考名词定义，而是看你能否从系统资源出发解释：它不近似 attention，而是通过 tiling 与 online softmax 减少 QK^T 、softmax 中间矩阵在 HBM 的反复读写，把更多工作留在 SRAM/register 中。关键收益来自 IO complexity 降低，而非 FLOPs 大幅减少。
- 面试中应先给结论，再给成本模型/瓶颈，再给适用边界。只背框架参数通常拿不到高分。

02 | 关键公式 / 成本模型

公式 $\text{Attention} = \text{softmax}(QK^T / \sqrt{d})V$ ；通过分块在线维护 softmax 的 max/sum，无需物化完整 $T \times T$ 矩阵。

03 | Know-Why：为什么成立

- 1. 它不近似 attention，而是通过 tiling 与 online softmax 减少 QK^T 、softmax 中间矩阵在 HBM 的反复读写，把更多工作留在 SRAM/register 中。关键收益来自 IO complexity 降低，而非 FLOPs 大幅减少。
- 把这一结论放进 Roofline / 内存容量 / 调度 / 通信四类模型中检查，确认瓶颈是否真的位于关键路径，而不是只优化了一个非关键算子。

04 | 工程场景

现场推演 长序列 Prefill 时中间 attention matrix 巨大，IO-aware 实现能显著减少 HBM traffic。

05 | 常见错误

- ✗ 说“FlashAttention 是稀疏/近似 attention”
- ✗ 混淆训练与 decode 的 kernel shape。

06 | 追问链

- → online softmax 怎么保证数值稳定？
- → 为什么长序列收益更大？
- → FA 与 PagedAttention 的职责差异？

07 | 面试官评分标准

- 及格：能给出正确概念和基本方向。
- 良好：能写出成本/显存/通信公式，能解释为什么。
- 优秀：能指出反例、适用边界，并能把问题落到 profiler、SLO 或真实系统配置。

Q032 FlashAttention-2 与 FlashAttention-3 的思路有什么差异？

难度 ★★★★★ · 章节 CUDA、Attention Kernel 与 Runtime · 关键词 FlashAttention3,Hopper

30 秒回答 FA2 进一步改善 work partition 与并行效率；FA3 面向 Hopper，利用 TMA、warp specialization、Tensor Core/softmax 异步重叠与 FP8 支持，把硬件特性显式纳入算法。核心启示是 kernel 优化必须与 GPU 架构共同设计。

01 | 考察本质

- 这道题不是考名词定义，而是看你能否从系统资源出发解释：FA2 进一步改善 work partition 与并行效率；FA3 面向 Hopper，利用 TMA、warp specialization、Tensor Core/softmax 异步重叠与 FP8 支持，把硬件特性显式纳入算法。核心启示是 kernel 优化必须与 GPU 架构共同设计。
- 面试中应先给结论，再给成本模型/瓶颈，再给适用边界。只背框架参数通常拿不到高分。

02 | 关键公式 / 成本模型

公式 目标从“减少 IO”进一步到“流水化数据搬运、matmul 与 softmax”。

03 | Know-Why：为什么成立

- 1. FA2 进一步改善 work partition 与并行效率
- 2. FA3 面向 Hopper，利用 TMA、warp specialization、Tensor Core/softmax 异步重叠与 FP8 支持，把硬件特性显式纳入算法。核心启示是 kernel 优化必须与 GPU 架构共同设计。
- 把这一结论放进 Roofline / 内存容量 / 调度 / 通信四类模型中检查，确认瓶颈是否真的位于关键路径，而不是只优化了一个非关键算子。

04 | 工程场景

现场推演 在 H100 上使用旧 kernel 可能只能利用部分硬件能力；FA3 针对 Hopper 的异步机制提高实际利用率。

05 | 常见错误

- ✗ 把论文 speedup 直接外推到 A100/B200
- ✗ 硬件专用优化必须按架构验证。

06 | 追问链

- → TMA 做什么？
- → warp specialization 为什么有效？
- → Blackwell 上是否仍应直接用 FA3？

07 | 面试官评分标准

- 及格：能给出正确概念和基本方向。
- 良好：能写出成本/显存/通信公式，能解释为什么。
- 优秀：能指出反例、适用边界，并能把问题落到 profiler、SLO 或真实系统配置。

Q033 FlashAttention 与 FlashInfer 有什么区别？

难度 ★★★★★ · 章节 CUDA、Attention Kernel 与 Runtime · 关键词 FlashInfer, attention-backend

30 秒回答 FlashAttention 是高性能 dense attention kernel 家族；FlashInfer 从 serving 视角处理 paged/ragged KV、可组合 attention pattern、动态负载与 JIT 定制，更强调 inference runtime 中多种形状和缓存布局。二者可以互补而非二选一。

01 | 考察本质

- 这道题不是考名词定义，而是看你能否从系统资源出发解释：FlashAttention 是高性能 dense attention kernel 家族；FlashInfer 从 serving 视角处理 paged/ragged KV、可组合 attention pattern、动态负载与 JIT 定制，更强调 inference runtime 中多种形状和缓存布局。二者可以互补而非二选一。
- 面试中应先给结论，再给成本模型/瓶颈，再给适用边界。只背框架参数通常拿不到高分。

02 | 关键公式 / 成本模型

公式 Serving attention 的输入通常是 ragged sequences + paged KV，而非单一矩形 tensor。

03 | Know-Why：为什么成立

- 1. FlashAttention 是高性能 dense attention kernel 家族
- 2. FlashInfer 从 serving 视角处理 paged/ragged KV、可组合 attention pattern、动态负载与 JIT 定制，更强调 inference runtime 中多种形状和缓存布局。二者可以互补而非二选一。
- 把这一结论放进 Roofline / 内存容量 / 调度 / 通信四类模型中检查，确认瓶颈是否真的位于关键路径，而不是只优化了一个非关键算子。

04 | 工程场景

现场推演 Decode batch 中每个请求 context 不同，FlashInfer 类引擎需要处理 load balancing 与 CUDAGraph 兼容。

05 | 常见错误

- ✗ 把“FlashInfer 更快”当通用结论
- ✗ 实际 backend 取决于模型、GPU、head_dim、KV layout。

06 | 追问链

- → 为什么 serving kernel 需要 plan/run 分离？
- → block-sparse KV 的意义？
- → 如何做 backend autotuning？

07 | 面试官评分标准

- 及格：能给出正确概念和基本方向。
- 良好：能写出成本/显存/通信公式，能解释为什么。
- 优秀：能指出反例、适用边界，并能把问题落到 profiler、SLO 或真实系统配置。

Q034 Kernel Fusion 为什么能提升性能?

难度 ★★★★★ · 章节 CUDA、Attention Kernel 与 Runtime · 关键词 kernel-fusion, CUDA

30 秒回答 融合相邻算子可以减少中间 tensor 的 HBM 往返和 kernel launch，尤其适合 RMSNorm、bias、activation、residual 等 memory-bound 小算子。是否值得融合取决于寄存器压力、occupancy 与可复用 kernel。

01 | 考察本质

- 这道题不是考名词定义，而是看你能否从系统资源出发解释：融合相邻算子可以减少中间 tensor 的 HBM 往返和 kernel launch，尤其适合 RMSNorm、bias、activation、residual 等 memory-bound 小算子。是否值得融合取决于寄存器压力、occupancy 与可复用 kernel。
- 面试中应先给结论，再给成本模型/瓶颈，再给适用边界。只背框架参数通常拿不到高分。

02 | 关键公式 / 成本模型

公式 $T_{\text{unfused}} \approx \Sigma(T_{\text{launch}} + T_{\text{read}} + T_{\text{compute}} + T_{\text{write}})$; fusion 尽量合并 read/write 与 launch。

03 | Know-Why: 为什么成立

- 1. 融合相邻算子可以减少中间 tensor 的 HBM 往返和 kernel launch，尤其适合 RMSNorm、bias、activation、residual 等 memory-bound 小算子。是否值得融合取决于寄存器压力、occupancy 与可复用 kernel。
- 把这一结论放进 Roofline / 内存容量 / 调度 / 通信四类模型中检查，确认瓶颈是否真的位于关键路径，而不是只优化了一个非关键算子。

04 | 工程场景

现场推演 RMSNorm → Linear 前后的某些 elementwise 逻辑融合后，可避免额外 HBM round-trip。

05 | 常见错误

- ✗ 无限融合
- ✗ 过大 kernel 可能导致 register spilling/occupancy 下降。

06 | 追问链

- → Epilogue fusion 是什么？
- → 为什么 GEMM 主体不一定适合和所有算子融合？
- → 如何通过 profiler 判断？

07 | 面试官评分标准

- 及格：能给出正确概念和基本方向。
- 良好：能写出成本/显存/通信公式，能解释为什么。
- 优秀：能指出反例、适用边界，并能把问题落到 profiler、SLO 或真实系统配置。

Q035 CUDA Graph 为什么适合 Decode?

难度 ★★★★★ · 章节 CUDA、Attention Kernel 与 Runtime · 关键词 CUDA-Graph,decode

30 秒回答 Decode 每步执行结构相似且 kernel 小，CPU 每 token 发起大量 launch 会形成明显空隙。CUDA Graph 可 capture 一组 GPU 操作并低开销 replay；难点是动态 batch/shape，因此 serving runtime 常用 shape bucketing、padding 或 piecewise graph。

01 | 考察本质

- 这道题不是考名词定义，而是看你能否从系统资源出发解释：Decode 每步执行结构相似且 kernel 小，CPU 每 token 发起大量 launch 会形成明显空隙。CUDA Graph 可 capture 一组 GPU 操作并低开销 replay；难点是动态 batch/shape，因此 serving runtime 常用 shape bucketing、padding 或 piecewise graph。
- 面试中应先给结论，再给成本模型/瓶颈，再给适用边界。只背框架参数通常拿不到高分。

02 | 关键公式 / 成本模型

公式 当 $\Sigma \text{launch_overhead}$ 与 kernel compute 同量级时，graph replay 的收益显著。

03 | Know-Why：为什么成立

- 1. Decode 每步执行结构相似且 kernel 小，CPU 每 token 发起大量 launch 会形成明显空隙。CUDA Graph 可 capture 一组 GPU 操作并低开销 replay
- 2. 难点是动态 batch/shape，因此 serving runtime 常用 shape bucketing、padding 或 piecewise graph。
- 把这一结论放进 Roofline / 内存容量 / 调度 / 通信四类模型中检查，确认瓶颈是否真的位于关键路径，而不是只优化了一个非关键算子。

04 | 工程场景

现场推演 batch=1 的小模型 decode 中，GPU timeline 若反复出现 CPU launch gap，graph 可能比换更快 GEMM 更有价值。

05 | 常见错误

- ✗ 以为 CUDA Graph 减少 FLOPs
- ✗ 忽略 graph capture 内存与动态控制流限制。

06 | 追问链

- → continuous batching 如何和 static graph 共存？
- → piecewise CUDA Graph 是什么？
- → 何时 graph 反而浪费内存？

07 | 面试官评分标准

- 及格：能给出正确概念和基本方向。
- 良好：能写出成本/显存/通信公式，能解释为什么。
- 优秀：能指出反例、适用边界，并能把问题落到 profiler、SLO 或真实系统配置。

Q036 torch.compile / Triton 在推理优化中解决什么？

难度 ★★★★★ · 章节 CUDA、Attention Kernel 与 Runtime · 关键词 torch.compile, Triton

30 秒回答 torch.compile 可做图级融合、常量传播、shape specialization 和 codegen；Triton 允许更快开发定制 GPU kernel。它们降低 Python/框架 overhead 并针对实际 shape 生成实现，但动态 shape、graph break 与 recompilation 可能导致 cold-start 或抖动。

01 | 考察本质

- 这道题不是考名词定义，而是看你能否从系统资源出发解释：torch.compile 可做图级融合、常量传播、shape specialization 和 codegen；Triton 允许更快开发定制 GPU kernel。它们降低 Python/框架 overhead 并针对实际 shape 生成实现，但动态 shape、graph break 与 recompilation 可能导致 cold-start 或抖动。
- 面试中应先给结论，再给成本模型/瓶颈，再给适用边界。只背框架参数通常拿不到高分。

02 | 关键公式 / 成本模型

公式 收益 = 更少 graph break/launch + 更优 fusion/kernel；代价 = compile time + code cache + specialization complexity。

03 | Know-Why：为什么成立

- 1. torch.compile 可做图级融合、常量传播、shape specialization 和 codegen
- 2. Triton 允许更快开发定制 GPU kernel。它们降低 Python/框架 overhead 并针对实际 shape 生成实现，但动态 shape、graph break 与 recompilation 可能导致 cold-start 或抖动。
- 把这一结论放进 Roofline / 内存容量 / 调度 / 通信四类模型中检查，确认瓶颈是否真的位于关键路径，而不是只优化了一个非关键算子。

04 | 工程场景

现场推演 同一线上服务 shape 分布稳定时 compile 缓存可复用；高度动态多模态输入可能导致大量 specialization。

05 | 常见错误

- ✗ 看到 compile=True 就期待稳定提速
- ✗ 未区分首次编译和 steady-state。

06 | 追问链

- → 如何定位 graph break？
- → Triton 与 CUTLASS 各适合什么？
- → 怎么管理编译 cache？

07 | 面试官评分标准

- 及格：能给出正确概念和基本方向。
- 良好：能写出成本/显存/通信公式，能解释为什么。
- 优秀：能指出反例、适用边界，并能把问题落到 profiler、SLO 或真实系统配置。

Q037 什么情况下 Kernel Launch Overhead 会成为瓶颈?

难度 ★★☆☆☆ · 章节 CUDA、Attention Kernel 与 Runtime · 关键词 launch-overhead,CUDA

30 秒回答 当单 kernel 计算量很小、batch 小、decode 每步包含大量短 kernel 时，CPU launch 和调度延迟占比上升。表现为 GPU timeline 中 kernel 间存在稳定微小空洞，SM/HBM 都没满。

01 | 考察本质

- 这道题不是考名词定义，而是看你能否从系统资源出发解释：当单 kernel 计算量很小、batch 小、decode 每步包含大量短 kernel 时，CPU launch 和调度延迟占比上升。表现为 GPU timeline 中 kernel 间存在稳定微小空洞，SM/HBM 都没满。
- 面试中应先给结论，再给成本模型/瓶颈，再给适用边界。只背框架参数通常拿不到高分。

02 | 关键公式 / 成本模型

公式 若 $T_{\text{kernel}} \approx$ 几微秒到几十微秒， T_{launch} 的占比不可忽略。

03 | Know-Why: 为什么成立

- 1. 当单 kernel 计算量很小、batch 小、decode 每步包含大量短 kernel 时，CPU launch 和调度延迟占比上升。表现为 GPU timeline 中 kernel 间存在稳定微小空洞，SM/HBM 都没满。
- 把这一结论放进 Roofline / 内存容量 / 调度 / 通信四类模型中检查，确认瓶颈是否真的位于关键路径，而不是只优化了一个非关键算子。

04 | 工程场景

现场推演 小 batch decoder-only 模型经常比大 Prefill 更容易出现 launch-bound。

05 | 常见错误

- ✗ 把低 GPU util 一律归因内存
- ✗ launch-bound 时 HBM 也可能很低。

06 | 追问链

- → 如何用 CUDA Graph 改善?
- → Fusion 和 graph 哪个先做?
- → 多进程/CPU affinity 会影响吗?

07 | 面试官评分标准

- 及格：能给出正确概念和基本方向。
- 良好：能写出成本/显存/通信公式，能解释为什么。
- 优秀：能指出反例、适用边界，并能把问题落到 profiler、SLO 或真实系统配置。

Q038 为什么 Prefill GEMM 和 Decode GEMM 优化完全不同?

难度 ★★★★★ · 章节 CUDA、Attention Kernel 与 Runtime · 关键词 GEMM,prefill-decode

30 秒回答 Prefill 的矩阵 M 通常是 token 数 $\times$ batch，较大；Decode M 可能只有 batch，形成 small-GEMM/GEMV。不同

M/N/K 使 tile、split-K、warp 数、epilogue 与 Tensor Core 利用率的最优点不同，因此不能用一个 kernel 配置覆盖全部。

01 | 考察本质

- 这道题不是考名词定义，而是看你能否从系统资源出发解释：Prefill 的矩阵 M 通常是 token 数 $\times$ batch，较大；Decode M 可能只有 batch，形成 small-GEMM/GEMV。不同 M/N/K 使 tile、split-K、warp 数、epilogue 与 Tensor Core 利用率的最优点不同，因此不能用一个 kernel 配置覆盖全部。
- 面试中应先给结论，再给成本模型/瓶颈，再给适用边界。只背框架参数通常拿不到高分。

02 | 关键公式 / 成本模型

公式 Prefill $[B \cdot T, K] \times [K, N]$ ；Decode $[B, K] \times [K, N]$ 。

03 | Know-Why：为什么成立

- 1. Prefill 的矩阵 M 通常是 token 数 $\times$ batch，较大
- 2. Decode M 可能只有 batch，形成 small-GEMM/GEMV。不同 M/N/K 使 tile、split-K、warp 数、epilogue 与 Tensor Core 利用率的最优点不同，因此不能用一个 kernel 配置覆盖全部。
- 把这一结论放进 Roofline / 内存容量 / 调度 / 通信四类模型中检查，确认瓶颈是否真的位于关键路径，而不是只优化了一个非关键算子。

04 | 工程场景

现场推演 同一个 FFN Linear，在 8K Prefill 可接近大 GEMM 峰值，在 batch=1 Decode 则受权重读取和 launch 限制。

05 | 常见错误

- **✗** 只根据模型 hidden size 选 kernel，不看 M 维与实际 batch。

06 | 追问链

- → split-K 何时有用？
- → 为什么 grouped GEMM 对 MoE 重要？
- → 如何做 shape profile？

07 | 面试官评分标准

- 及格：能给出正确概念和基本方向。
- 良好：能写出成本/显存/通信公式，能解释为什么。
- 优秀：能指出反例、适用边界，并能把问题落到 profiler、SLO 或真实系统配置。

Q039 为什么 Tensor Core 对矩阵 Shape 很敏感？

难度 ★★★★★ · 章节 CUDA、Attention Kernel 与 Runtime · 关键词 Tensor-Core, shape

30 秒回答 Tensor Core 有特定 tile/数据类型/对齐要求。矩阵维度不对齐、M 太小或 ragged 会造成 padding、低 occupancy 或无法走最优 kernel；因此 hidden size、head_dim、group size 也会成为系统性能参数。

01 | 考察本质

- 这道题不是考名词定义，而是看你能否从系统资源出发解释：Tensor Core 有特定 tile/数据类型/对齐要求。矩阵维度不对齐、M 太小或 ragged 会造成 padding、低 occupancy 或无法走最优 kernel；因此 hidden size、head_dim、group size 也会成为系统性能参数。
- 面试中应先给结论，再给成本模型/瓶颈，再给适用边界。只背框架参数通常拿不到高分。

02 | 关键公式 / 成本模型

公式 有效利用率 $\approx \text{useful_math} / (\text{padded_math} + \text{stalls})$ 。

03 | Know-Why：为什么成立

- 1. Tensor Core 有特定 tile/数据类型/对齐要求。矩阵维度不对齐、M 太小或 ragged 会造成 padding、低 occupancy 或无法走最优 kernel
- 2. 因此 hidden size、head_dim、group size 也会成为系统性能参数。
- 把这一结论放进 Roofline / 内存容量 / 调度 / 通信四类模型中检查，确认瓶颈是否真的位于关键路径，而不是只优化了一个非关键算子。

04 | 工程场景

现场推演 某量化模型 group size 与 tensor core tile 不匹配，理论 bit 数更低却可能 kernel 更慢。

05 | 常见错误

- ✗ 只看 dtype 支持
- ✗ 忽略 alignment、layout 与 epilogue。

06 | 追问链

- → head_dim=128 为什么常见？
- → 量化 group size 如何影响 kernel？
- → MoE expert 小 M 如何提高利用率？

07 | 面试官评分标准

- 及格：能给出正确概念和基本方向。
- 良好：能写出成本/显存/通信公式，能解释为什么。
- 优秀：能指出反例、适用边界，并能把问题落到 profiler、SLO 或真实系统配置。

Q040 Nsight 里看到 GPU Util=100%，为什么模型仍可能很慢？

难度 ★★★★★ · 章节 CUDA、Attention Kernel 与 Runtime · 关键词 Nsight, profiling

30 秒回答 “GPU active” 只说明设备有工作，不代表 Tensor Core 或 HBM 达到有效峰值。可能是单个低效率 kernel、memcpy、串行 NCCL 或大量 stall。必须继续看 SM throughput、Tensor pipe、DRAM throughput、occupancy、stall reasons 与时间线重叠。

01 | 考察本质

- 这道题不是考名词定义，而是看你能否从系统资源出发解释：“GPU active”只说明设备有工作，不代表 Tensor Core 或 HBM 达到有效峰值。可能是单个低效率 kernel、memcpy、串行 NCCL 或大量 stall。必须继续看 SM throughput、Tensor pipe、DRAM throughput、occupancy、stall reasons 与时间线重叠。
- 面试中应先给结论，再给成本模型/瓶颈，再给适用边界。只背框架参数通常拿不到高分。

02 | 关键公式 / 成本模型

公式 Utilization 是“是否忙”的指标，不是“忙得有效”的指标。

03 | Know-Why：为什么成立

- 1. “GPU active”只说明设备有工作，不代表 Tensor Core 或 HBM 达到有效峰值。可能是单个低效率 kernel、memcpy、串行 NCCL 或大量 stall。必须继续看 SM throughput、Tensor pipe、DRAM throughput、occupancy、stall reasons 与时间线重叠。
- 把这一结论放进 Roofline / 内存容量 / 调度 / 通信四类模型中检查，确认瓶颈是否真的位于关键路径，而不是只优化了一个非关键算子。

04 | 工程场景

现场推演 一个 memory copy 持续占 GPU 时间也可能让利用率显示很高，但模型 FLOP/s 很低。

05 | 常见错误

- **✗** 用 `nvidia-smi GPU-Util` 作为优化终点。

06 | 追问链

- → Nsight Systems 与 Compute 各看哪些 counter?
- → 如何识别 memory throttling?
- → 如何找 NCCL overlap?

07 | 面试官评分标准

- 及格：能给出正确概念和基本方向。
- 良好：能写出成本/显存/通信公式，能解释为什么。
- 优秀：能指出反例、适用边界，并能把问题落到 profiler、SLO 或真实系统配置。

CHAPTER 05

量化与低精度推理

比较 GPTQ、AWQ、SmoothQuant、FP8/FP4、KV Quantization 及其硬件约束。

本章题目 Q041 - Q050 · 共 10 题

本章学习目标

- 能比较主流 PTQ/低精度路线。
- 能解释为何 bit 更低不一定更快。
- 会做量化后的质量-性能联合回归。

建议刷题方法：先用 30 秒回答自测，再遮住正文推导公式，最后只看“追问链”进行模拟面试。

Q041 Weight-only Quantization 与 W8A8 有什么区别？

难度 ★★★★★ · 章节 量化与低精度推理 · 关键词 quantization, W4A16, W8A8

30 秒回答 Weight-only 如 W4A16 主要降低权重容量与读取带宽，activation 保持高精度并在 GEMM 前/中 dequant；W8A8 同时量化 activation，可真正利用低精度 Tensor Core 提升计算吞吐，但需要处理 activation outlier 与 calibration。

01 | 考察本质

- 这道题不是考名词定义，而是看你能否从系统资源出发解释：Weight-only 如 W4A16 主要降低权重容量与读取带宽，activation 保持高精度并在 GEMM 前/中 dequant；W8A8 同时量化 activation，可真正利用低精度 Tensor Core 提升计算吞吐，但需要处理 activation outlier 与 calibration。
- 面试中应先给结论，再给成本模型/瓶颈，再给适用边界。只背框架参数通常拿不到高分。

02 | 关键公式 / 成本模型

公式 W4A16: W 低比特, A 高精度；W8A8: W/A 均低精度。

03 | Know-Why: 为什么成立

- 1. Weight-only 如 W4A16 主要降低权重容量与读取带宽，activation 保持高精度并在 GEMM 前/中 dequant
- 2. W8A8 同时量化 activation，可真正利用低精度 Tensor Core 提升计算吞吐，但需要处理 activation outlier 与 calibration。
- 把这一结论放进 Roofline / 内存容量 / 调度 / 通信四类模型中检查，确认瓶颈是否真的位于关键路径，而不是只优化了一个非关键算子。

04 | 工程场景

现场推演 低 batch Decode 主要受权重带宽限制，weight-only 很有吸引力；大 Prefill compute-bound 时 W8A8/FP8 的计算吞吐优势更明显。

05 | 常见错误

- ✗ 认为 bit 越低越快
- ✗ dequant kernel 和硬件原生支持决定实际速度。

06 | 追问链

- → 为什么 W4A16 对 Prefill 可能收益有限？
- → W8A8 如何处理 outlier？
- → 量化 scale 放在哪里？

07 | 面试官评分标准

- 及格：能给出正确概念和基本方向。
- 良好：能写出成本/显存/通信公式，能解释为什么。
- 优秀：能指出反例、适用边界，并能把问题落到 profiler、SLO 或真实系统配置。

Q042 GPTQ 原理是什么？

难度 ★★★★★ · 章节 量化与低精度推理 · 关键词 GPTQ, PTQ

30 秒回答 GPTQ 是 post-training weight quantization，通过对一小批 calibration activation 估计二阶敏感度，逐列/分组量化并对尚未量化的权重做误差补偿，从而在 3/4-bit 下控制输出误差。

01 | 考察本质

- 这道题不是考名词定义，而是看你能否从系统资源出发解释：GPTQ 是 post-training weight quantization，通过对一小批 calibration activation 估计二阶敏感度，逐列/分组量化并对尚未量化的权重做误差补偿，从而在 3/4-bit 下控制输出误差。
- 面试中应先给结论，再给成本模型/瓶颈，再给适用边界。只背框架参数通常拿不到高分。

02 | 关键公式 / 成本模型

公式 目标近似最小化 $\|WX - W_qX\|^2$ ，并用 Hessian $H \approx XX^T$ 的逆/近似指导补偿。

03 | Know-Why：为什么成立

- 1. GPTQ 是 post-training weight quantization，通过对一小批 calibration activation 估计二阶敏感度，逐列/分组量化并对尚未量化的权重做误差补偿，从而在 3/4-bit 下控制输出误差。
- 把这一结论放进 Roofline / 内存容量 / 调度 / 通信四类模型中检查，确认瓶颈是否真的位于关键路径，而不是只优化了一个非关键算子。

04 | 工程场景

现场推演 相同 4-bit，简单 round-to-nearest 可能破坏敏感方向，GPTQ 用校准数据捕捉这些方向。

05 | 常见错误

- ✗ 把 GPTQ 说成训练中量化
- ✗ 它是典型 PTQ。

06 | 追问链

- → 为什么需要 calibration data?
- → group size 如何影响?
- → GPTQ 与 AWQ 的敏感度定义有何不同?

07 | 面试官评分标准

- 及格：能给出正确概念和基本方向。
- 良好：能写出成本/显存/通信公式，能解释为什么。
- 优秀：能指出反例、适用边界，并能把问题落到 profiler、SLO 或真实系统配置。

Q043 AWQ 和 GPTQ 最大区别是什么？

难度 ★★★★★ · 章节 量化与低精度推理 · 关键词 AWQ, quantization

30 秒回答 AWQ 的核心观察是少量 salient weight channel 对 activation 很敏感，通过 activation-aware scaling 保护这些

通道，再做低比特量化；GPTQ 更依赖二阶误差补偿。AWQ 的方法更轻量，且容易与高效 weight-only kernel 结合。

01 | 考察本质

- 这道题不是考名词定义，而是看你能否从系统资源出发解释：AWQ 的核心观察是少量 salient weight channel 对 activation 很敏感，通过 activation-aware scaling 保护这些通道，再做低比特量化；GPTQ 更依赖二阶误差补偿。AWQ 的方法更轻量，且容易与高效 weight-only kernel 结合。
- 面试中应先给结论，再给成本模型/瓶颈，再给适用边界。只背框架参数通常拿不到高分。

02 | 关键公式 / 成本模型

公式 AWQ 通过等价 scaling 改变量化前权重动态范围，保护重要 channel。

03 | Know-Why：为什么成立

- 1. AWQ 的核心观察是少量 salient weight channel 对 activation 很敏感，通过 activation-aware scaling 保护这些通道，再做低比特量化
- 2. GPTQ 更依赖二阶误差补偿。AWQ 的方法更轻量，且容易与高效 weight-only kernel 结合。
- 把这一结论放进 Roofline / 内存容量 / 调度 / 通信四类模型中检查，确认瓶颈是否真的位于关键路径，而不是只优化了一个非关键算子。

04 | 工程场景

现场推演 对某层，activation 幅度大的输入通道对应权重更应被精细保留；统一 scale 可能对其伤害大。

05 | 常见错误

- ✗ 把 “activation-aware” 误解为 activation 也量化。

06 | 追问链

- → AWQ 为什么只需要少量 calibration?
- → salient channel 比例怎么选?
- → 和 SmoothQuant 的 scaling 有何不同?

07 | 面试官评分标准

- 及格：能给出正确概念和基本方向。
- 良好：能写出成本/显存/通信公式，能解释为什么。
- 优秀：能指出反例、适用边界，并能把问题落到 profiler、SLO 或真实系统配置。

Q044 SmoothQuant 为什么叫“把难度从 Activation 搬到 Weight”？

难度 ★★★★★ · 章节 量化与低精度推理 · 关键词 SmoothQuant, INT8

30 秒回答 激活 outlier 让 INT8 动态范围难覆盖。SmoothQuant 用通道缩放 S 做等价变换 $XW = (XS^{-1})(SW)$ ，把激活中的幅值差异转移到相对更容易离线量化的权重，从而实现更稳定的 W8A8。

01 | 考察本质

- 这道题不是考名词定义，而是看你能否从系统资源出发解释：激活 outlier 让 INT8 动态范围难覆盖。SmoothQuant 用通道缩放 S 做等价变换 $XW=(XS^{-1})(SW)$ ，把激活中的幅值差异转移到相对更容易离线量化的权重，从而实现更稳定的 W8A8。
- 面试中应先给结论，再给成本模型/瓶颈，再给适用边界。只背框架参数通常拿不到高分。

02 | 关键公式 / 成本模型

公式 $XW=(XS^{-1})(SW)$ ，平滑系数决定 activation 与 weight 间的难度分配。

03 | Know-Why：为什么成立

- 1. 激活 outlier 让 INT8 动态范围难覆盖。SmoothQuant 用通道缩放 S 做等价变换 $XW=(XS^{-1})(SW)$ ，把激活中的幅值差异转移到相对更容易离线量化的权重，从而实现更稳定的 W8A8。
- 把这一结论放进 Roofline / 内存容量 / 调度 / 通信四类模型中检查，确认瓶颈是否真的位于关键路径，而不是只优化了一个非关键算子。

04 | 工程场景

现场推演 LLM 某些 channel 激活极大，per-tensor activation scale 会浪费大量量化级；平滑后可提高普通值分辨率。

05 | 常见错误

- ✗ 认为它降低了模型 FLOPs
- ✗ 本质是数值重参数化。

06 | 追问链

- → α 参数控制什么？
- → 为什么权重更容易承受 outlier？
- → 和 per-channel activation quant 比较。

07 | 面试官评分标准

- 及格：能给出正确概念和基本方向。
- 良好：能写出成本/显存/通信公式，能解释为什么。
- 优秀：能指出反例、适用边界，并能把问题落到 profiler、SLO 或真实系统配置。

Q045 FP8 与 INT8 的工程区别是什么？

难度 ★★★★★ · 章节 量化与低精度推理 · 关键词 FP8,INT8

30 秒回答 INT8 是定点/整数表示，需要 scale 映射，范围与精度由 scale 强烈决定；FP8 带 exponent，动态范围更自然，并可由 Hopper/Blackwell Transformer Engine 原生支持。工程上需看具体 E4M3/E5M2、scale 粒度、accumulation dtype 和 kernel。

01 | 考察本质

- 这道题不是考名词定义，而是看你能否从系统资源出发解释：INT8 是定点/整数表示，需要 scale 映射，范围与精度由 scale 强烈决定；FP8 带 exponent，动态范围更自然，并可由 Hopper/Blackwell Transformer Engine 原生支持。工程上需看具体 E4M3/E5M2、scale 粒度、accumulation dtype 和 kernel。
- 面试中应先给结论，再给成本模型/瓶颈，再给适用边界。只背框架参数通常拿不到高分。

02 | 关键公式 / 成本模型

公式 FP8 的 exponent/mantissa 在动态范围与精度间折中；通常仍使用更高精度累加。

03 | Know-Why：为什么成立

- 1. INT8 是定点/整数表示，需要 scale 映射，范围与精度由 scale 强烈决定
- 2. FP8 带 exponent，动态范围更自然，并可由 Hopper/Blackwell Transformer Engine 原生支持。工程上需看具体 E4M3/E5M2、scale 粒度、accumulation dtype 和 kernel。
- 把这一结论放进 Roofline / 内存容量 / 调度 / 通信四类模型中检查，确认瓶颈是否真的位于关键路径，而不是只优化了一个非关键算子。

04 | 工程场景

现场推演 激活范围变化大的模型在 FP8 下可能比 INT8 更容易保持稳定，同时享受原生 Tensor Core。

05 | 常见错误

- ✗ “FP8 一定比 INT8 准确/快”
- ✗ 硬件和 kernel 决定。

06 | 追问链

- → E4M3 vs E5M2 怎么选？
- → FP8 weight+activation 如何校准？
- → FP8 KV 与 FP8 GEMM 是否同一问题？

07 | 面试官评分标准

- 及格：能给出正确概念和基本方向。
- 良好：能写出成本/显存/通信公式，能解释为什么。
- 优秀：能指出反例、适用边界，并能把问题落到 profiler、SLO 或真实系统配置。

Q046 FP4 / NVFP4 为什么到 Blackwell 才更加实用？

难度 ★★★★★ · 章节 量化与低精度推理 · 关键词 FP4,NVFP4,Blackwell

30 秒回答 4-bit 浮点能显著降低权重/activation 流量，但动态范围很窄，需要细粒度缩放和硬件原生 Tensor Core 才能兼顾准确性与速度。Blackwell 引入针对 FP4/NVFP4 的硬件路径，使“低 bit + 细粒度 scale”不必付出过高软件开销。

01 | 考察本质

- 这道题不是考名词定义，而是看你能否从系统资源出发解释：4-bit 浮点能显著降低权重/activation 流量，但动态范围很窄，需要细粒度缩放和硬件原生 Tensor Core 才能兼顾准确性与速度。Blackwell 引入针对 FP4/NVFP4 的硬件路径，使“低 bit + 细粒度 scale”不必付出过高软件开销。
- 面试中应先给结论，再给成本模型/瓶颈，再给适用边界。只背框架参数通常拿不到高分。

02 | 关键公式 / 成本模型

公式 理论存储：BF16 2 byte → FP4 0.5 byte，主权重约 4× 缩减。

03 | Know-Why：为什么成立

- 1. 4-bit 浮点能显著降低权重/activation 流量，但动态范围很窄，需要细粒度缩放和硬件原生 Tensor Core 才能兼顾准确性与速度。Blackwell 引入针对 FP4/NVFP4 的硬件路径，使“低 bit + 细粒度 scale”不必付出过高软件开销。
- 把这一结论放进 Roofline / 内存容量 / 调度 / 通信四类模型中检查，确认瓶颈是否真的位于关键路径，而不是只优化了一个非关键算子。

04 | 工程场景

现场推演 如果 GPU 没有原生 FP4 matrix path，只靠软件 unpack/dequant，容量省了但 latency 未必下降。

05 | 常见错误

- ✗ 忽略 scale metadata 和 accumulation 精度
- ✗ 把 FP4 等同于 INT4。

06 | 追问链

- → NVFP4 的 scale 粒度为什么重要？
- → Blackwell Tensor Core 新特性如何影响？
- → 何时仍选择 FP8？

07 | 面试官评分标准

- 及格：能给出正确概念和基本方向。
- 良好：能写出成本/显存/通信公式，能解释为什么。
- 优秀：能指出反例、适用边界，并能把问题落到 profiler、SLO 或真实系统配置。

Q047 Per-tensor、Per-channel、Per-group quantization 怎么选？

难度 ★★★★★ · 章节 量化与低精度推理 · 关键词 quantization,scales

30 秒回答 scale 越细，越能适配不同通道/局部分布，精度更高，但 metadata、load、kernel 复杂度也更大。Per-tensor 最简单；Per-channel 精细；Group-wise 常用 32/64/128 等组大小在准确性与吞吐之间折中。

01 | 考察本质

- 这道题不是考名词定义，而是看你能否从系统资源出发解释：scale 越细，越能适配不同通道/局部分布，精度更高，但 metadata、load、kernel 复杂度也更大。Per-tensor 最简单；Per-channel 精细；Group-wise 常用 32/64/128 等组大小在准确性与吞吐之间折中。
- 面试中应先给结论，再给成本模型/瓶颈，再给适用边界。只背框架参数通常拿不到高分。

02 | 关键公式 / 成本模型

公式 $\text{quant}(x) = \text{round}(x/s_g)$ ，g 可为 tensor/channel/group。

03 | Know-Why：为什么成立

- 1. scale 越细，越能适配不同通道/局部分布，精度更高，但 metadata、load、kernel 复杂度也更大。Per-tensor 最简单
- 2. Per-channel 精细
- 3. Group-wise 常用 32/64/128 等组大小在准确性与吞吐之间折中。
- 把这一结论放进 Roofline / 内存容量 / 调度 / 通信四类模型中检查，确认瓶颈是否真的位于关键路径，而不是只优化了一个非关键算子。

04 | 工程场景

现场推演 INT4 weight-only 常见 group-wise；group 太小会让 scale 占比和 kernel 开销增加。

05 | 常见错误

- ✗ 只以 perplexity 选择 group size
- ✗ 应同时测 kernel 和显存。

06 | 追问链

- → group size 为什么影响 Tensor Core 对齐？
- → scale dtype 用 FP16 还是 FP32？
- → zero-point 有什么影响？

07 | 面试官评分标准

- 及格：能给出正确概念和基本方向。
- 良好：能写出成本/显存/通信公式，能解释为什么。
- 优秀：能指出反例、适用边界，并能把问题落到 profiler、SLO 或真实系统配置。

Q048 为什么 KV Quantization 和 Weight Quantization 是两个问题？

难度 ★★★★★ · 章节 量化与低精度推理 · 关键词 KV-quant,dynamic

30 秒回答 权重静态、可离线校准与预打包；KV 是每请求在线生成、随 context 增长且每层动态变化。KV quant 需要在线确定 scale、写入低精度 cache，并在 attention 中高效 dequant/compute；错误还可能沿长上下文累积。

01 | 考察本质

- 这道题不是考名词定义，而是看你能否从系统资源出发解释：权重静态、可离线校准与预打包；KV 是每请求在线生成、随 context 增长且每层动态变化。KV quant 需要在线确定 scale、写入低精度 cache，并在 attention 中高效 dequant/compute；错误还可能沿长上下文累积。
- 面试中应先给结论，再给成本模型/瓶颈，再给适用边界。只背框架参数通常拿不到高分。

02 | 关键公式 / 成本模型

公式 Weight quant cost 多在 load/dequant；KV quant cost 包含 online quantize + cache store + repeated read/dequant。

03 | Know-Why：为什么成立

- 1. 权重静态、可离线校准与预打包
- 2. KV 是每请求在线生成、随 context 增长且每层动态变化。KV quant 需要在线确定 scale、写入低精度 cache，并在 attention 中高效 dequant/compute
- 3. 错误还可能沿长上下文累积。
- 把这一结论放进 Roofline / 内存容量 / 调度 / 通信四类模型中检查，确认瓶颈是否真的位于关键路径，而不是只优化了一个非关键算子。

04 | 工程场景

现场推演 权重 INT4 已非常小，但 128K context 的 KV 仍可能占据主要显存，因此两者需要独立规划。

05 | 常见错误

- ✗ 直接套 GPTQ/AWQ 到 KV
- ✗ 这类方法并不针对动态 KV。

06 | 追问链

- → KV scale 按 token/head/channel 哪种更合理？
- → 如何避免量化写入成为 bottleneck？
- → 长上下文怎么验证？

07 | 面试官评分标准

- 及格：能给出正确概念和基本方向。
- 良好：能写出成本/显存/通信公式，能解释为什么。
- 优秀：能指出反例、适用边界，并能把问题落到 profiler、SLO 或真实系统配置。

Q049 量化后的模型如何正确评估？

难度 ★★★★★ · 章节 量化与低精度推理 · 关键词 quant-eval, quality

30 秒回答 必须同时评估质量与系统收益：perplexity/任务准确率、reasoning/code/multilingual、长上下文；系统侧测 TTFT/TPOT/throughput/VRAM/cost。量化方案只有在目标 workload 的速度-质量 Pareto 上占优才算成功。

01 | 考察本质

- 这道题不是考名词定义，而是看你能否从系统资源出发解释：必须同时评估质量与系统收益：perplexity/任务准确率、reasoning/code/multilingual、长上下文；系统侧测 TTFT/TPOT/throughput/VRAM/cost。量化方案只有在目标 workload 的速度-质量 Pareto 上占优才算成功。
- 面试中应先给结论，再给成本模型/瓶颈，再给适用边界。只背框架参数通常拿不到高分。

02 | 关键公式 / 成本模型

公式 Compare Δ quality 与 speedup、memory_saved、goodput_gain，而非单一指标。

03 | Know-Why：为什么成立

- 1. 必须同时评估质量与系统收益：perplexity/任务准确率、reasoning/code/multilingual、长上下文
- 2. 系统侧测 TTFT/TPOT/throughput/VRAM/cost。量化方案只有在目标 workload 的速度-质量 Pareto 上占优才算成功。
- 把这一结论放进 Roofline / 内存容量 / 调度 / 通信四类模型中检查，确认瓶颈是否真的位于关键路径，而不是只优化了一个非关键算子。

04 | 工程场景

现场推演 某 INT4 在 WikiText perplexity 几乎不变，却可能在数学推理和长 context 上明显退化。

05 | 常见错误

- **✗** 只测短文本 perplexity
- **✗** benchmark 使用与线上不同 kernel/框架。

06 | 追问链

- → 如何设回归阈值？
- → 采样随机性如何控制？
- → 为什么 structured output 也应测试？

07 | 面试官评分标准

- 及格：能给出正确概念和基本方向。
- 良好：能写出成本/显存/通信公式，能解释为什么。
- 优秀：能指出反例、适用边界，并能把问题落到 profiler、SLO 或真实系统配置。

Q050 为什么 INT4 模型有时候不比 FP16 快？

难度 ★★★★★ · 章节 量化与低精度推理 · 关键词 INT4, performance

30 秒回答 容量/带宽减少并不自动等价于端到端延迟降低。若 dequant、unpack、scale load、kernel launch 或不友好的 shape 抵消收益；或工作负载已 compute/attention/communication-bound，权重压缩不会解决关键路径。

01 | 考察本质

- 这道题不是考名词定义，而是看你能否从系统资源出发解释：容量/带宽减少并不自动等价于端到端延迟降低。若 dequant、unpack、scale load、kernel launch 或不友好的 shape 抵消收益；或工作负载已 compute/attention/communication-bound，权重压缩不会解决关键路径。
- 面试中应先给结论，再给成本模型/瓶颈，再给适用边界。只背框架参数通常拿不到高分。

02 | 关键公式 / 成本模型

公式 $\text{Speedup} \approx \text{baseline_critical_path} / \text{new_critical_path}$ ；只缩短非关键项不会提速。

03 | Know-Why：为什么成立

- 1. 容量/带宽减少并不自动等价于端到端延迟降低。若 dequant、unpack、scale load、kernel launch 或不友好的 shape 抵消收益
- 2. 或工作负载已 compute/attention/communication-bound，权重压缩不会解决关键路径。
- 把这一结论放进 Roofline / 内存容量 / 调度 / 通信四类模型中检查，确认瓶颈是否真的位于关键路径，而不是只优化了一个非关键算子。

04 | 工程场景

现场推演 大 batch Prefill 已被 Tensor Core GEMM 限制，某 W4A16 kernel 需要先解码成 FP16，反而可能慢于原生 FP16/FP8。

05 | 常见错误

- **✗** 用模型文件大小推断性能
- **✗** 不看实际 kernel backend。

06 | 追问链

- → 如何识别 dequant overhead?
- → 什么时候 FP8 比 INT4 更合适?
- → 如何做 microbenchmark 与 E2E 对照?

07 | 面试官评分标准

- 及格：能给出正确概念和基本方向。
- 良好：能写出成本/显存/通信公式，能解释为什么。
- 优秀：能指出反例、适用边界，并能把问题落到 profiler、SLO 或真实系统配置。

CHAPTER 06

分布式推理与通信

掌握 TP/PP/DP/EP/CP，理解 NVLink/IB 拓扑与 collective 的成本模型。

本章题目 Q051 - Q060 · 共 10 题

本章学习目标

- 能从拓扑选择 TP/PP/DP/EP/CP。
- 会估 collective 与 overlap。
- 能设计多节点 MoE/长上下文部署。

建议刷题方法：先用 30 秒回答自测，再遮住正文推导公式，最后只看“追问链”进行模拟面试。

Q051 Tensor Parallelism 是什么？

难度 ★★★★★ · 章节 分布式推理与通信 · 关键词 TP, distributed

30 秒回答 TP 把同一层的大矩阵沿行/列切到多 GPU 并行，每层都需要某些 AllReduce/AllGather/ReduceScatter 来组合结果。优点是单请求可同时用多卡、降低单卡权重容量；代价是高频同步，非常依赖 scale-up interconnect。

01 | 考察本质

- 这道题不是考名词定义，而是看你能否从系统资源出发解释：TP 把同一层的大矩阵沿行/列切到多 GPU 并行，每层都需要某些 AllReduce/AllGather/ReduceScatter 来组合结果。优点是单请求可同时用多卡、降低单卡权重容量；代价是高频同步，非常依赖 scale-up interconnect。
- 面试中应先给结论，再给成本模型/瓶颈，再给适用边界。只背框架参数通常拿不到高分。

02 | 关键公式 / 成本模型

公式 $Linear Y=XW$ ；W column/row sharding 使每 rank 计算局部结果，再 collective。

03 | Know-Why：为什么成立

- 1. TP 把同一层的大矩阵沿行/列切到多 GPU 并行，每层都需要某些 AllReduce/AllGather/ReduceScatter 来组合结果。优点是单请求可同时用多卡、降低单卡权重容量
- 2. 代价是高频同步，非常依赖 scale-up interconnect。
- 把这一结论放进 Roofline / 内存容量 / 调度 / 通信四类模型中检查，确认瓶颈是否真的位于关键路径，而不是只优化了一个非关键算子。

04 | 工程场景

现场推演 8×H100 同节点 NVLink 上 TP=8 可能可行；跨慢网络 TP=8 通信可能占主导。

05 | 常见错误

- ✗ 认为 TP 数越大 latency 越低
- ✗ collective 与 small GEMM 会导致扩展效率下降。

06 | 追问链

- → Column-parallel 与 row-parallel 各需要什么 collective？
- → Attention heads 如何切？
- → TP 与 GQA 有何约束？

07 | 面试官评分标准

- 及格：能给出正确概念和基本方向。
- 良好：能写出成本/显存/通信公式，能解释为什么。
- 优秀：能指出反例、适用边界，并能把问题落到 profiler、SLO 或真实系统配置。

Q052 Pipeline Parallelism 是什么？

难度 ★★★★★ · 章节 分布式推理与通信 · 关键词 PP, pipeline

30 秒回答 PP 按层把模型切成多个 stage，请求 activation 在 stage 间传递。通信字节通常小于 TP 高频 collective，但存在 pipeline bubble、stage imbalance 与单请求串行经过各 stage 的延迟。Serving 中需用 microbatch/interleaving 才能提升利用率。

01 | 考察本质

- 这道题不是考名词定义，而是看你能否从系统资源出发解释：PP 按层把模型切成多个 stage，请求 activation 在 stage 间传递。通信字节通常小于 TP 高频 collective，但存在 pipeline bubble、stage imbalance 与单请求串行经过各 stage 的延迟。Serving 中需用 microbatch/interleaving 才能提升利用率。
- 面试中应先给结论，再给成本模型/瓶颈，再给适用边界。只背框架参数通常拿不到高分。

02 | 关键公式 / 成本模型

公式 Stage_i: layers[a:b]; activation size $\approx$ tokens $\times$ hidden $\times$ dtype。

03 | Know-Why：为什么成立

- 1. PP 按层把模型切成多个 stage，请求 activation 在 stage 间传递。通信字节通常小于 TP 高频 collective，但存在 pipeline bubble、stage imbalance 与单请求串行经过各 stage 的延迟。Serving 中需用 microbatch/interleaving 才能提升利用率。
- 把这一结论放进 Roofline / 内存容量 / 调度 / 通信四类模型中检查，确认瓶颈是否真的位于关键路径，而不是只优化了一个非关键算子。

04 | 工程场景

现场推演 跨节点网络较慢时，把每节点内部做 TP、节点之间做 PP 是常见拓扑感知方案。

05 | 常见错误

- ✗ 把训练 PP 的 1F1B 直接套到自回归 serving
- ✗ 推理的 microbatch 与请求调度不同。

06 | 追问链

- → PP 如何与 continuous batching 结合？
- → stage 不均衡怎么办？
- → TP $\times$ PP 的 rank topology 怎么布置？

07 | 面试官评分标准

- 及格：能给出正确概念和基本方向。
- 良好：能写出成本/显存/通信公式，能解释为什么。
- 优秀：能指出反例、适用边界，并能把问题落到 profiler、SLO 或真实系统配置。

Q053 Data Parallel inference 有什么意义?

难度 ★★★★★ · 章节 分布式推理与通信 · 关键词 DP, replica

30 秒回答 每个 replica 持有完整模型或完整 TP/PP group，不同请求路由到不同 replica。它是最直接的横向扩容方式，适合独立请求和高 QPS；挑战在负载均衡、prefix/KV locality 与不同 replica 的队列状态。

01 | 考察本质

- 这道题不是考名词定义，而是看你能否从系统资源出发解释：每个 replica 持有完整模型或完整 TP/PP group，不同请求路由到不同 replica。它是最直接的横向扩容方式，适合独立请求和高 QPS；挑战在负载均衡、prefix/KV locality 与不同 replica 的队列状态。
- 面试中应先给结论，再给成本模型/瓶颈，再给适用边界。只背框架参数通常拿不到高分。

02 | 关键公式 / 成本模型

公式 总容量 $\approx$ replicas $\times$ 单 replica goodput，前提是请求可均衡。

03 | Know-Why：为什么成立

- 1. 每个 replica 持有完整模型或完整 TP/PP group，不同请求路由到不同 replica。它是最直接的横向扩容方式，适合独立请求和高 QPS
- 2. 挑战在负载均衡、prefix/KV locality 与不同 replica 的队列状态。
- 把这一结论放进 Roofline / 内存容量 / 调度 / 通信四类模型中检查，确认瓶颈是否真的位于关键路径，而不是只优化了一个非关键算子。

04 | 工程场景

现场推演 TP=2 的模型组复制 4 份得到 4 个 DP replica，通常比单纯 TP=8 更适合高并发。

05 | 常见错误

- ✗ 只按 round-robin 路由
- ✗ 长短请求会造成 replica straggler。

06 | 追问链

- → 如何做 KV-aware routing?
- → DP autoscaling 看 QPS 还是 token rate?
- → 不同 replica cache 如何共享?

07 | 面试官评分标准

- 及格：能给出正确概念和基本方向。
- 良好：能写出成本/显存/通信公式，能解释为什么。
- 优秀：能指出反例、适用边界，并能把问题落到 profiler、SLO 或真实系统配置。

Q054 Expert Parallelism 是什么？

难度 ★★★★★ · 章节 分布式推理与通信 · 关键词 EP, MoE

30 秒回答 EP 按 expert 维度分布 MoE 权重。router 决定 token 目标 expert，系统做 dispatch all-to-all、local grouped GEMM、再 combine。EP 让巨大的 expert 权重自然分摊，但通信模式从 TP 的 collective 变成数据依赖的 all-to-all。

01 | 考察本质

- 这道题不是考名词定义，而是看你能否从系统资源出发解释：EP 按 expert 维度分布 MoE 权重。router 决定 token 目标 expert，系统做 dispatch all-to-all、local grouped GEMM、再 combine。EP 让巨大的 expert 权重自然分摊，但通信模式从 TP 的 collective 变成数据依赖的 all-to-all。
- 面试中应先给结论，再给成本模型/瓶颈，再给适用边界。只背框架参数通常拿不到高分。

02 | 关键公式 / 成本模型

公式 token→top-k experts; $T \approx \text{dispatch} + \text{expert_compute} + \text{combine}$ 。

03 | Know-Why: 为什么成立

- 1. EP 按 expert 维度分布 MoE 权重。router 决定 token 目标 expert，系统做 dispatch all-to-all、local grouped GEMM、再 combine。EP 让巨大的 expert 权重自然分摊，但通信模式从 TP 的 collective 变成数据依赖的 all-to-all。
- 把这一结论放进 Roofline / 内存容量 / 调度 / 通信四类模型中检查，确认瓶颈是否真的位于关键路径，而不是只优化了一个非关键算子。

04 | 工程场景

现场推演 如果每 GPU 放一组 experts，token 可能跨节点发送；expert placement 与网络拓扑直接影响性能。

05 | 常见错误

- ✗ 把 EP 当成 “MoE 的 DP”
- ✗ token 实际会跨 rank。

06 | 追问链

- → Top-k 如何改变通信量？
- → expert replication 有何用？
- → EP 与 TP 如何组合？

07 | 面试官评分标准

- 及格：能给出正确概念和基本方向。
- 良好：能写出成本/显存/通信公式，能解释为什么。
- 优秀：能指出反例、适用边界，并能把问题落到 profiler、SLO 或真实系统配置。

Q055 Context Parallelism 为什么越来越重要?

难度 ★★★★★ · 章节 分布式推理与通信 · 关键词 CP, long-context

30秒回答 超长 context 的 Prefill compute 和 KV 容量可能超过单 GPU。CP 沿 sequence/context 维切分，使多个 GPU 分担 attention/KV；Prefill CP 关注长 prompt 并行，Decode CP 还要高效聚合分片 KV 的 attention 结果。

01 | 考察本质

- 这道题不是考名词定义，而是看你能否从系统资源出发解释：超长 context 的 Prefill compute 和 KV 容量可能超过单 GPU。CP 沿 sequence/context 维切分，使多个 GPU 分担 attention/KV；Prefill CP 关注长 prompt 并行，Decode CP 还要高效聚合分片 KV 的 attention 结果。
- 面试中应先给结论，再给成本模型/瓶颈，再给适用边界。只背框架参数通常拿不到高分。

02 | 关键公式 / 成本模型

公式 Context T 分到 C ranks，每 rank 持有约 T/C 的 KV；需额外通信聚合 attention。

03 | Know-Why：为什么成立

- 1. 超长 context 的 Prefill compute 和 KV 容量可能超过单 GPU。CP 沿 sequence/context 维切分，使多个 GPU 分担 attention/KV
- 2. Prefill CP 关注长 prompt 并行，Decode CP 还要高效聚合分片 KV 的 attention 结果。
- 把这一结论放进 Roofline / 内存容量 / 调度 / 通信四类模型中检查，确认瓶颈是否真的位于关键路径，而不是只优化了一个非关键算子。

04 | 工程场景

现场推演 1M token context 即使模型权重可放下，KV 也可能无法单卡容纳，CP 成为容量手段。

05 | 常见错误

- ✗ 认为 CP 免费把显存除 C
- ✗ attention 需要跨分片规约，通信不可忽略。

06 | 追问链

- → Ring attention/sequence parallel 与 CP 的关系？
- → Decode CP 的通信是什么？
- → CP 和 P/D 怎么组合？

07 | 面试官评分标准

- 及格：能给出正确概念和基本方向。
- 良好：能写出成本/显存/通信公式，能解释为什么。
- 优秀：能指出反例、适用边界，并能把问题落到 profiler、SLO 或真实系统配置。

Q056 TP 和 PP 怎么选?

难度 ★★★★★ · 章节 分布式推理与通信 · 关键词 TP,PP,topology

30 秒回答 看模型容量、单请求 latency、节点内/跨节点带宽与目标并发。高带宽 NVLink 域内适合 TP；跨节点若 AllReduce 成本太高，可增加 PP 或 DP。最常见是 topology-aware hybrid：节点内 TP，节点间 PP/EP/DP。

01 | 考察本质

- 这道题不是考名词定义，而是看你能否从系统资源出发解释：看模型容量、单请求 latency、节点内/跨节点带宽与目标并发。高带宽 NVLink 域内适合 TP；跨节点若 AllReduce 成本太高，可增加 PP 或 DP。最常见是 topology-aware hybrid：节点内 TP，节点间 PP/EP/DP。
- 面试中应先给结论，再给成本模型/瓶颈，再给适用边界。只背框架参数通常拿不到高分。

02 | 关键公式 / 成本模型

公式 Scale-up bandwidth 高 → TP；scale-out bandwidth/latency 较差 → 减少高频 collective。

03 | Know-Why：为什么成立

- 1. 看模型容量、单请求 latency、节点内/跨节点带宽与目标并发。高带宽 NVLink 域内适合 TP
- 2. 跨节点若 AllReduce 成本太高，可增加 PP 或 DP。最常见是 topology-aware hybrid：节点内 TP，节点间 PP/EP/DP。
- 把这一结论放进 Roofline / 内存容量 / 调度 / 通信四类模型中检查，确认瓶颈是否真的位于关键路径，而不是只优化了一个非关键算子。

04 | 工程场景

现场推演 2 节点 × 8 GPU，可考虑每节点 TP=8、PP=2，而不是 TP=16 跨 IB。

05 | 常见错误

- ✗ 只看显存是否放得下
- ✗ 忽略 collective frequency 与 batch。

06 | 追问链

- → 如何估算 TP AllReduce 时间？
- → PP bubble 如何估算？
- → 模型有 MoE 时怎么改？

07 | 面试官评分标准

- 及格：能给出正确概念和基本方向。
- 良好：能写出成本/显存/通信公式，能解释为什么。
- 优秀：能指出反例、适用边界，并能把问题落到 profiler、SLO 或真实系统配置。

Q057 LLM Tensor Parallel 最常出现哪些 collective?

难度 ★★★★★ · 章节 分布式推理与通信 · 关键词 NCCL,collectives

30 秒回答 主要是 AllReduce、AllGather、ReduceScatter，具体取决于 row/column parallel、sequence parallel 与 attention/MLP 切法。理解“哪一步传多少字节、每层几次”比死记框架配置更重要。

01 | 考察本质

- 这道题不是考名词定义，而是看你能否从系统资源出发解释：主要是 AllReduce、AllGather、ReduceScatter，具体取决于 row/column parallel、sequence parallel 与 attention/MLP 切法。理解“哪一步传多少字节、每层几次”比死记框架配置更重要。
- 面试中应先给结论，再给成本模型/瓶颈，再给适用边界。只背框架参数通常拿不到高分。

02 | 关键公式 / 成本模型

公式 $T_{\text{comm}} \approx \alpha \cdot \text{messages} + \text{bytes}/\text{effective_bandwidth}$ ；小消息 latency α 主导，大消息 bandwidth 主导。

03 | Know-Why：为什么成立

- 1. 主要是 AllReduce、AllGather、ReduceScatter，具体取决于 row/column parallel、sequence parallel 与 attention/MLP 切法。理解“哪一步传多少字节、每层几次”比死记框架配置更重要。
- 把这一结论放进 Roofline / 内存容量 / 调度 / 通信四类模型中检查，确认瓶颈是否真的位于关键路径，而不是只优化了一个非关键算子。

04 | 工程场景

现场推演 Decode 的 token batch 小，collective payload 可能较小，此时 latency 比带宽更关键；Prefill 大矩阵则更看 bandwidth。

05 | 常见错误

- ✗ 把 NCCL bandwidth 峰值当实际
- ✗ 忽略算法 ring/tree 与拓扑。

06 | 追问链

- → AllReduce 与 ReduceScatter+AllGather 的关系？
- → 为什么 sequence parallel 常配 ReduceScatter？
- → NVLS 有何作用？

07 | 面试官评分标准

- 及格：能给出正确概念和基本方向。
- 良好：能写出成本/显存/通信公式，能解释为什么。
- 优秀：能指出反例、适用边界，并能把问题落到 profiler、SLO 或真实系统配置。

Q058 为什么 NVLink、PCIe、InfiniBand 拓扑会直接改变最优 Parallelism?

难度 ★★★★★ · 章节 分布式推理与通信 · 关键词 topology, NVLink, IB

30 秒回答 并行算法的通信频率和字节量固定后，真正成本由链路带宽、延迟、交换层级与 contention 决定。节点内 NVLink/NVSwitch 与跨节点 IB 的性能特征不同，所以同一个 TP=8 在不同物理映射上可能差很多。

01 | 考察本质

- 这道题不是考名词定义，而是看你能否从系统资源出发解释：并行算法的通信频率和字节量固定后，真正成本由链路带宽、延迟、交换层级与 contention 决定。节点内 NVLink/NVSwitch 与跨节点 IB 的性能特征不同，所以同一个 TP=8 在不同物理映射上可能差很多。
- 面试中应先给结论，再给成本模型/瓶颈，再给适用边界。只背框架参数通常拿不到高分。

02 | 关键公式 / 成本模型

公式 $T_{comm} \approx \text{latency_hops} + \text{bytes}/\text{BW_effective} + \text{contention}$ 。

03 | Know-Why: 为什么成立

- 1. 并行算法的通信频率和字节量固定后，真正成本由链路带宽、延迟、交换层级与 contention 决定。节点内 NVLink/NVSwitch 与跨节点 IB 的性能特征不同，所以同一个 TP=8 在不同物理映射上可能差很多。
- 把这一结论放进 Roofline / 内存容量 / 调度 / 通信四类模型中检查，确认瓶颈是否真的位于关键路径，而不是只优化了一个非关键算子。

04 | 工程场景

现场推演 把高频 AllReduce 留在同一 NVSwitch 域，跨节点只传 stage activation 或 expert traffic，是典型 topology-aware 设计。

05 | 常见错误

- **✗** 只根据 GPU 型号做方案，不画 topology 图。

06 | 追问链

- → 如何读取 `nvidia-smi topo -m`?
- → NUMA/PCIe root complex 为什么影响?
- → 多 rail IB 如何利用?

07 | 面试官评分标准

- 及格：能给出正确概念和基本方向。
- 良好：能写出成本/显存/通信公式，能解释为什么。
- 优秀：能指出反例、适用边界，并能把问题落到 profiler、SLO 或真实系统配置。

Q059 Communication/Computation Overlap 怎么做?

难度 ★★★★★ · 章节 分布式推理与通信 · 关键词 overlap, communication

30 秒回答 将独立或可分块的通信与 GEMM 放在不同 stream/pipeline 中重叠，目标把串行 $T_{\text{compute}} + T_{\text{comm}}$ 逼近 $\max(T_{\text{compute}}, T_{\text{comm}})$ 。需要细粒度 chunk、正确依赖和足够 SM/Copy Engine/网络资源，否则会资源争抢。

01 | 考察本质

- 这道题不是考名词定义，而是看你能否从系统资源出发解释：将独立或可分块的通信与 GEMM 放在不同 stream/pipeline 中重叠，目标把串行 $T_{\text{compute}} + T_{\text{comm}}$ 逼近 $\max(T_{\text{compute}}, T_{\text{comm}})$ 。需要细粒度 chunk、正确依赖和足够 SM/Copy Engine/网络资源，否则会资源争抢。
- 面试中应先给结论，再给成本模型/瓶颈，再给适用边界。只背框架参数通常拿不到高分。

02 | 关键公式 / 成本模型

公式 理想 $T \approx \max(T_{\text{compute}}, T_{\text{comm}})$ ，实际还受 synchronization 与 contention 影响。

03 | Know-Why: 为什么成立

- 1. 将独立或可分块的通信与 GEMM 放在不同 stream/pipeline 中重叠，目标把串行 $T_{\text{compute}} + T_{\text{comm}}$ 逼近 $\max(T_{\text{compute}}, T_{\text{comm}})$ 。需要细粒度 chunk、正确依赖和足够 SM/Copy Engine/网络资源，否则会资源争抢。
- 把这一结论放进 Roofline / 内存容量 / 调度 / 通信四类模型中检查，确认瓶颈是否真的位于关键路径，而不是只优化了一个非关键算子。

04 | 工程场景

现场推演 MoE 可让一批 expert GEMM 计算时异步 dispatch 下一批 token，或把 TP collective 与后续可独立计算重叠。

05 | 常见错误

- ✗ 以为异步 API 就自动 overlap
- ✗ 若 compute 占满 SM 或通信依赖未切开，仍串行。

06 | 追问链

- → 如何在 Nsight 中确认 overlap?
- → 通信 kernel 会占 SM 吗?
- → chunk 多细才合适?

07 | 面试官评分标准

- 及格：能给出正确概念和基本方向。
- 良好：能写出成本/显存/通信公式，能解释为什么。
- 优秀：能指出反例、适用边界，并能把问题落到 profiler、SLO 或真实系统配置。

Q060 一个 671B MoE 模型如何做多节点部署规划?

难度 ★★★★★ · 章节 分布式推理与通信 · 关键词 MoE,system-design

30 秒回答 先算权重/量化容量与 KV，再按 expert 数设计 EP，使大多数 expert 通信尽量落在高速域；根据每 expert GEMM 尺寸决定是否再 TP；评估 all-to-all、shared expert、routing balance、DP replica 与 P/D 分离。最后用真实输入/输出分布做 capacity test。

01 | 考察本质

- 这道题不是考名词定义，而是看你能否从系统资源出发解释：先算权重/量化容量与 KV，再按 expert 数设计 EP，使大多数 expert 通信尽量落在高速域；根据每 expert GEMM 尺寸决定是否再 TP；评估 all-to-all、shared expert、routing balance、DP replica 与 P/D 分离。最后用真实输入/输出分布做 capacity test。
- 面试中应先给结论，再给成本模型/瓶颈，再给适用边界。只背框架参数通常拿不到高分。

02 | 关键公式 / 成本模型

公式 $\text{Memory} = \text{local_expert_weights} + \text{shared/dense_weights} + \text{KV} + \text{runtime}$; $\text{Latency} = \max(\text{compute}, \text{A2A}, \text{TP collectives}) + \text{overheads}$ 。

03 | Know-Why: 为什么成立

- 1. 先算权重/量化容量与 KV，再按 expert 数设计 EP，使大多数 expert 通信尽量落在高速域
- 2. 根据每 expert GEMM 尺寸决定是否再 TP
- 3. 评估 all-to-all、shared expert、routing balance、DP replica 与 P/D 分离。最后用真实输入/输出分布做 capacity test。
- 把这一结论放进 Roofline / 内存容量 / 调度 / 通信四类模型中检查，确认瓶颈是否真的位于关键路径，而不是只优化了一个非关键算子。

04 | 工程场景

现场推演 大 EP 能把权重摊开，但如果跨节点 A2A 太慢，需要 expert replication、node-limited routing 或更合理 placement。

05 | 常见错误

- ✗ 只按总参数/显存算“能放下”就结束
- ✗ MoE 性能通常死在通信和小 GEMM。

06 | 追问链

- → EP size 怎么选？
- → 专家热度不均如何处理？
- → P/D 分离后两侧并行策略是否相同？

07 | 面试官评分标准

- 及格：能给出正确概念和基本方向。
- 良好：能写出成本/显存/通信公式，能解释为什么。
- 优秀：能指出反例、适用边界，并能把问题落到 profiler、SLO 或真实系统配置。

CHAPTER 07

Speculative Decoding

理解 draft-verify、acceptance、Medusa/EAGLE/MTP，以及何时加速、何时负优化。

本章题目 Q061 - Q070 · 共 10 题

本章学习目标

- 理解 lossless speculative sampling。
- 会根据 acceptance 与 draft cost 判断收益。
- 能比较 N-gram/Medusa/EAGLE/MTP。

建议刷题方法：先用 30 秒回答自测，再遮住正文推导公式，最后只看“追问链”进行模拟面试。

Q061 Speculative Decoding 的基本原理是什么？

难度 ★★★★★ · 章节 Speculative Decoding · 关键词 speculative-decoding, draft

30 秒回答 用更便宜的 proposer/draft 一次猜多个 token，再让 target 用一次或少数 forward 并行验证；若平均接受多个 token，就减少 target 的串行 decode step。核心不是“少算 target FLOPs”，而是用一次较大的验证替代多次小的 memory-bound decode。

01 | 考察本质

- 这道题不是考名词定义，而是看你能否从系统资源出发解释：用更便宜的 proposer/draft 一次猜多个 token，再让 target 用一次或少数 forward 并行验证；若平均接受多个 token，就减少 target 的串行 decode step。核心不是“少算 target FLOPs”，而是用一次较大的验证替代多次小的 memory-bound decode。
- 面试中应先给结论，再给成本模型/瓶颈，再给适用边界。只背框架参数通常拿不到高分。

02 | 关键公式 / 成本模型

公式 Speedup 取决于 $\text{accepted_tokens} / (\text{draft_cost} + \text{verify_cost})$ 。

03 | Know-Why：为什么成立

- 1. 用更便宜的 proposer/draft 一次猜多个 token，再让 target 用一次或少数 forward 并行验证
- 2. 若平均接受多个 token，就减少 target 的串行 decode step。核心不是“少算 target FLOPs”，而是用一次较大的验证替代多次小的 memory-bound decode。
- 把这一结论放进 Roofline / 内存容量 / 调度 / 通信四类模型中检查，确认瓶颈是否真的位于关键路径，而不是只优化了一个非关键算子。

04 | 工程场景

现场推演 draft 每轮提 5 个 token，target 平均接受 4 个，则一次 target 验证推进约 4 token，而非 1 token。

05 | 常见错误

- ✗ 把 draft 输出直接当最终答案
- ✗ 标准 speculative sampling 需要正确接受/拒绝规则。

06 | 追问链

- → 为什么它特别适合 memory-bound decode?
- → 验证为什么可以并行多个 token?
- → 采样分布如何保持一致?

07 | 面试官评分标准

- 及格：能给出正确概念和基本方向。
- 良好：能写出成本/显存/通信公式，能解释为什么。
- 优秀：能指出反例、适用边界，并能把问题落到 profiler、SLO 或真实系统配置。

Q062 为什么标准 speculative sampling 可以保持输出分布不变?

难度 ★★★★★ · 章节 Speculative Decoding · 关键词 speculative-sampling, correctness

30 秒回答 算法根据 target 与 draft 的概率比接受 proposal；拒绝时从修正后的 residual distribution 采样，使边缘分布严格等于 target。它不是 greedy heuristic，而是一种带 rejection correction 的采样算法。

01 | 考察本质

- 这道题不是考名词定义，而是看你能否从系统资源出发解释：算法根据 target 与 draft 的概率比接受 proposal；拒绝时从修正后的 residual distribution 采样，使边缘分布严格等于 target。它不是 greedy heuristic，而是一种带 rejection correction 的采样算法。
- 面试中应先给结论，再给成本模型/瓶颈，再给适用边界。只背框架参数通常拿不到高分。

02 | 关键公式 / 成本模型

公式 接受概率通常与 $\min(1, p(x)/q(x))$ 相关；拒绝后从正差分布进行修正采样。

03 | Know-Why: 为什么成立

- 1. 算法根据 target 与 draft 的概率比接受 proposal
- 2. 拒绝时从修正后的 residual distribution 采样，使边缘分布严格等于 target。它不是 greedy heuristic，而是一种带 rejection correction 的采样算法。
- 把这一结论放进 Roofline / 内存容量 / 调度 / 通信四类模型中检查，确认瓶颈是否真的位于关键路径，而不是只优化了一个非关键算子。

04 | 工程场景

现场推演 即使 draft 与 target 分布不同，只要按正确规则接受/修正，最终样本仍来自 target 分布。

05 | 常见错误

- ✗ 只在 greedy decoding 下解释“相同输出”
- ✗ 忽略 stochastic sampling 的数学保证。

06 | 追问链

- → 如果只做 argmax verification 是否仍 lossless?
- → temperature/top-p 如何处理?
- → draft tokenizer 不同怎么办?

07 | 面试官评分标准

- 及格：能给出正确概念和基本方向。
- 良好：能写出成本/显存/通信公式，能解释为什么。
- 优秀：能指出反例、适用边界，并能把问题落到 profiler、SLO 或真实系统配置。

Q063 Acceptance Rate 为什么是 Spec Decode 最关键指标之一？

难度 ★★★★★ · 章节 Speculative Decoding · 关键词 acceptance, spec-decode

30 秒回答 它直接决定每次昂贵 target verification 能推进多少 token。acceptance 高但 draft 太慢、或 acceptance 低但 draft 很便宜，都需要综合成本；因此更准确指标是平均 accepted length 与单位时间推进率。

01 | 考察本质

- 这道题不是考名词定义，而是看你能否从系统资源出发解释：它直接决定每次昂贵 target verification 能推进多少 token。acceptance 高但 draft 太慢、或 acceptance 低但 draft 很便宜，都需要综合成本；因此更准确指标是平均 accepted length 与单位时间推进率。
- 面试中应先给结论，再给成本模型/瓶颈，再给适用边界。只背框架参数通常拿不到高分。

02 | 关键公式 / 成本模型

公式 $\text{effective_tokens_per_target_call} \approx E[\text{accepted_prefix_length}] + \text{可能的 correction token}。$

03 | Know-Why: 为什么成立

- 1. 它直接决定每次昂贵 target verification 能推进多少 token。acceptance 高但 draft 太慢、或 acceptance 低但 draft 很便宜，都需要综合成本
- 2. 因此更准确指标是平均 accepted length 与单位时间推进率。
- 把这一结论放进 Roofline / 内存容量 / 调度 / 通信四类模型中检查，确认瓶颈是否真的位于关键路径，而不是只优化了一个非关键算子。

04 | 工程场景

现场推演 代码/模板重复场景通常更可预测，acceptance 可能高；开放式高温生成 acceptance 往往下降。

05 | 常见错误

- **✗** 只报 acceptance percentage，不报 draft/verify latency。

06 | 追问链

- → 如何在线调 speculative length?
- → 高温为什么影响 acceptance?
- → 不同 domain 是否应该用不同 draft?

07 | 面试官评分标准

- 及格：能给出正确概念和基本方向。
- 良好：能写出成本/显存/通信公式，能解释为什么。
- 优秀：能指出反例、适用边界，并能把问题落到 profiler、SLO 或真实系统配置。

Q064 Draft Model 为什么不能越小越好？

难度 ★★★★★ · 章节 Speculative Decoding · 关键词 draft-model, tradeoff

30 秒回答 更小 draft 计算便宜，但和 target 分布偏差大，acceptance 降低；更大 draft 更准确却消耗更多算力/显存。最优点取决于 target 大小、batch、硬件、输出域和 speculative length。

01 | 考察本质

- 这道题不是考名词定义，而是看你能否从系统资源出发解释：更小 draft 计算便宜，但和 target 分布偏差大，acceptance 降低；更大 draft 更准确却消耗更多算力/显存。最优点取决于 target 大小、batch、硬件、输出域和 speculative length。
- 面试中应先给结论，再给成本模型/瓶颈，再给适用边界。只背框架参数通常拿不到高分。

02 | 关键公式 / 成本模型

公式 优化目标 $\max E[\text{accepted_tokens}] / (T_{\text{draft}} + T_{\text{verify}})$ 。

03 | Know-Why：为什么成立

- 1. 更小 draft 计算便宜，但和 target 分布偏差大，acceptance 降低
- 2. 更大 draft 更准确却消耗更多算力/显存。最优点取决于 target 大小、batch、硬件、输出域和 speculative length。
- 把这一结论放进 Roofline / 内存容量 / 调度 / 通信四类模型中检查，确认瓶颈是否真的位于关键路径，而不是只优化了一个非关键算子。

04 | 工程场景

现场推演 若 draft 从 7B 换 1B 速度 3×，但平均接受从 4 token 降到 1.2 token，整体可能反而变慢。

05 | 常见错误

- ✗ 只用参数量判断 draft 好坏
- ✗ 训练/蒸馏与 tokenizer 兼容也很重要。

06 | 追问链

- → 如何选择 draft/target ratio？
- → draft 与 target 是否必须同词表？
- → 共享层/feature 能否替代完整 draft？

07 | 面试官评分标准

- 及格：能给出正确概念和基本方向。
- 良好：能写出成本/显存/通信公式，能解释为什么。
- 优秀：能指出反例、适用边界，并能把问题落到 profiler、SLO 或真实系统配置。

Q065 N-gram / Suffix speculation 为什么不需要 Draft Model？

难度 ★★★★★ · 章节 Speculative Decoding · 关键词 ngram, speculation

30 秒回答 它从当前上下文中寻找已出现的 token n-gram/suffix 模式，用其后续 token 作为 proposal。对代码、JSON、复

制型文本成本极低，不需要额外权重与 KV；泛化能力则明显弱于 learned draft。

01 | 考察本质

- 这道题不是考名词定义，而是看你能否从系统资源出发解释：它从当前上下文中寻找已出现的 token n-gram/suffix 模式，用其后续 token 作为 proposal。对代码、JSON、复制型文本成本极低，不需要额外权重与 KV；泛化能力则明显弱于 learned draft。
- 面试中应先给结论，再给成本模型/瓶颈，再给适用边界。只背框架参数通常拿不到高分。

02 | 关键公式 / 成本模型

公式 proposal 来自 context reuse，而非神经网络 forward。

03 | Know-Why：为什么成立

- 1. 它从当前上下文中寻找已出现的 token n-gram/suffix 模式，用其后续 token 作为 proposal。对代码、JSON、复制型文本成本极低，不需要额外权重与 KV
- 2. 泛化能力则明显弱于 learned draft。
- 把这一结论放进 Roofline / 内存容量 / 调度 / 通信四类模型中检查，确认瓶颈是否真的位于关键路径，而不是只优化了一个非关键算子。

04 | 工程场景

现场推演 生成结构化 JSON 或复制文档片段时，上下文已有相同模式，n-gram proposal 很可能连续命中。

05 | 常见错误

- ✗ 认为它适合所有自然语言
- ✗ 在高熵文本上命中率可能太低。

06 | 追问链

- → 如何实现高效 suffix lookup?
- → 与 prefix cache 是否共享数据结构?
- → 什么时候自动关闭?

07 | 面试官评分标准

- 及格：能给出正确概念和基本方向。
- 良好：能写出成本/显存/通信公式，能解释为什么。
- 优秀：能指出反例、适用边界，并能把问题落到 profiler、SLO 或真实系统配置。

Q066 Medusa 是什么?

难度 ★★★★★ · 章节 Speculative Decoding · 关键词 Medusa,spec-decode

30 秒回答 Medusa 在 target backbone 上增加多个轻量预测 head，分别预测未来多个位置，再构造 candidate tree 由 target-style attention 验证。它避免维护完整独立 draft 模型，减少额外显存与服务复杂度。

01 | 考察本质

- 这道题不是考名词定义，而是看你能否从系统资源出发解释：Medusa 在 target backbone 上增加多个轻量预测 head，分别预测未来多个位置，再构造 candidate tree 由 target-style attention 验证。它避免维护完整独立 draft 模型，减少额外显存与服务复杂度。
- 面试中应先给结论，再给成本模型/瓶颈，再给适用边界。只背框架参数通常拿不到高分。

02 | 关键公式 / 成本模型

公式 shared backbone + multiple future-token heads → candidate tree → tree verification。

03 | Know-Why：为什么成立

- 1. Medusa 在 target backbone 上增加多个轻量预测 head，分别预测未来多个位置，再构造 candidate tree 由 target-style attention 验证。它避免维护完整独立 draft 模型，减少额外显存与服务复杂度。
- 把这一结论放进 Roofline / 内存容量 / 调度 / 通信四类模型中检查，确认瓶颈是否真的位于关键路径，而不是只优化了一个非关键算子。

04 | 工程场景

现场推演 部署一个 70B target 时，不再额外常驻一个数十亿参数 draft，而是加载小 head。

05 | 常见错误

- ✗ 把多个 head 当作独立 step 串行预测
- ✗ 其价值在并行 proposal 和 tree verification。

06 | 追问链

- → Medusa head 如何训练？
- → tree width/depth 如何影响？
- → 与 EAGLE 的 feature prediction 有何差异？

07 | 面试官评分标准

- 及格：能给出正确概念和基本方向。
- 良好：能写出成本/显存/通信公式，能解释为什么。
- 优秀：能指出反例、适用边界，并能把问题落到 profiler、SLO 或真实系统配置。

Q067 EAGLE 与普通 Draft Model 的关键区别？

难度 ★★★★★ · 章节 Speculative Decoding · 关键词 EAGLE, speculation

30 秒回答 EAGLE 类方法利用 target 的内部 hidden feature，预测未来 feature/token，再让 target 验证；因为 feature 比纯 token 分布包含更多目标模型信息，proposal 质量通常更高。EAGLE-2 进一步根据置信度动态扩展树。

01 | 考察本质

- 这道题不是考名词定义，而是看你能否从系统资源出发解释：EAGLE 类方法利用 target 的内部 hidden feature，预测未来 feature/token，再让 target 验证；因为 feature 比纯 token 分布包含更多目标模型信息，proposal 质量通常更高。EAGLE-2 进一步根据置信度动态扩展树。
- 面试中应先给结论，再给成本模型/瓶颈，再给适用边界。只背框架参数通常拿不到高分。

02 | 关键公式 / 成本模型

公式 feature-conditioned draft → dynamic tree → target verification。

03 | Know-Why：为什么成立

- 1. EAGLE 类方法利用 target 的内部 hidden feature，预测未来 feature/token，再让 target 验证
- 2. 因为 feature 比纯 token 分布包含更多目标模型信息，proposal 质量通常更高。EAGLE-2 进一步根据置信度动态扩展树。
- 把这一结论放进 Roofline / 内存容量 / 调度 / 通信四类模型中检查，确认瓶颈是否真的位于关键路径，而不是只优化了一个非关键算子。

04 | 工程场景

现场推演 相比完全独立的小语言模型，EAGLE 与 target 表示空间更贴近，可能在相同 draft cost 下提高 acceptance。

05 | 常见错误

- **✗** 只记“EAGLE 更快”不理解需要额外训练与 runtime support。

06 | 追问链

- → 为什么 feature prediction 更容易？
- → dynamic tree 如何选择节点？
- → EAGLE 对 batch throughput 有何影响？

07 | 面试官评分标准

- 及格：能给出正确概念和基本方向。
- 良好：能写出成本/显存/通信公式，能解释为什么。
- 优秀：能指出反例、适用边界，并能把问题落到 profiler、SLO 或真实系统配置。

Q068 MTP 为什么既可以用于训练，也可以用于推理加速？

难度 ★★★★★ · 章节 Speculative Decoding · 关键词 MTP, DeepSeek

30 秒回答 Multi-Token Prediction 在训练时让模型同时预测未来多个 token，作为辅助目标提高表示学习；推理时这些额外 head/模块可直接产出 speculative proposals，因此训练目标与 serving 加速共享结构。

01 | 考察本质

- 这道题不是考名词定义，而是看你能否从系统资源出发解释：Multi-Token Prediction 在训练时让模型同时预测未来多个 token，作为辅助目标提高表示学习；推理时这些额外 head/模块可直接产出 speculative proposals，因此训练目标与 serving 加速共享结构。
- 面试中应先给结论，再给成本模型/瓶颈，再给适用边界。只背框架参数通常拿不到高分。

02 | 关键公式 / 成本模型

公式 $L = L_{\text{next-token}} + \sum_k \lambda_k L_{\{t+k\}}$ ；推理时多步预测作为 draft。

03 | Know-Why：为什么成立

- 1. Multi-Token Prediction 在训练时让模型同时预测未来多个 token，作为辅助目标提高表示学习
- 2. 推理时这些额外 head/模块可直接产出 speculative proposals，因此训练目标与 serving 加速共享结构。
- 把这一结论放进 Roofline / 内存容量 / 调度 / 通信四类模型中检查，确认瓶颈是否真的位于关键路径，而不是只优化了一个非关键算子。

04 | 工程场景

现场推演 模型原生带 MTP head 时，无需再部署完全独立 draft，可在特定框架里做 speculative decoding。

05 | 常见错误

- **✗** 把训练 MTP 等同于一次 forward 直接无损输出多个 token
- **✗** 最终仍需验证/接受机制。

06 | 追问链

- → MTP 会改变模型质量吗？
- → head 数怎么选？
- → 和 Medusa 的差别？

07 | 面试官评分标准

- 及格：能给出正确概念和基本方向。
- 良好：能写出成本/显存/通信公式，能解释为什么。
- 优秀：能指出反例、适用边界，并能把问题落到 profiler、SLO 或真实系统配置。

Q069 Tree Attention 为什么能加速 speculative verification?

难度 ★★★★★ · 章节 Speculative Decoding · 关键词 tree-attention, verification

30秒回答 多个候选序列共享相同 ancestor prefix。若逐候选验证会重复计算共享部分；Tree Attention 用树形 attention mask/共享 KV 一次处理多个分支，仅在分叉后做额外计算。

01 | 考察本质

- 这道题不是考名词定义，而是看你能否从系统资源出发解释：多个候选序列共享相同 ancestor prefix。若逐候选验证会重复计算共享部分；Tree Attention 用树形 attention mask/共享 KV 一次处理多个分支，仅在分叉后做额外计算。

- 面试中应先给结论，再给成本模型/瓶颈，再给适用边界。只背框架参数通常拿不到高分。

02 | 关键公式 / 成本模型

公式 $\text{compute} \approx \text{shared_prefix_once} + \sum \text{branch_unique_part}$ ，而非 $\sum \text{full_candidate}$ 。

03 | Know-Why: 为什么成立

- 1. 多个候选序列共享相同 ancestor prefix。若逐候选验证会重复计算共享部分
- 2. Tree Attention 用树形 attention mask/共享 KV 一次处理多个分支，仅在分叉后做额外计算。
- 把这一结论放进 Roofline / 内存容量 / 调度 / 通信四类模型中检查，确认瓶颈是否真的位于关键路径，而不是只优化了一个非关键算子。

04 | 工程场景

现场推演 候选 AB(C/D/E) 三支可复用 AB 的 KV 与 attention。

05 | 常见错误

- ✗ 树越宽越好
- ✗ 过宽会扩大 verification token 数并降低 GPU efficiency。

06 | 追问链

- → tree mask 如何构造？
- → Paged KV 如何表示分支？
- → beam search 可否用类似思想？

07 | 面试官评分标准

- 及格：能给出正确概念和基本方向。
- 良好：能写出成本/显存/通信公式，能解释为什么。
- 优秀：能指出反例、适用边界，并能把问题落到 profiler、SLO 或真实系统配置。

Q070 Speculative Decoding 在什么情况下反而变慢？

难度 ★★★★★ · 章节 Speculative Decoding · 关键词 spec-decode, negative-speedup

30秒回答 高 QPS 时 target 已被大 batch 充分利用，额外 draft/verification 会争夺 compute；acceptance 低、输出很短、draft 太大或 verification kernel 不好也会负优化。它通常更适合中低并发、memory-bound、长输出。

01 | 考察本质

- 这道题不是考名词定义，而是看你能否从系统资源出发解释：高 QPS 时 target 已被大 batch 充分利用，额外 draft/verification 会争夺 compute；acceptance 低、输出很短、draft 太大或 verification kernel 不好也会负优化。它通常更适合中低并发、memory-bound、长输出。
- 面试中应先给结论，再给成本模型/瓶颈，再给适用边界。只背框架参数通常拿不到高分。

02 | 关键公式 / 成本模型

公式 若 $T_{\text{draft}} + T_{\text{verify}} > \text{accepted_tokens} \times T_{\text{target_baseline}}$ ，则负收益。

03 | Know-Why：为什么成立

- 1. 高 QPS 时 target 已被大 batch 充分利用，额外 draft/verification 会争夺 compute
- 2. acceptance 低、输出很短、draft 太大或 verification kernel 不好也会负优化。它通常更适合中低并发、memory-bound、长输出。
- 把这一结论放进 Roofline / 内存容量 / 调度 / 通信四类模型中检查，确认瓶颈是否真的位于关键路径，而不是只优化了一个非关键算子。

04 | 工程场景

现场推演 离线大 batch 生成已是高效 GEMM，spec decode 可能把大矩阵拆成更多复杂工作，吞吐不升反降。

05 | 常见错误

- **×** 以单请求 latency speedup 推断 cluster throughput。

06 | 追问链

- → 如何建立自动启停策略？
- → QPS 提升时 speculative length 是否应下降？
- → 如何与 continuous batching 联动？

07 | 面试官评分标准

- 及格：能给出正确概念和基本方向。
- 良好：能写出成本/显存/通信公式，能解释为什么。
- 优秀：能指出反例、适用边界，并能把问题落到 profiler、SLO 或真实系统配置。

CHAPTER 08

MoE、MLA 与模型-系统协同

分析 Expert Parallel、All-to-All、负载不均衡、MLA 与现代 MoE serving。

本章题目 Q071 - Q080 · 共 10 题

本章学习目标

- 能把 MoE 的瓶颈从 FLOPs 转为 all-to-all 与 load balance。
- 理解 MLA/Shared Expert 的系统意义。
- 具备模型-系统协同思维。

建议刷题方法：先用 30 秒回答自测，再遮住正文推导公式，最后只看“追问链”进行模拟面试。

Q071 为什么 MoE 理论 FLOPs 很低，线上 latency 却未必低？

难度 ★★★★★ · 章节 MoE、MLA 与模型-系统协同 · 关键词 MoE,latency

30 秒回答 MoE 只激活少数 expert 降低计算，但 routing/dispatch/all-to-all、expert 小 batch、负载不均衡和同步会成为关键路径。现代 MoE 优化的核心往往是提高每 expert token 聚合、降低网络延迟并重叠通信，而非再减少 FLOPs。

01 | 考察本质

- 这道题不是考名词定义，而是看你能否从系统资源出发解释：MoE 只激活少数 expert 降低计算，但 routing/dispatch/all-to-all、expert 小 batch、负载不均衡和同步会成为关键路径。现代 MoE 优化的核心往往是提高每 expert token 聚合、降低网络延迟并重叠通信，而非再减少 FLOPs。
- 面试中应先给结论，再给成本模型/瓶颈，再给适用边界。只背框架参数通常拿不到高分。

02 | 关键公式 / 成本模型

公式 $T \approx T_{\text{router}} + T_{\text{A2A_dispatch}} + \max_i T_{\text{expert}_i} + T_{\text{A2A_combine}}$ 。

03 | Know-Why：为什么成立

- 1. MoE 只激活少数 expert 降低计算，但 routing/dispatch/all-to-all、expert 小 batch、负载不均衡和同步会成为关键路径。现代 MoE 优化的核心往往是提高每 expert token 聚合、降低网络延迟并重叠通信，而非再减少 FLOPs。
- 把这一结论放进 Roofline / 内存容量 / 调度 / 通信四类模型中检查，确认瓶颈是否真的位于关键路径，而不是只优化了一个非关键算子。

04 | 工程场景

现场推演 active params 只有 37B 的大 MoE，如果每层都跨节点 all-to-all 且 token 分散，可能比 dense 37B 难服务得多。

05 | 常见错误

- ✗ 只用 FLOPs/token 比较 Dense 与 MoE。

06 | 追问链

- 低并发为什么尤其差？
- 如何做 expert placement？
- 如何通过 grouped GEMM 提升？

07 | 面试官评分标准

- 及格：能给出正确概念和基本方向。
- 良好：能写出成本/显存/通信公式，能解释为什么。
- 优秀：能指出反例、适用边界，并能把问题落到 profiler、SLO 或真实系统配置。

Q072 Expert Parallel 和 Tensor Parallel 的本质区别？

难度 ★★★★★ · 章节 MoE、MLA 与模型-系统协同 · 关键词 EP,TP,MoE

30 秒回答 TP 切一个矩阵本身，所有 token 通常参与同步 collective；EP 切不同 experts，只有被路由到某 expert 的 token

发送到对应 rank，通信是数据依赖的 all-to-all。两者可叠加：expert 内部做 TP，experts 之间做 EP。

01 | 考察本质

- 这道题不是考名词定义，而是看你能否从系统资源出发解释：TP 切一个矩阵本身，所有 token 通常参与同步 collective；EP 切不同 experts，只有被路由到某 expert 的 token 发送到对应 rank，通信是数据依赖的 all-to-all。两者可叠加：expert 内部做 TP，experts 之间做 EP。
- 面试中应先给结论，再给成本模型/瓶颈，再给适用边界。只背框架参数通常拿不到高分。

02 | 关键公式 / 成本模型

公式 TP communication 与 layer structure 固定；EP traffic 由 routing distribution 决定。

03 | Know-Why：为什么成立

- 1. TP 切一个矩阵本身，所有 token 通常参与同步 collective
- 2. EP 切不同 experts，只有被路由到某 expert 的 token 发送到对应 rank，通信是数据依赖的 all-to-all。两者可叠加：expert 内部做 TP，experts 之间做 EP。
- 把这一结论放进 Roofline / 内存容量 / 调度 / 通信四类模型中检查，确认瓶颈是否真的位于关键路径，而不是只优化了一个非关键算子。

04 | 工程场景

现场推演 如果一个 expert 太大单卡放不下，可 EP 分 expert、再对单 expert TP。

05 | 常见错误

- ✗ 把 EP size 直接等价于 expert 数
- ✗ 一个 rank 可放多个 expert，也可复制 expert。

06 | 追问链

- → EP×TP rank 怎么布局？
- → 何时 expert replication 优于更大 EP？
- → shared expert 属于哪一层？

07 | 面试官评分标准

- 及格：能给出正确概念和基本方向。
- 良好：能写出成本/显存/通信公式，能解释为什么。
- 优秀：能指出反例、适用边界，并能把问题落到 profiler、SLO 或真实系统配置。

Q073 MoE Expert Load Imbalance 为什么严重？

难度 ★★★★★ · 章节 MoE、MLA 与模型-系统协同 · 关键词 load-balance, MoE

30 秒回答 一个 step 的完成时间由最慢/最拥挤 expert 决定。即使平均 token 数均匀，只要 tail expert 过载就形成 straggler；并且小 expert batch 也会降低 GEMM 效率。训练侧的 balance loss 不保证线上特定流量完全平衡。

01 | 考察本质

- 这道题不是考名词定义，而是看你能否从系统资源出发解释：一个 step 的完成时间由最慢/最拥挤 expert 决定。即使平均 token 数均匀，只要 tail expert 过载就形成 straggler；并且小 expert batch 也会降低 GEMM 效率。训练侧的 balance loss 不保证线上特定流量完全平衡。
- 面试中应先给结论，再给成本模型/瓶颈，再给适用边界。只背框架参数通常拿不到高分。

02 | 关键公式 / 成本模型

公式 $T_{\text{step}} \approx \max_i (f(\text{tokens}_i) + \text{communication})$ ； $\text{variance}(\text{tokens}_i)$ 越大越危险。

03 | Know-Why：为什么成立

- 1. 一个 step 的完成时间由最慢/最拥挤 expert 决定。即使平均 token 数均匀，只要 tail expert 过载就形成 straggler
- 2. 并且小 expert batch 也会降低 GEMM 效率。训练侧的 balance loss 不保证线上特定流量完全平衡。
- 把这一结论放进 Roofline / 内存容量 / 调度 / 通信四类模型中检查，确认瓶颈是否真的位于关键路径，而不是只优化了一个非关键算子。

04 | 工程场景

现场推演 代码请求可能长期偏向某些 experts，与训练平均分布不同，导致某几个 GPU 持续热点。

05 | 常见错误

- **✗ 只看平均 expert utilization**
- **✗ 应看 max/percentile 与时间相关性。**

06 | 追问链

- → capacity factor 在推理中怎么理解？
- → aux-loss-free balance 有何作用？
- → 能否动态迁移 expert？

07 | 面试官评分标准

- 及格：能给出正确概念和基本方向。
- 良好：能写出成本/显存/通信公式，能解释为什么。
- 优秀：能指出反例、适用边界，并能把问题落到 profiler、SLO 或真实系统配置。

Q074 All-to-All 为什么是 MoE 推理的关键瓶颈？

难度 ★★★★★ · 章节 MoE、MLA 与模型-系统协同 · 关键词 AllToAll, MoE

30 秒回答 每层 token 需按 router 结果从来源 rank 分发到 expert rank，再把输出发回。小 batch 时大量小消息受网络 latency 限制，大 batch 时总字节受 bandwidth 限制；跨节点还受拓扑和拥塞影响。

01 | 考察本质

- 这道题不是考名词定义，而是看你能否从系统资源出发解释：每层 token 需按 router 结果从来源 rank 分发到 expert rank，再把输出发回。小 batch 时大量小消息受网络 latency 限制，大 batch 时总字节受 bandwidth 限制；跨节点还受拓扑和拥塞影响。
- 面试中应先给结论，再给成本模型/瓶颈，再给适用边界。只背框架参数通常拿不到高分。

02 | 关键公式 / 成本模型

公式 $T_{A2A} \approx \alpha \cdot \text{peers} + \text{bytes}/\text{BW_eff}$ ，且需考虑不均匀 message sizes。

03 | Know-Why：为什么成立

- 1. 每层 token 需按 router 结果从来源 rank 分发到 expert rank，再把输出发回。小 batch 时大量小消息受网络 latency 限制，大 batch 时总字节受 bandwidth 限制
- 2. 跨节点还受拓扑和拥塞影响。
- 把这一结论放进 Roofline / 内存容量 / 调度 / 通信四类模型中检查，确认瓶颈是否真的位于关键路径，而不是只优化了一个非关键算子。

04 | 工程场景

现场推演 EP=64 时每个 rank 与很多 peer 交换小片段，若网络对小消息性能差，FLOPs 很少也会被通信吞掉。

05 | 常见错误

- ✗ 只测 NCCL AllReduce
- ✗ MoE 的 all-to-all 形状与负载完全不同。

06 | 追问链

- → 如何做 dispatch/combine overlap?
- → node-limited routing 是什么思路?
- → RDMA/NVLink 对 EP 有何影响?

07 | 面试官评分标准

- 及格：能给出正确概念和基本方向。
- 良好：能写出成本/显存/通信公式，能解释为什么。
- 优秀：能指出反例、适用边界，并能把问题落到 profiler、SLO 或真实系统配置。

Q075 Shared Expert 有什么意义?

难度 ★★★★★ · 章节 MoE、MLA 与模型-系统协同 · 关键词 shared-expert, MoE

30 秒回答 把跨 token 普遍需要的能力放入始终激活的 shared expert，把稀疏 specialist 放 routed experts，可减少某些共享知识被重复学习、改善专家分工。系统上 shared expert 是固定计算，可与 routed expert 计算/通信做 overlap。

01 | 考察本质

- 这道题不是考名词定义，而是看你能否从系统资源出发解释：把跨 token 普遍需要的能力放入始终激活的 shared expert，把稀疏 specialist 放 routed experts，可减少某些共享知识被重复学习、改善专家分工。系统上 shared expert 是固定计算，可与 routed expert 计算/通信做 overlap。
- 面试中应先给结论，再给成本模型/瓶颈，再给适用边界。只背框架参数通常拿不到高分。

02 | 关键公式 / 成本模型

公式 $\text{output} = \text{shared_expert}(x) + \sum \text{gate_i} \cdot \text{expert_i}(x)$ 。

03 | Know-Why：为什么成立

- 1. 把跨 token 普遍需要的能力放入始终激活的 shared expert，把稀疏 specialist 放 routed experts，可减少某些共享知识被重复学习、改善专家分工。系统上 shared expert 是固定计算，可与 routed expert 计算/通信做 overlap。
- 把这一结论放进 Roofline / 内存容量 / 调度 / 通信四类模型中检查，确认瓶颈是否真的位于关键路径，而不是只优化了一个非关键算子。

04 | 工程场景

现场推演 如果 shared expert 与 A2A dispatch 同时执行，可用本地计算隐藏部分网络延迟。

05 | 常见错误

- ✗ 只从模型质量解释 shared expert
- ✗ 它还改变 runtime 的可重叠工作。

06 | 追问链

- → shared expert 是否也做 TP?
- → 如何 overlap?
- → shared expert 太大有什么问题?

07 | 面试官评分标准

- 及格：能给出正确概念和基本方向。
- 良好：能写出成本/显存/通信公式，能解释为什么。
- 优秀：能指出反例、适用边界，并能把问题落到 profiler、SLO 或真实系统配置。

Q076 MLA 与 GQA 谁更省 KV Cache?

难度 ★★★★★ · 章节 MoE、MLA 与模型-系统协同 · 关键词 MLA,GQA,KV

30 秒回答 GQA 通过减少 KV head 数压缩缓存；MLA 进一步缓存低维 latent representation，通常能把 KV 压得更激进。比较不能只看名词，要代入模型具体 latent dim、RoPE dim 与缓存布局。

01 | 考察本质

- 这道题不是考名词定义，而是看你能否从系统资源出发解释：GQA 通过减少 KV head 数压缩缓存；MLA 进一步缓存低维 latent representation，通常能把 KV 压得更激进。比较不能只看名词，要代入模型具体 latent dim、RoPE dim 与缓存布局。
- 面试中应先给结论，再给成本模型/瓶颈，再给适用边界。只背框架参数通常拿不到高分。

02 | 关键公式 / 成本模型

公式 $GQA\ KV \propto H_{kv} \cdot D_{head}$ ； $MLA\ KV \approx D_{latent} + \text{部分 positional components}$ 。

03 | Know-Why：为什么成立

- 1. GQA 通过减少 KV head 数压缩缓存
- 2. MLA 进一步缓存低维 latent representation，通常能把 KV 压得更激进。比较不能只看名词，要代入模型具体 latent dim、RoPE dim 与缓存布局。
- 把这一结论放进 Roofline / 内存容量 / 调度 / 通信四类模型中检查，确认瓶颈是否真的位于关键路径，而不是只优化了一个非关键算子。

04 | 工程场景

现场推演 在相同 hidden size 下，MLA 若 latent dim 远小于 $H_{kv} \cdot D_{head}$ ，长上下文显存优势显著。

05 | 常见错误

- **✗ 忽略 MLA 需要重构 K/V 的额外计算与 kernel 复杂度。**

06 | 追问链

- → 如何从 config 计算 MLA 每 token KV？
- → 为什么 decode kernel 可能更复杂？
- → 量化 MLA cache 有何不同？

07 | 面试官评分标准

- 及格：能给出正确概念和基本方向。
- 良好：能写出成本/显存/通信公式，能解释为什么。
- 优秀：能指出反例、适用边界，并能把问题落到 profiler、SLO 或真实系统配置。

Q077 DeepSeek 类模型为什么对推理框架提出新要求？

难度 ★★★★★ · 章节 MoE、MLA 与模型-系统协同 · 关键词 DeepSeek,framework

30 秒回答 同时包含 MLA、超大 MoE、shared expert、MTP、长 context 和低精度路径，要求框架在 attention backend、expert parallel、all-to-all、spec decode、quantization 与调度层协同。仅为 dense Llama MHA 优化的 runtime 很难达到高效率。

01 | 考察本质

- 这道题不是考名词定义，而是看你能否从系统资源出发解释：同时包含 MLA、超大 MoE、shared expert、MTP、长 context 和低精度路径，要求框架在 attention backend、expert parallel、all-to-all、spec decode、quantization 与调度层协同。仅为 dense Llama MHA 优化的 runtime 很难达到高效率。
- 面试中应先给结论，再给成本模型/瓶颈，再给适用边界。只背框架参数通常拿不到高分。

02 | 关键公式 / 成本模型

公式 性能由多条关键路径共同决定：MLA attention + routed expert + shared expert + comm + MTP。

03 | Know-Why：为什么成立

- 1. 同时包含 MLA、超大 MoE、shared expert、MTP、长 context 和低精度路径，要求框架在 attention backend、expert parallel、all-to-all、spec decode、quantization 与调度层协同。仅为 dense Llama MHA 优化的 runtime 很难达到高效率。
- 把这一结论放进 Roofline / 内存容量 / 调度 / 通信四类模型中检查，确认瓶颈是否真的位于关键路径，而不是只优化了一个非关键算子。

04 | 工程场景

现场推演 某框架即使 dense benchmark 很快，若没有高效 MLA/EP kernel，在 DeepSeek workload 上可能明显落后。

05 | 常见错误

- **✗** 用“支持模型能跑”代替“支持模型高效跑”。

06 | 追问链

- → 如何验证 EP backend?
- → MLA backend 有哪些关键算子?
- → MTP 怎么接 scheduler?

07 | 面试官评分标准

- 及格：能给出正确概念和基本方向。
- 良好：能写出成本/显存/通信公式，能解释为什么。
- 优秀：能指出反例、适用边界，并能把问题落到 profiler、SLO 或真实系统配置。

Q078 MoE 量化为什么比 Dense 模型更复杂?

难度 ★★★★★ · 章节 MoE、MLA 与模型-系统协同 · 关键词 MoE-quant,calibration

30 秒回答 不同 expert 的 activation/权重分布、调用频率和 calibration coverage 不一致。冷门 expert 在校准集上样本不足，统一 scale 或统一策略可能损伤严重；量化后 expert kernel 还要支持 grouped GEMM 与不规则 M。

01 | 考察本质

- 这道题不是考名词定义，而是看你能否从系统资源出发解释：不同 expert 的 activation/权重分布、调用频率和 calibration coverage 不一致。冷门 expert 在校准集上样本不足，统一 scale 或统一策略可能损伤严重；量化后 expert kernel 还要支持 grouped GEMM 与不规则 M。
- 面试中应先给结论，再给成本模型/瓶颈，再给适用边界。只背框架参数通常拿不到高分。

02 | 关键公式 / 成本模型

公式 Calibration should cover routing distribution; scale may need per-expert/per-group granularity。

03 | Know-Why：为什么成立

- 1. 不同 expert 的 activation/权重分布、调用频率和 calibration coverage 不一致。冷门 expert 在校准集上样本不足，统一 scale 或统一策略可能损伤严重
- 2. 量化后 expert kernel 还要支持 grouped GEMM 与不规则 M。
- 把这一结论放进 Roofline / 内存容量 / 调度 / 通信四类模型中检查，确认瓶颈是否真的位于关键路径，而不是只优化了一个非关键算子。

04 | 工程场景

现场推演 若某 expert 只在数学输入被激活，而 calibration 主要是通用聊天，它的 INT4 scale 可能失真。

05 | 常见错误

- ✗ 随机抽几百条文本校准所有 expert
- ✗ 不统计 expert activation coverage。

06 | 追问链

- → 如何构造 calibration routing coverage?
- → 不同 expert 可否用不同 bit?
- → 量化会改变 routing 吗?

07 | 面试官评分标准

- 及格：能给出正确概念和基本方向。
- 良好：能写出成本/显存/通信公式，能解释为什么。
- 优秀：能指出反例、适用边界，并能把问题落到 profiler、SLO 或真实系统配置。

Q079 Wide Expert Parallelism 为什么可能比大规模 TP 更适合 MoE?

难度 ★★★★★ · 章节 MoE、MLA 与模型-系统协同 · 关键词 wide-EP, MoE

30秒回答 把更多 GPU 用于放不同 expert，可以避免每个小 expert GEMM 再做高频 TP collective；代价转为 all-to-all。若网络支持高效 dispatch，wide EP 能让 expert 计算本地化并提升总容量。

01 | 考察本质

- 这道题不是考名词定义，而是看你能否从系统资源出发解释：把更多 GPU 用于放不同 expert，可以避免每个小 expert GEMM 再做高频 TP collective；代价转为 all-to-all。若网络支持高效 dispatch，wide EP 能让 expert 计算本地化并提升总容量。
- 面试中应先给结论，再给成本模型/瓶颈，再给适用边界。只背框架参数通常拿不到高分。

02 | 关键公式 / 成本模型

公式 Tradeoff: smaller TP collective per expert vs larger EP all-to-all domain。

03 | Know-Why：为什么成立

- 1. 把更多 GPU 用于放不同 expert，可以避免每个小 expert GEMM 再做高频 TP collective
- 2. 代价转为 all-to-all。若网络支持高效 dispatch，wide EP 能让 expert 计算本地化并提升总容量。
- 把这一结论放进 Roofline / 内存容量 / 调度 / 通信四类模型中检查，确认瓶颈是否真的位于关键路径，而不是只优化了一个非关键算子。

04 | 工程场景

现场推演 单 expert 已能一张 GPU 高效计算时，继续 TP=8 只会让每 expert M 更小并增加 collective；EP 扩展可能更合理。

05 | 常见错误

- **✗ 一味增大 EP**
- **✗ all-to-all peer 数和跨节点 latency 也会恶化。**

06 | 追问链

- → 如何根据 expert hidden size 决定 TP?
- → EP domain 是否限制在节点内?
- → expert replication 如何降低跨节点?

07 | 面试官评分标准

- 及格：能给出正确概念和基本方向。
- 良好：能写出成本/显存/通信公式，能解释为什么。
- 优秀：能指出反例、适用边界，并能把问题落到 profiler、SLO 或真实系统配置。

Q080 MoE 为什么低并发和高并发性能差异特别大?

难度 ★★★★★ · 章节 MoE、MLA 与模型-系统协同 · 关键词 MoE, batching

30 秒回答 低并发时每 expert 得到的 token 很少，形成 tiny GEMM、kernel launch 与通信延迟占主导；高并发可以跨请求聚合更多 token，让 grouped GEMM 变大、通信消息更饱满，提高硬件利用率。

01 | 考察本质

- 这道题不是考名词定义，而是看你能否从系统资源出发解释：低并发时每 expert 得到的 token 很少，形成 tiny GEMM、kernel launch 与通信延迟占主导；高并发可以跨请求聚合更多 token，让 grouped GEMM 变大、通信消息更饱满，提高硬件利用率。
- 面试中应先给结论，再给成本模型/瓶颈，再给适用边界。只背框架参数通常拿不到高分。

02 | 关键公式 / 成本模型

公式 $\text{tokens_per_expert} \approx \text{batch_tokens} \times \text{top_k} / \text{num_experts}$ （只是一阶均匀估算）。

03 | Know-Why：为什么成立

- 1. 低并发时每 expert 得到的 token 很少，形成 tiny GEMM、kernel launch 与通信延迟占主导
- 2. 高并发可以跨请求聚合更多 token，让 grouped GEMM 变大、通信消息更饱满，提高硬件利用率。
- 把这一结论放进 Roofline / 内存容量 / 调度 / 通信四类模型中检查，确认瓶颈是否真的位于关键路径，而不是只优化了一个非关键算子。

04 | 工程场景

现场推演 EP=64、每 step 仅 64 token、top-2 时平均每 expert 只有约 2 token，几乎无法形成高效 GEMM。

05 | 常见错误

- **✗** 只在高并发 benchmark 得出模型“很快”
- **✗** 交互低 QPS 可能完全不同。

06 | 追问链

- → 如何 microbatch expert tokens?
- → spec decode 会改变 expert batch 吗?
- → P/D 分离对 MoE 有何帮助?

07 | 面试官评分标准

- 及格：能给出正确概念和基本方向。
- 良好：能写出成本/显存/通信公式，能解释为什么。
- 优秀：能指出反例、适用边界，并能把问题落到 profiler、SLO 或真实系统配置。

CHAPTER 09

vLLM / SGLang / TensorRT-LLM

从框架内部抽象到技术选型，理解缓存、调度、kernel、量化与 P/D 能力。

本章题目 Q081 - Q090 · 共 10 题

本章学习目标

- 能解释三个主流框架的内部重点与版本差异。
- 能做 feature fit + benchmark 的技术选型。
- 理解 cache、structured output、LoRA 与 cold-start。

建议刷题方法：先用 30 秒回答自测，再遮住正文推导公式，最后只看“追问链”进行模拟面试。

Q081 请描述 vLLM 的核心架构思想。

难度 ★★★★★ · 章节 vLLM / SGLang / TensorRT-LLM · 关键词 vLLM, architecture

30 秒回答 从服务视角可拆为 API/request processor、scheduler、KV cache manager、model runner/executor、attention/GEMM kernels 与 sampler。核心价值是把动态请求调度、分页 KV、连续批处理和硬件 kernel 统一到高吞吐 runtime 中。V1 又进一步统一 token budget 与执行架构。

01 | 考察本质

- 这道题不是考名词定义，而是看你能否从系统资源出发解释：从服务视角可拆为 API/request processor、scheduler、KV cache manager、model runner/executor、attention/GEMM kernels 与 sampler。核心价值是把动态请求调度、分页 KV、连续批处理和硬件 kernel 统一到高吞吐 runtime 中。V1 又进一步统一 token budget 与执行架构。
- 面试中应先给结论，再给成本模型/瓶颈，再给适用边界。只背框架参数通常拿不到高分。

02 | 关键公式 / 成本模型

公式 Request → Scheduler → KV allocation → Model Runner → Kernels → Sampler/Output。

03 | Know-Why：为什么成立

- 1. 从服务视角可拆为 API/request processor、scheduler、KV cache manager、model runner/executor、attention/GEMM kernels 与 sampler。核心价值是把动态请求调度、分页 KV、连续批处理和硬件 kernel 统一到高吞吐 runtime 中。V1 又进一步统一 token budget 与执行架构。
- 把这一结论放进 Roofline / 内存容量 / 调度 / 通信四类模型中检查，确认瓶颈是否真的位于关键路径，而不是只优化了一个非关键算子。

04 | 工程场景

现场推演 理解架构后才能解释为什么 max_num_batched_tokens、prefix cache 或 preemption 会影响同一条请求的 TTFT/TPOT。

05 | 常见错误

- ✗ 只背 “PagedAttention”
- ✗ vLLM 远不止一个 attention kernel。

06 | 追问链

- → SchedulerOutput 包含什么？
- → KV manager 和 scheduler 如何交互？
- → model runner 如何利用 CUDA Graph？

07 | 面试官评分标准

- 及格：能给出正确概念和基本方向。
- 良好：能写出成本/显存/通信公式，能解释为什么。
- 优秀：能指出反例、适用边界，并能把问题落到 profiler、SLO 或真实系统配置。

Q082 vLLM V1 Scheduler 与早期设计有什么重要变化？

难度 ★★★★★ · 章节 vLLM / SGLang / TensorRT-LLM · 关键词 vLLM-V1,scheduler

30 秒回答 V1 用统一 token budget 表示每请求本轮需要计算的 token，不再严格把 prompt/output 作为两类完全独立调度对象，便于统一支持 chunked prefill、prefix caching 与 speculative tokens；同时旧式 GPU↔CPU KV swapping 已移除。

01 | 考察本质

- 这道题不是考名词定义，而是看你能否从系统资源出发解释：V1 用统一 token budget 表示每请求本轮需要计算的 token，不再严格把 prompt/output 作为两类完全独立调度对象，便于统一支持 chunked prefill、prefix caching 与 speculative tokens；同时旧式 GPU↔CPU KV swapping 已移除。
- 面试中应先给结论，再给成本模型/瓶颈，再给适用边界。只背框架参数通常拿不到高分。

02 | 关键公式 / 成本模型

公式 scheduler output 可理解为 {request_id: num_scheduled_tokens} + cache/block metadata。

03 | Know-Why：为什么成立

- 1. V1 用统一 token budget 表示每请求本轮需要计算的 token，不再严格把 prompt/output 作为两类完全独立调度对象，便于统一支持 chunked prefill、prefix caching 与 speculative tokens
- 2. 同时旧式 GPU↔CPU KV swapping 已移除。
- 把这一结论放进 Roofline / 内存容量 / 调度 / 通信四类模型中检查，确认瓶颈是否真的位于关键路径，而不是只优化了一个非关键算子。

04 | 工程场景

现场推演 同一请求可能本轮计算部分 Prefill，也可能验证 speculative token；统一 token accounting 简化组合。

05 | 常见错误

- ✗ 用 V0 经验回答 V1 版本题
- ✗ 框架快速演进，必须注明版本。

06 | 追问链

- → 为什么统一 scheduler 对 spec decode 有帮助？
- → preemption 如何处理？
- → V1 的 chunked prefill 默认策略是什么？

07 | 面试官评分标准

- 及格：能给出正确概念和基本方向。
- 良好：能写出成本/显存/通信公式，能解释为什么。
- 优秀：能指出反例、适用边界，并能把问题落到 profiler、SLO 或真实系统配置。

Q083 SGLang RadixAttention 相比普通 Prefix Caching 的核心优势?

难度 ★★★★★ · 章节 vLLM / SGLang / TensorRT-LLM · 关键词 SGLang, RadixAttention

30 秒回答 它将共享 prefix 组织成 radix tree，可精确追踪部分前缀、分支与引用，并让 scheduler 利用 longest-prefix match 做 locality-aware routing。对多轮、agent、few-shot 等结构化程序尤其有价值。

01 | 考察本质

- 这道题不是考名词定义，而是看你能否从系统资源出发解释：它将共享 prefix 组织成 radix tree，可精确追踪部分前缀、分支与引用，并让 scheduler 利用 longest-prefix match 做 locality-aware routing。对多轮、agent、few-shot 等结构化程序尤其有价值。
- 面试中应先给结论，再给成本模型/瓶颈，再给适用边界。只背框架参数通常拿不到高分。

02 | 关键公式 / 成本模型

公式 tree node/edge 对应 token prefix 与 KV blocks；scheduler 依据 match length 评估 reuse。

03 | Know-Why：为什么成立

- 1. 它将共享 prefix 组织成 radix tree，可精确追踪部分前缀、分支与引用，并让 scheduler 利用 longest-prefix match 做 locality-aware routing。对多轮、agent、few-shot 等结构化程序尤其有价值。
- 把这一结论放进 Roofline / 内存容量 / 调度 / 通信四类模型中检查，确认瓶颈是否真的位于关键路径，而不是只优化了一个非关键算子。

04 | 工程场景

现场推演 多个 agent 请求共享系统工具描述和前几轮消息时，树结构可复用到分叉点。

05 | 常见错误

- ✗ 只比较 cache lookup 数据结构速度
- ✗ 真正价值是缓存+调度共同设计。

06 | 追问链

- → eviction 怎么做？
- → Radix tree 与 paged KV 如何映射？
- → DP replica 之间如何做 cache-aware routing？

07 | 面试官评分标准

- 及格：能给出正确概念和基本方向。
- 良好：能写出成本/显存/通信公式，能解释为什么。
- 优秀：能指出反例、适用边界，并能把问题落到 profiler、SLO 或真实系统配置。

Q084 TensorRT-LLM 和 vLLM 怎么选?

难度 ★★★★★ · 章节 vLLM / SGLang / TensorRT-LLM · 关键词 framework-selection, TRTLLM, vLLM

30 秒回答 不能用“谁一定快”回答。vLLM 强在生态、模型适配、灵活 runtime、多 backend；TensorRT-LLM 对 NVIDIA GPU 的 kernel、低精度、engine 与生产集成优化更深。最终根据模型结构、GPU、量化、SLO、开发速度与真实 workload benchmark 选择。

01 | 考察本质

- 这道题不是考名词定义，而是看你能否从系统资源出发解释：不能用“谁一定快”回答。vLLM 强在生态、模型适配、灵活 runtime、多 backend；TensorRT-LLM 对 NVIDIA GPU 的 kernel、低精度、engine 与生产集成优化更深。最终根据模型结构、GPU、量化、SLO、开发速度与真实 workload benchmark 选择。
- 面试中应先给结论，再给成本模型/瓶颈，再给适用边界。只背框架参数通常拿不到高分。

02 | 关键公式 / 成本模型

公式 Selection = feature fit × measured goodput × operational cost × time-to-production。

03 | Know-Why：为什么成立

- 1. 不能用“谁一定快”回答。vLLM 强在生态、模型适配、灵活 runtime、多 backend
- 2. TensorRT-LLM 对 NVIDIA GPU 的 kernel、低精度、engine 与生产集成优化更深。最终根据模型结构、GPU、量化、SLO、开发速度与真实 workload benchmark 选择。
- 把这一结论放进 Roofline / 内存容量 / 调度 / 通信四类模型中检查，确认瓶颈是否真的位于关键路径，而不是只优化了一个非关键算子。

04 | 工程场景

现场推演 新研究模型可先 vLLM 快速上线；固定 NVIDIA fleet、FP4/FP8、极致性能目标可深入 TRT-LLM。

05 | 常见错误

- ✗ 把某个公开 benchmark 结果跨模型/跨硬件外推。

06 | 追问链

- → 如何设计公平对比？
- → 哪些参数必须对齐？
- → TensorRT engine build 的运维成本是什么？

07 | 面试官评分标准

- 及格：能给出正确概念和基本方向。
- 良好：能写出成本/显存/通信公式，能解释为什么。
- 优秀：能指出反例、适用边界，并能把问题落到 profiler、SLO 或真实系统配置。

Q085 TensorRT-LLM 的 KV Cache system 有什么特点?

难度 ★★★★★ · 章节 vLLM / SGLang / TensorRT-LLM · 关键词 TensorRT-LLM, KV

30 秒回答 采用 block/paged KV pool，并围绕 KV reuse、offloading、eviction 与不同 attention window 进行管理；在 NVIDIA 生态中还与 disaggregated serving 的 KV transfer 结合。核心仍是把动态 sequence 的 cache 作为一级资源管理。

01 | 考察本质

- 这道题不是考名词定义，而是看你能否从系统资源出发解释：采用 block/paged KV pool，并围绕 KV reuse、offloading、eviction 与不同 attention window 进行管理；在 NVIDIA 生态中还与 disaggregated serving 的 KV transfer 结合。核心仍是把动态 sequence 的 cache 作为一级资源管理。
- 面试中应先给结论，再给成本模型/瓶颈，再给适用边界。只背框架参数通常拿不到高分。

02 | 关键公式 / 成本模型

公式 KV blocks 可有优先级/生命周期，必要时在 device/host/远端层级移动或重用。

03 | Know-Why：为什么成立

- 1. 采用 block/paged KV pool，并围绕 KV reuse、offloading、eviction 与不同 attention window 进行管理
- 2. 在 NVIDIA 生态中还与 disaggregated serving 的 KV transfer 结合。核心仍是把动态 sequence 的 cache 作为一级资源管理。
- 把这一结论放进 Roofline / 内存容量 / 调度 / 通信四类模型中检查，确认瓶颈是否真的位于关键路径，而不是只优化了一个非关键算子。

04 | 工程场景

现场推演 长 system prompt 可标记高复用价值，减少重复 Prefill；P/D 模式下 block 需要跨 worker 传输。

05 | 常见错误

- ✗ 认为 TRT-LLM 只做静态 TensorRT engine
- ✗ 现代 serving 层同样需要动态 KV。

06 | 追问链

- → KV reuse 如何识别？
- → offload 的 break-even 如何算？
- → P/D KV transmission 如何 overlap？

07 | 面试官评分标准

- 及格：能给出正确概念和基本方向。
- 良好：能写出成本/显存/通信公式，能解释为什么。
- 优秀：能指出反例、适用边界，并能把问题落到 profiler、SLO 或真实系统配置。

Q086 SGLang HiCache 是什么？

难度 ★★★★★ · 章节 vLLM / SGLang / TensorRT-LLM · 关键词 HiCache, SGLang

30 秒回答 HiCache 将 KV 扩展为分层缓存：GPU HBM 是低延迟 L1，CPU DRAM/主存是更大 L2，远端或分布式存储可作为更大层级。目标是让长 context、多轮和高 prefix reuse workload 不必把所有可复用 KV 常驻 GPU。

01 | 考察本质

- 这道题不是考名词定义，而是看你能否从系统资源出发解释：HiCache 将 KV 扩展为分层缓存：GPU HBM 是低延迟 L1，CPU DRAM/主存是更大 L2，远端或分布式存储可作为更大层级。目标是让长 context、多轮和高 prefix reuse workload 不必把所有可复用 KV 常驻 GPU。
- 面试中应先给结论，再给成本模型/瓶颈，再给适用边界。只背框架参数通常拿不到高分。

02 | 关键公式 / 成本模型

公式 Hit hierarchy: GPU → host → distributed; benefit 取决于 load latency 与 recompute cost 的比较。

03 | Know-Why：为什么成立

- 1. HiCache 将 KV 扩展为分层缓存：GPU HBM 是低延迟 L1，CPU DRAM/主存是更大 L2，远端或分布式存储可作为更大层级。目标是让长 context、多轮和高 prefix reuse workload 不必把所有可复用 KV 常驻 GPU。
- 把这一结论放进 Roofline / 内存容量 / 调度 / 通信四类模型中检查，确认瓶颈是否真的位于关键路径，而不是只优化了一个非关键算子。

04 | 工程场景

现场推演 超长 agent 会话暂时不活跃时，可把冷 KV 下沉到 host，恢复会话时再提升。

05 | 常见错误

- ✗ 把层次缓存等同于普通 swap
- ✗ HiCache 更强调复用、层级与调度。

06 | 追问链

- → 何时从 L2 load 比 recompute 更划算？
- → 如何做 eviction priority？
- → 多节点一致性如何管理？

07 | 面试官评分标准

- 及格：能给出正确概念和基本方向。
- 良好：能写出成本/显存/通信公式，能解释为什么。
- 优秀：能指出反例、适用边界，并能把问题落到 profiler、SLO 或真实系统配置。

Q087 Multi-LoRA Serving 怎么优化?

难度 ★★★★★ · 章节 vLLM / SGLang / TensorRT-LLM · 关键词 LoRA,serving

30 秒回答 Base weights 在所有请求间共享，仅 adapter A/B 小矩阵随请求不同。关键是避免按 adapter 把 batch 切碎：把同一 batch 中不同 LoRA 的低秩增量用 batched/grouped kernel 计算，同时缓存热门 adapter，减少切换和加载。

01 | 考察本质

- 这道题不是考名词定义，而是看你能否从系统资源出发解释：Base weights 在所有请求间共享，仅 adapter A/B 小矩阵随请求不同。关键是避免按 adapter 把 batch 切碎：把同一 batch 中不同 LoRA 的低秩增量用 batched/grouped kernel 计算，同时缓存热门 adapter，减少切换和加载。
- 面试中应先给结论，再给成本模型/瓶颈，再给适用边界。只背框架参数通常拿不到高分。

02 | 关键公式 / 成本模型

公式 $y = xW_{\text{base}} + xA_iB_i$; W_{base} 共享， A_i/B_i 按 request 选择。

03 | Know-Why: 为什么成立

- 1. Base weights 在所有请求间共享，仅 adapter A/B 小矩阵随请求不同。关键是避免按 adapter 把 batch 切碎：把同一 batch 中不同 LoRA 的低秩增量用 batched/grouped kernel 计算，同时缓存热门 adapter，减少切换和加载。
- 把这一结论放进 Roofline / 内存容量 / 调度 / 通信四类模型中检查，确认瓶颈是否真的位于关键路径，而不是只优化了一个非关键算子。

04 | 工程场景

现场推演 100 个租户各有 LoRA，如果逐租户独立 batch，GPU 利用率会崩；fused multi-LoRA kernel 可保留主 batch。

05 | 常见错误

- ✗ 只关注 adapter 显存很小，忽略 adapter diversity 对 batching 的破坏。

06 | 追问链

- LoRA rank 如何影响 kernel?
- adapter cache 怎么做?
- 不同 TP rank 如何切 LoRA?

07 | 面试官评分标准

- 及格：能给出正确概念和基本方向。
- 良好：能写出成本/显存/通信公式，能解释为什么。
- 优秀：能指出反例、适用边界，并能把问题落到 profiler、SLO 或真实系统配置。

Q088 Structured Output 为什么会影响推理性能?

难度 ★★★★★ · 章节 vLLM / SGLang / TensorRT-LLM · 关键词 structured-output,FSM

30 秒回答 JSON schema/grammar constrained decoding 每步需维护自动机状态并限制可选 token，可能产生 CPU mask

计算、GPU logits masking 与同步开销。高性能方案会预编译 FSM、压缩 token transition、缓存 mask，或在 GPU 侧融合。

01 | 考察本质

- 这道题不是考名词定义，而是看你能否从系统资源出发解释：JSON schema/grammar constrained decoding 每步需维护自动机状态并限制可选 token，可能产生 CPU mask 计算、GPU logits masking 与同步开销。高性能方案会预编译 FSM、压缩 token transition、缓存 mask，或在 GPU 侧融合。
- 面试中应先给结论，再给成本模型/瓶颈，再给适用边界。只背框架参数通常拿不到高分。

02 | 关键公式 / 成本模型

公式 每 token extra cost = FSM transition + allowed-token mask + logits filtering。

03 | Know-Why：为什么成立

- 1. JSON schema/grammar constrained decoding 每步需维护自动机状态并限制可选 token，可能产生 CPU mask 计算、GPU logits masking 与同步开销。高性能方案会预编译 FSM、压缩 token transition、缓存 mask，或在 GPU 侧融合。
- 把这一结论放进 Roofline / 内存容量 / 调度 / 通信四类模型中检查，确认瓶颈是否真的位于关键路径，而不是只优化了一个非关键算子。

04 | 工程场景

现场推演 超短模型输出中 grammar 开销占比可能很高；长 reasoning 输出则被模型 forward 摊薄。

05 | 常见错误

- ✗ 认为 structured output 只是后处理
- ✗ 真正约束通常参与每个 decoding step。

06 | 追问链

- → Outlines/Guidance/grammar backend 差异？
- → 为什么 tokenizer trie 很重要？
- → 如何 batch 不同 grammar？

07 | 面试官评分标准

- 及格：能给出正确概念和基本方向。
- 良好：能写出成本/显存/通信公式，能解释为什么。
- 优秀：能指出反例、适用边界，并能把问题落到 profiler、SLO 或真实系统配置。

Q089 为什么模型刚启动时延迟明显更高？

难度 ★★★★★ · 章节 vLLM / SGLang / TensorRT-LLM · 关键词 cold-start, startup

30 秒回答 cold start 包含权重读取/传输、NCCL 初始化、kernel JIT、Triton/torch.compile、CUDA Graph capture、量化 packing、KV pool 分配与 page fault/cache warmup。基准测试要单独报告 cold-start 与 steady-state。

01 | 考察本质

- 这道题不是考名词定义，而是看你能否从系统资源出发解释：cold start 包含权重读取/传输、NCCL 初始化、kernel JIT、Triton/torch.compile、CUDA Graph capture、量化 packing、KV pool 分配与 page fault/cache warmup。基准测试要单独报告 cold-start 与 steady-state。
- 面试中应先给结论，再给成本模型/瓶颈，再给适用边界。只背框架参数通常拿不到高分。

02 | 关键公式 / 成本模型

公式 $T_{\text{startup}} = \text{load} + \text{init} + \text{compile} + \text{capture} + \text{warmup}$ ；首请求 latency 还加正常 TTFT。

03 | Know-Why：为什么成立

- 1. cold start 包含权重读取/传输、NCCL 初始化、kernel JIT、Triton/torch.compile、CUDA Graph capture、量化 packing、KV pool 分配与 page fault/cache warmup。基准测试要单独报告 cold-start 与 steady-state。
- 把这一结论放进 Roofline / 内存容量 / 调度 / 通信四类模型中检查，确认瓶颈是否真的位于关键路径，而不是只优化了一个非关键算子。

04 | 工程场景

现场推演 Autoscaling 新 pod 刚接流量时若没有 warmup，会让 p99 暴涨。

05 | 常见错误

- **✗** 把首请求慢归因模型本身
- **✗** 或把 warmup 后结果当冷启动承诺。

06 | 追问链

- → 如何做 readiness gate?
- → 哪些编译产物可缓存?
- → 模型权重如何并行加载?

07 | 面试官评分标准

- 及格：能给出正确概念和基本方向。
- 良好：能写出成本/显存/通信公式，能解释为什么。
- 优秀：能指出反例、适用边界，并能把问题落到 profiler、SLO 或真实系统配置。

Q090 如果 vLLM、SGLang、TensorRT-LLM 三选一，你怎么做技术选型？

难度 ★★★★★ · 章节 vLLM / SGLang / TensorRT-LLM · 关键词 framework-selection, production

30 秒回答 建立决策矩阵：模型结构(Dense/MoE/MLA/VLM)、GPU/加速器、quant、prefix reuse、P/D、structured output、LoRA、运维成熟度、社区/模型支持，以及真实 workload 的 Goodput/成本。先排除 feature gap，再 benchmark Pareto frontier。

01 | 考察本质

- 这道题不是考名词定义，而是看你能否从系统资源出发解释：建立决策矩阵：模型结构(Dense/MoE/MLA/VLM)、GPU/加速器、quant、prefix reuse、P/D、structured output、LoRA、运维成熟度、社区/模型支持，以及真实 workload 的 Goodput/成本。先排除 feature gap，再 benchmark Pareto frontier。
- 面试中应先给结论，再给成本模型/瓶颈，再给适用边界。只背框架参数通常拿不到高分。

02 | 关键公式 / 成本模型

公式 不要只比较峰值 tokens/s；至少比较 SLO-constrained goodput 与 \$/M token。

03 | Know-Why：为什么成立

- 1. 建立决策矩阵：模型结构(Dense/MoE/MLA/VLM)、GPU/加速器、quant、prefix reuse、P/D、structured output、LoRA、运维成熟度、社区/模型支持，以及真实 workload 的 Goodput/成本。先排除 feature gap，再 benchmark Pareto frontier。
- 把这一结论放进 Roofline / 内存容量 / 调度 / 通信四类模型中检查，确认瓶颈是否真的位于关键路径，而不是只优化了一个非关键算子。

04 | 工程场景

现场推演 Agent/prefix-heavy workload 可能偏好强 cache/scheduler；固定 NVIDIA low-precision fleet 可能偏好 TRT-LLM；模型迭代快则生态适配很关键。

05 | 常见错误

- **✗** 选型表里没有“升级成本/可观测性/故障恢复”。

06 | 追问链

- → 如何做两周 PoC？
- → 谁负责模型新架构适配？
- → 如何避免 vendor/framework lock-in？

07 | 面试官评分标准

- 及格：能给出正确概念和基本方向。
- 良好：能写出成本/显存/通信公式，能解释为什么。
- 优秀：能指出反例、适用边界，并能把问题落到 profiler、SLO 或真实系统配置。

CHAPTER 10

Benchmark、生产部署与系统设计

把算法优化落到 SLO、Goodput、成本、监控、故障定位与容量规划。

本章题目 Q091 - Q100 · 共 10 题

本章学习目标

- 能设计可复现 benchmark。
- 能把 SLO、Goodput、成本和容量规划串起来。
- 能系统定位线上 30% 性能回归。

建议刷题方法：先用 30 秒回答自测，再遮住正文推导公式，最后只看“追问链”进行模拟面试。

Q091 你会如何 Benchmark 一个 LLM Serving Engine?

难度 ★★★★★ · 章节 Benchmark、生产部署与系统设计 · 关键词 benchmark,serving

30 秒回答 固定模型、revision、量化、GPU、并行策略、采样配置；用真实或拟真 input/output 长度联合分布和到达过程压测，逐步提高 offered load，报告 TTFT/TPOT/E2E p50/p95/p99、throughput、goodput、KV 使用、prefix hit 和 GPU/NCCL。

01 | 考察本质

- 这道题不是考名词定义，而是看你能否从系统资源出发解释：固定模型、revision、量化、GPU、并行策略、采样配置；用真实或拟真 input/output 长度联合分布和到达过程压测，逐步提高 offered load，报告 TTFT/TPOT/E2E p50/p95/p99、throughput、goodput、KV 使用、prefix hit 和 GPU/NCCL。
- 面试中应先给结论，再给成本模型/瓶颈，再给适用边界。只背框架参数通常拿不到高分。

02 | 关键公式 / 成本模型

公式 Benchmark 不是单点，而是 latency-throughput curve / goodput frontier。

03 | Know-Why：为什么成立

- 1. 固定模型、revision、量化、GPU、并行策略、采样配置
- 2. 用真实或拟真 input/output 长度联合分布和到达过程压测，逐步提高 offered load，报告 TTFT/TPOT/E2E p50/p95/p99、throughput、goodput、KV 使用、prefix hit 和 GPU/NCCL。
- 把这一结论放进 Roofline / 内存容量 / 调度 / 通信四类模型中检查，确认瓶颈是否真的位于关键路径，而不是只优化了一个非关键算子。

04 | 工程场景

现场推演 用 Poisson 或 trace replay 模拟到达；对 Chat/RAG/Reasoning 分 workload profile。

05 | 常见错误

- ✗ 只跑 offline batch
- ✗ 或比较框架时参数、tokenizer、max length 不一致。

06 | 追问链

- → 如何处理 warmup?
- → 怎样避免 client 成为瓶颈?
- → 如何测 saturation point?

07 | 面试官评分标准

- 及格：能给出正确概念和基本方向。
- 良好：能写出成本/显存/通信公式，能解释为什么。
- 优秀：能指出反例、适用边界，并能把问题落到 profiler、SLO 或真实系统配置。

Q092 为什么用固定 512→512 benchmark 很危险?

难度 ★★★★★ · 章节 Benchmark、生产部署与系统设计 · 关键词 benchmark, workload

30 秒回答 LLM workload 的成本由 input/output 长度、prefix reuse、并发、到达率共同决定。单一 512→512 可能掩盖长 Prefill stall、长 Decode KV、短请求 launch overhead 和 RAG cache reuse，导致错误选型。

01 | 考察本质

- 这道题不是考名词定义，而是看你能否从系统资源出发解释：LLM workload 的成本由 input/output 长度、prefix reuse、并发、到达率共同决定。单一 512→512 可能掩盖长 Prefill stall、长 Decode KV、短请求 launch overhead 和 RAG cache reuse，导致错误选型。
- 面试中应先给结论，再给成本模型/瓶颈，再给适用边界。只背框架参数通常拿不到高分。

02 | 关键公式 / 成本模型

公式 应采样联合分布 $P(L_{in}, L_{out}, arrival, prefix_group)$ 。

03 | Know-Why：为什么成立

- 1. LLM workload 的成本由 input/output 长度、prefix reuse、并发、到达率共同决定。单一 512→512 可能掩盖长 Prefill stall、长 Decode KV、短请求 launch overhead 和 RAG cache reuse，导致错误选型。
- 把这一结论放进 Roofline / 内存容量 / 调度 / 通信四类模型中检查，确认瓶颈是否真的位于关键路径，而不是只优化了一个非关键算子。

04 | 工程场景

现场推演 Chat 2K→300、RAG 20K→500、Reasoning 1K→10K 即使平均 token 类似，最优 scheduler 完全不同。

05 | 常见错误

- ✗ 只匹配平均长度，不匹配长尾
- ✗ p99 往往由长尾请求驱动。

06 | 追问链

- → 如何构造 synthetic trace 保留相关性？
- → output length 未知怎么压测？
- → prefix group 怎么模拟？

07 | 面试官评分标准

- 及格：能给出正确概念和基本方向。
- 良好：能写出成本/显存/通信公式，能解释为什么。
- 优秀：能指出反例、适用边界，并能把问题落到 profiler、SLO 或真实系统配置。

Q093 为什么 p99 比平均 latency 更重要？

难度 ★★★★★ · 章节 Benchmark、生产部署与系统设计 · 关键词 p99,tail-latency

30 秒回答 在线交互中少量极慢请求直接影响体验与 SLO，且 tail 常揭示系统临界状态：队列拥塞、KV thrash、长 Prefill、straggler、NCCL 抖动、autoscaling cold-start。优化平均值可能掩盖这些故障。

01 | 考察本质

- 这道题不是考名词定义，而是看你能否从系统资源出发解释：在线交互中少量极慢请求直接影响体验与 SLO，且 tail 常揭示系统临界状态：队列拥塞、KV thrash、长 Prefill、straggler、NCCL 抖动、autoscaling cold-start。优化平均值可能掩盖这些故障。
- 面试中应先给结论，再给成本模型/瓶颈，再给适用边界。只背框架参数通常拿不到高分。

02 | 关键公式 / 成本模型

公式 p99 表示 99% 请求不超过该值；tail amplification 在多阶段系统中更明显。

03 | Know-Why：为什么成立

- 1. 在线交互中少量极慢请求直接影响体验与 SLO，且 tail 常揭示系统临界状态：队列拥塞、KV thrash、长 Prefill、straggler、NCCL 抖动、autoscaling cold-start。优化平均值可能掩盖这些故障。
- 把这一结论放进 Roofline / 内存容量 / 调度 / 通信四类模型中检查，确认瓶颈是否真的位于关键路径，而不是只优化了一个非关键算子。

04 | 工程场景

现场推演 平均 200ms、p99 8s 的服务对 1% 用户仍不可接受，且这 1% 往往集中在高价值长请求。

05 | 常见错误

- ✗ 只报告全局 p99
- ✗ 应按 workload/tenant/length bucket 分解。

06 | 追问链

- → 如何找 p99 请求共性？
- → 排队 tail 与执行 tail 如何区分？
- → 多 stage P/D 如何做 trace correlation？

07 | 面试官评分标准

- 及格：能给出正确概念和基本方向。
- 良好：能写出成本/显存/通信公式，能解释为什么。
- 优秀：能指出反例、适用边界，并能把问题落到 profiler、SLO 或真实系统配置。

Q094 Throughput 和 Goodput 的区别是什么？

难度 ★★★★★ · 章节 Benchmark、生产部署与系统设计 · 关键词 goodput, SLO

30 秒回答 Throughput 统计所有完成工作；Goodput 只统计满足服务质量约束的完成工作。随着 offered load 上升，throughput 可能继续升，而 queue/p99 导致 goodput 达峰后下降或饱和。

01 | 考察本质

- 这道题不是考名词定义，而是看你能否从系统资源出发解释：Throughput 统计所有完成工作；Goodput 只统计满足服务质量约束的完成工作。随着 offered load 上升，throughput 可能继续升，而 queue/p99 导致 goodput 达峰后下降或饱和。
- 面试中应先给结论，再给成本模型/瓶颈，再给适用边界。只背框架参数通常拿不到高分。

02 | 关键公式 / 成本模型

公式 $\text{Goodput} = \text{completed_within_SLO} / \text{time}$ 。

03 | Know-Why：为什么成立

- 1. Throughput 统计所有完成工作
- 2. Goodput 只统计满足服务质量约束的完成工作。随着 offered load 上升，throughput 可能继续升，而 queue/p99 导致 goodput 达峰后下降或饱和。
- 把这一结论放进 Roofline / 内存容量 / 调度 / 通信四类模型中检查，确认瓶颈是否真的位于关键路径，而不是只优化了一个非关键算子。

04 | 工程场景

现场推演 系统在 120 req/s 时吞吐最高，但只有 70 req/s 满足 SLO；在 90 req/s 时有 85 req/s 满足 SLO，后者容量更有意义。

05 | 常见错误

- **x** 把 SLA 超时后仍完成的请求计入“可服务容量”。

06 | 追问链

- → 如何定义多维 SLO？
- → Goodput 与 admission control 的关系？
- → 如何用它做 autoscaling？

07 | 面试官评分标准

- 及格：能给出正确概念和基本方向。
- 良好：能写出成本/显存/通信公式，能解释为什么。
- 优秀：能指出反例、适用边界，并能把问题落到 profiler、SLO 或真实系统配置。

Q095 如何计算每百万输出 token 成本?

难度 ★★★★★ · 章节 Benchmark、生产部署与系统设计 · 关键词 cost, TCO

30 秒回答 先用集群小时成本除以稳定输出 token/hour，再乘一百万；生产还要折算 GPU 利用率、冗余、CPU/网络/存储、失败与 idle capacity。输入与输出 token 应分别核算，因为 Prefill 与 Decode 的成本结构不同。

01 | 考察本质

- 这道题不是考名词定义，而是看你能否从系统资源出发解释：先用集群小时成本除以稳定输出 token/hour，再乘一百万；生产还要折算 GPU 利用率、冗余、CPU/网络/存储、失败与 idle capacity。输入与输出 token 应分别核算，因为 Prefill 与 Decode 的成本结构不同。
- 面试中应先给结论，再给成本模型/瓶颈，再给适用边界。只背框架参数通常拿不到高分。

02 | 关键公式 / 成本模型

公式 $\text{Cost/Mout} = (\text{GPU}\$/\text{h} \times \text{GPU_count} + \text{infra}\$/\text{h}) / \text{output_tokens_per_h} \times 10^6$ 。

03 | Know-Why：为什么成立

- 1. 先用集群小时成本除以稳定输出 token/hour，再乘一百万
- 2. 生产还要折算 GPU 利用率、冗余、CPU/网络/存储、失败与 idle capacity。输入与输出 token 应分别核算，因为 Prefill 与 Decode 的成本结构不同。
- 把这一结论放进 Roofline / 内存容量 / 调度 / 通信四类模型中检查，确认瓶颈是否真的位于关键路径，而不是只优化了一个非关键算子。

04 | 工程场景

现场推演 同一硬件在低 QPS 夜间利用率 20%，实际 \$/M token 可能远高于满载 benchmark。

05 | 常见错误

- ✗ 用云 GPU 标价/理论 tokens/s 得出成本
- ✗ 忽略 goodput 与闲置。

06 | 追问链

- → 怎么把 P/D 两池成本合并？
- → 量化节省显存但质量下降如何折算？
- → reserved capacity 怎么计？

07 | 面试官评分标准

- 及格：能给出正确概念和基本方向。
- 良好：能写出成本/显存/通信公式，能解释为什么。
- 优秀：能指出反例、适用边界，并能把问题落到 profiler、SLO 或真实系统配置。

Q096 LLM Serving 如何做 Admission Control 和 Autoscaling?

难度 ★★★★★ · 章节 Benchmark、生产部署与系统设计 · 关键词 admission-control,autoscaling

30 秒回答 不能只看 requests/s，要估计 token workload：input token rate、active decode sequences、预期 output、KV occupancy、queue 与 SLO。Admission control 防止超载进入 thrashing；autoscaling 依据未来 token demand 与冷启动时间提前扩容。

01 | 考察本质

- 这道题不是考名词定义，而是看你能否从系统资源出发解释：不能只看 requests/s，要估计 token workload：input token rate、active decode sequences、预期 output、KV occupancy、queue 与 SLO。Admission control 防止超载进入 thrashing；autoscaling 依据未来 token demand 与冷启动时间提前扩容。
- 面试中应先给结论，再给成本模型/瓶颈，再给适用边界。只背框架参数通常拿不到高分。

02 | 关键公式 / 成本模型

公式 $\text{Load} \approx \lambda \times E[\text{prefill_cost} + \text{decode_steps} \times \text{per_step_cost}]$ ，同时受 KV 生命周期约束。

03 | Know-Why：为什么成立

- 1. 不能只看 requests/s，要估计 token workload：input token rate、active decode sequences、预期 output、KV occupancy、queue 与 SLO。Admission control 防止超载进入 thrashing
- 2. autoscaling 依据未来 token demand 与冷启动时间提前扩容。
- 把这一结论放进 Roofline / 内存容量 / 调度 / 通信四类模型中检查，确认瓶颈是否真的位于关键路径，而不是只优化了一个非关键算子。

04 | 工程场景

现场推演 10 个 100K prompt 请求的压力可能远大于 100 个短 chat；单纯 QPS 会误判。

05 | 常见错误

- ✗ 直到 OOM 才拒绝
- ✗ 这时通常 p99 已经崩溃。

06 | 追问链

- → 如何预测 output length?
- → watermark 怎么设?
- → cold-start 很长时如何扩容?

07 | 面试官评分标准

- 及格：能给出正确概念和基本方向。
- 良好：能写出成本/显存/通信公式，能解释为什么。
- 优秀：能指出反例、适用边界，并能把问题落到 profiler、SLO 或真实系统配置。

Q097 线上发生 KV Cache OOM，你怎么处理？

难度 ★★★★★ · 章节 Benchmark、生产部署与系统设计 · 关键词 OOM,KV,operations

30 秒回答 先做内存分解：weights、KV、workspace、graph、allocator fragmentation；确认是容量不足还是泄漏/配置。短期降低并发/最大 context、启用 admission；中期 KV quant、GQA/MLA、page/allocator 优化、offload/HiCache/CP；长期 scale-out 或模型结构调整。

01 | 考察本质

- 这道题不是考名词定义，而是看你能否从系统资源出发解释：先做内存分解：weights、KV、workspace、graph、allocator fragmentation；确认是容量不足还是泄漏/配置。短期降低并发/最大 context、启用 admission；中期 KV quant、GQA/MLA、page/allocator 优化、offload/HiCache/CP；长期 scale-out 或模型结构调整。
- 面试中应先给结论，再给成本模型/瓶颈，再给适用边界。只背框架参数通常拿不到高分。

02 | 关键公式 / 成本模型

公式 $M_{total} = M_{weights} + M_{KV} + M_{workspace} + M_{graph} + M_{misc} < VRAM_{budget}$ 。

03 | Know-Why：为什么成立

- 1. 先做内存分解：weights、KV、workspace、graph、allocator fragmentation
- 2. 确认是容量不足还是泄漏/配置。短期降低并发/最大 context、启用 admission
- 3. 中期 KV quant、GQA/MLA、page/allocator 优化、offload/HiCache/CP
- 4. 长期 scale-out 或模型结构调整。
- 把这一结论放进 Roofline / 内存容量 / 调度 / 通信四类模型中检查，确认瓶颈是否真的位于关键路径，而不是只优化了一个非关键算子。

04 | 工程场景

现场推演 如果权重仅占 40% 显存、KV 55%，继续把权重从 INT8 降 INT4 只释放有限总空间，KV 优化更直接。

05 | 常见错误

- ✗ 第一反应 empty_cache
- ✗ 它不能解决活跃 KV 的真实容量需求。

06 | 追问链

- → 如何判断 fragmentation?
- → 如何设置 max concurrency?
- → preemption storm 的症状是什么?

07 | 面试官评分标准

- 及格：能给出正确概念和基本方向。
- 良好：能写出成本/显存/通信公式，能解释为什么。
- 优秀：能指出反例、适用边界，并能把问题落到 profiler、SLO 或真实系统配置。

Q098 生产 LLM Server 最重要的监控指标有哪些？

难度 ★★★★★ · 章节 Benchmark、生产部署与系统设计 · 关键词 observability,metrics

30 秒回答 需要 request、scheduler、cache、GPU/network 四层可观测性，并可用 request_id 串联。请求层看 QPS/queue/TTFT/TPOT/p99/error；scheduler 看 running/waiting/batch/tokens/preemption；cache 看 occupancy/hit/eviction；硬件看 SM/Tensor/HBM/VRAM/NCCL/power。

01 | 考察本质

- 这道题不是考名词定义，而是看你能否从系统资源出发解释：需要 request、scheduler、cache、GPU/network 四层可观测性，并可用 request_id 串联。请求层看 QPS/queue/TTFT/TPOT/p99/error；scheduler 看 running/waiting/batch/tokens/preemption；cache 看 occupancy/hit/eviction；硬件看 SM/Tensor/HBM/VRAM/NCCL/power。
- 面试中应先给结论，再给成本模型/瓶颈，再给适用边界。只背框架参数通常拿不到高分。

02 | 关键公式 / 成本模型

公式 可观测性目标：从一次 SLO violation 能追到对应 scheduler step、GPU rank 与 kernel。

03 | Know-Why：为什么成立

- 1. 需要 request、scheduler、cache、GPU/network 四层可观测性，并可用 request_id 串联。请求层看 QPS/queue/TTFT/TPOT/p99/error
- 2. scheduler 看 running/waiting/batch/tokens/preemption
- 3. cache 看 occupancy/hit/eviction
- 4. 硬件看 SM/Tensor/HBM/VRAM/NCCL/power。
- 把这一结论放进 Roofline / 内存容量 / 调度 / 通信四类模型中检查，确认瓶颈是否真的位于关键路径，而不是只优化了一个非关键算子。

04 | 工程场景

现场推演 prefix hit rate 下降可能先表现为 TTFT 增加，再提高 GPU Prefill 负载并造成整体 queue。

05 | 常见错误

- ✗ 指标很多但没有 trace correlation
- ✗ 无法回答“哪一批请求导致 p99”。

06 | 追问链

- → 哪些指标适合告警？
- → 如何区分原因指标和结果指标？
- → P/D 分离后如何关联？

07 | 面试官评分标准

- 及格：能给出正确概念和基本方向。
- 良好：能写出成本/显存/通信公式，能解释为什么。
- 优秀：能指出反例、适用边界，并能把问题落到 profiler、SLO 或真实系统配置。

Q099 系统设计题：8×H100，部署一个 70B Chat Model，你怎么设计？

难度 ★★★★★ · 章节 Benchmark、生产部署与系统设计 · 关键词 system-design,H100,70B

30 秒回答 先定义模型 dtype、输入/输出分布和 SLO，再算权重+KV。BF16 70B 仅权重约 140GB，需要至少跨卡；候选可比较 TP=2×4 replicas、TP=4×2 replicas、FP8/INT4 后更小 TP 等。节点内 NVLink 适合 TP，最终用真实 trace 比较 Goodput、p99 与成本。

01 | 考察本质

- 这道题不是考名词定义，而是看你能否从系统资源出发解释：先定义模型 dtype、输入/输出分布和 SLO，再算权重+KV。BF16 70B 仅权重约 140GB，需要至少跨卡；候选可比较 TP=2×4 replicas、TP=4×2 replicas、FP8/INT4 后更小 TP 等。节点内 NVLink 适合 TP，最终用真实 trace 比较 Goodput、p99 与成本。
- 面试中应先给结论，再给成本模型/瓶颈，再给适用边界。只背框架参数通常拿不到高分。

02 | 关键公式 / 成本模型

公式 $M_{\text{weights}} \approx \text{params} \times \text{bytes}$ ； M_{KV} 用 Q11 公式；
 $\text{capacity} = \min(\text{compute_capacity}, \text{KV_capacity}, \text{SLO_capacity})$ 。

03 | Know-Why：为什么成立

- 1. 先定义模型 dtype、输入/输出分布和 SLO，再算权重+KV。BF16 70B 仅权重约 140GB，需要至少跨卡
- 2. 候选可比较 TP=2×4 replicas、TP=4×2 replicas、FP8/INT4 后更小 TP 等。节点内 NVLink 适合 TP，最终用真实 trace 比较 Goodput、p99 与成本。
- 把这一结论放进 Roofline / 内存容量 / 调度 / 通信四类模型中检查，确认瓶颈是否真的位于关键路径，而不是只优化了一个非关键算子。

04 | 工程场景

现场推演 高 QPS Chat 通常优先更多 replica；极低 latency 单请求可能使用更高 TP。若长 context，KV 容量可能决定 TP/replica 方案。

05 | 常见错误

- ✗ 直接答 TP=8
- ✗ 没有 SLO、长度分布和 dtype 就无法唯一决策。

06 | 追问链

- 如果换 FP8 呢？
- 如果平均 input 32K 呢？
- 如果需要 99.9% TTFT<1s 呢？

07 | 面试官评分标准

- 及格：能给出正确概念和基本方向。
- 良好：能写出成本/显存/通信公式，能解释为什么。
- 优秀：能指出反例、适用边界，并能把问题落到 profiler、SLO 或真实系统配置。

Q100 终极综合题：线上模型突然慢了 30%，你如何定位？

难度 ★★★★★ · 章节 Benchmark、生产部署与系统设计 · 关键词 debugging, production, systematic

30 秒回答 先确认变更与流量是否变化；将退化拆为 queue/TTFT/TPOT，按请求长度和 tenant 分桶。再看 scheduler/KV 指标、GPU timeline、kernel/HBM、NCCL 与 CPU。做 fixed-input controlled experiment 并逐项关闭 cache/spec/compile/parallel features，最终用 A/B 或回滚验证根因。

01 | 考察本质

- 这道题不是考名词定义，而是看你能否从系统资源出发解释：先确认变更与流量是否变化；将退化拆为 queue/TTFT/TPOT，按请求长度和 tenant 分桶。再看 scheduler/KV 指标、GPU timeline、kernel/HBM、NCCL 与 CPU。做 fixed-input controlled experiment 并逐项关闭 cache/spec/compile/parallel features，最终用 A/B 或回滚验证根因。
- 面试中应先给结论，再给成本模型/瓶颈，再给适用边界。只背框架参数通常拿不到高分。

02 | 关键公式 / 成本模型

公式 诊断树：Regression scope → TTFT/TPOT split → Scheduler/KV → GPU/Kernel → Network/CPU → Controlled isolation → Root-cause proof。

03 | Know-Why：为什么成立

- 1. 先确认变更与流量是否变化
- 2. 将退化拆为 queue/TTFT/TPOT，按请求长度和 tenant 分桶。再看 scheduler/KV 指标、GPU timeline、kernel/HBM、NCCL 与 CPU。做 fixed-input controlled experiment 并逐项关闭 cache/spec/compile/parallel features，最终用 A/B 或回滚验证根因。
- 把这一结论放进 Roofline / 内存容量 / 调度 / 通信四类模型中检查，确认瓶颈是否真的位于关键路径，而不是只优化了一个非关键算子。

04 | 工程场景

现场推演 若 TTFT 升而 TPOT 不变，优先查 Prefill/queue/cache；若 TPOT 升，查 Decode batch、HBM、KV、NCCL、spec acceptance。

05 | 常见错误

- ✗ 直接猜“驱动问题”
- ✗ 没有时间线、没有对照、一次改多个变量。

06 | 追问链

- → 如何确认是 workload drift?
- → 如何做 performance bisect?
- → 怎么防止同类回归再次发生?

07 | 面试官评分标准

- 及格：能给出正确诊断路径，并能把 TTFT/TPOT 分开。
- 优秀：能用 controlled experiment、profile 与回滚/A-B 建立可证伪的根因证据链。

最后 20 题 | 高频优先级

建议 如果只有 3 天，先吃透下面 20 题；要求不是背答案，而是每题能连续回答 3 个追问。

题号	能力	题目	题号	能力	题目
Q001	prefill,decode,roofline	为什么 LLM 的 Prefill 和 Decode 是两种完全不同的计算？	Q031	FlashAttention,IO	FlashAttention 为什么更快？
Q003	decode,memory-bandwidth	为什么 batch=1 的 Decode 经常是 Memory Bound？	Q041	quantization,W4A16,W8A8	Weight-only Quantization 与 W8A8 有什么区别？
Q004	roofline,profiling	什么是 Roofline Model？如何用于分析 LLM inference？	Q050	INT4,performance	为什么 INT4 模型有时候不比 FP16 快？
Q005	estimation,bandwidth	如何估算一个模型 Decode 的理论 token/s？	Q051	TP,distributed	Tensor Parallelism 是什么？
Q011	KV-cache,memory	KV Cache 显存怎么计算？	Q054	EP,MoE	Expert Parallelism 是什么？
Q013	PagedAttention,KV	PagedAttention 到底解决了什么？	Q061	speculative-decoding,draft	Speculative Decoding 的基本原理是什么？
Q016	prefix-cache,cache	Prefix Caching 为什么有时收益巨大，有时几乎没用？	Q071	MoE,latency	为什么 MoE 理论 FLOPs 很低，线上 latency 却未必低？
Q021	continuous-batching,scheduler	Static Batching 和 Continuous Batching 有什么区别？	Q081	vLLM,architecture	请描述 vLLM 的核心架构思想。
Q024	chunked-prefill,Sarathi	Chunked Prefill 为什么能解决这个问题？	Q099	system-design,H100,70B	系统设计题：8×H100，部署一个 70B Chat Model，你怎么设计？
Q030	PD-disaggregation,DistServe	什么是 Prefill-Decode Disaggregation？	Q100	debugging,production,systematic	终极综合题：线上模型突然慢了 30%，你如何定位？

面试答题模板 | 从“会背”到“会做系统”

Step 1 | 先给一句结论 控制在 20-30 秒。例：“Decode 低 batch 通常偏 memory-bound，因为每 token 需要重新流式读取大权重，而权重复用不足。”

Step 2 | 写成本模型 至少给一个公式：KV、Roofline、通信、Goodput、成本中的任意一个。

Step 3 | 指出边界 使用“通常”“在低 batch/长 context/跨节点条件下”等限定语，避免绝对化。

Step 4 | 给工程证据 说明你会看哪些 profiler/metrics：TTFT/TPOT、HBM、Tensor Core、NCCL、cache hit、queue。

Step 5 | 主动给反例 说明该优化何时无效或负优化，这是高级候选人最明显的区分点。

术语表

缩写	解释
TTFT	Time To First Token, 首 token 延迟
TPOT	Time Per Output Token, 输出 token 平均时间
ITL	Inter-Token Latency, token 间延迟
Goodput	满足 SLO 的有效吞吐
KV Cache	保存历史 Key/Value, 避免自回归重复计算
PagedAttention	分页管理 KV Cache 的 serving 思想
GQA	Grouped-Query Attention, 多个 Q head 共享较少 KV head
MLA	Multi-head Latent Attention, 低维 latent KV 表示
TP/PP/DP	Tensor/Pipeline/Data Parallelism
EP	Expert Parallelism
CP	Context Parallelism
IFB	In-Flight Batching, 常用于描述动态/连续 batching
Spec Decode	Speculative Decoding, proposal + target verify
P/D	Prefill/Decode Disaggregation
HBM	High Bandwidth Memory
NCCL	NVIDIA Collective Communications Library

参考文献与官方文档

以下资料用于校验体系与版本信息。PDF 中的题目是对公开技术与工业实践的重新组织与解释，不复制论文或书籍原文。

- [1] vLLM Documentation. <https://docs.vllm.ai/en/stable/>
- [2] Kwon et al., Efficient Memory Management for LLM Serving with PagedAttention / vLLM. <https://arxiv.org/abs/2309.06180>
- [3] Yu et al., Orca: A Distributed Serving System for Transformer-Based Generative Models.
<https://www.usenix.org/conference/osdi22/presentation/yu>
- [4] Agrawal et al., Sarathi-Serve: Taming Throughput-Latency Tradeoff in LLM Inference.
<https://www.usenix.org/conference/osdi24/presentation/agrawal>
- [5] Zhong et al., DistServe: Disaggregating Prefill and Decoding for Goodput-optimized LLM Serving.
<https://www.usenix.org/conference/osdi24/presentation/zhong-yinmin>
- [6] Dao et al., FlashAttention. <https://arxiv.org/abs/2205.14135>
- [7] Shah et al., FlashAttention-3. <https://arxiv.org/abs/2407.08608>
- [8] Ye et al., FlashInfer, MLSys 2025.
https://proceedings.mlsys.org/paper_files/paper/2025/hash/dbf02b21d77409a2db30e56866a8ab3a-Abstract-Conference.html
- [9] Frantar et al., GPTQ. <https://arxiv.org/abs/2210.17323>
- [10] Lin et al., AWQ. <https://arxiv.org/abs/2306.00978>
- [11] Xiao et al., SmoothQuant. <https://arxiv.org/abs/2211.10438>
- [12] Cai et al., Medusa. <https://arxiv.org/abs/2401.10774>
- [13] Li et al., EAGLE-2. <https://arxiv.org/abs/2406.16858>
- [14] DeepSeek-AI, DeepSeek-V3 Technical Report. <https://arxiv.org/abs/2412.19437>
- [15] SGLang: Efficient Execution of Structured Language Model Programs. <https://arxiv.org/abs/2312.07104>
- [16] SGLang Documentation. <https://docs.sglang.ai/>
- [17] TensorRT-LLM Documentation. <https://nvidia.github.io/TensorRT-LLM/>
- [18] NVIDIA H100 Tensor Core GPU. <https://www.nvidia.com/en-us/data-center/h100/>

— END —

LLM Deployment & Inference Optimization Interview 100 · 2026 Edition